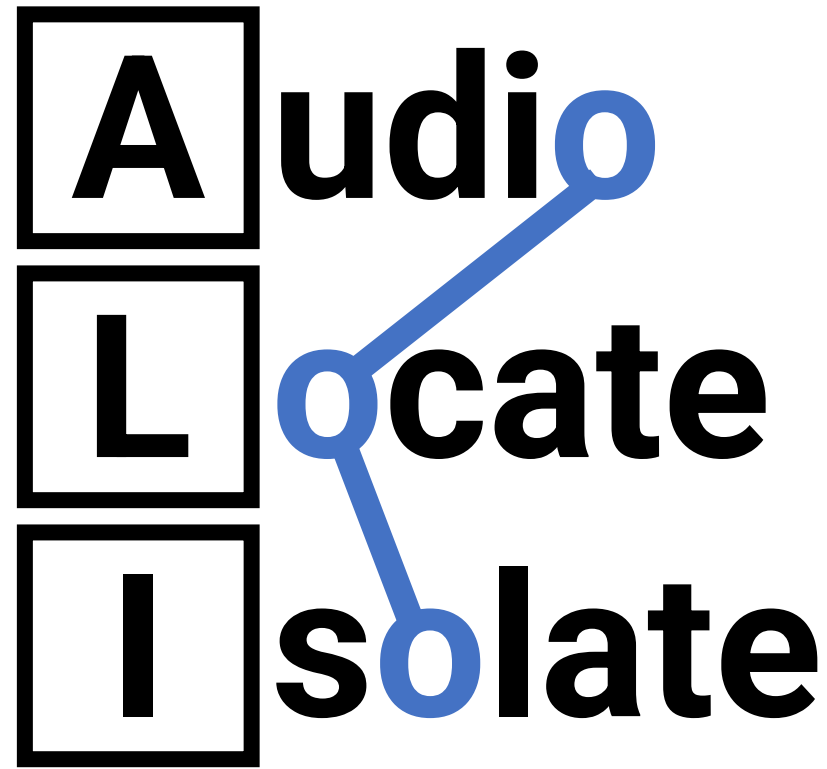


Audio
Locate
Isolate



TIPE | Louis H. | 00000

Introduction



Patrón cocktail bar, 2011, *Financial Times* (modifiée)

ÉTUDE THÉORIQUE DE LA LOCALISATION ET DE LA SÉPARATION INFORMATIQUE DE SOURCES SONORES.

Comment convertir des signaux sonores en informations de localisation et les transformer informatiquement pour isoler les sources sonores ?

Sommaire

- **Cadre de l'étude et stratégie**
- **Localisation d'une source sonore**
- **Isolation d'une source sonore**
- **Annexes**

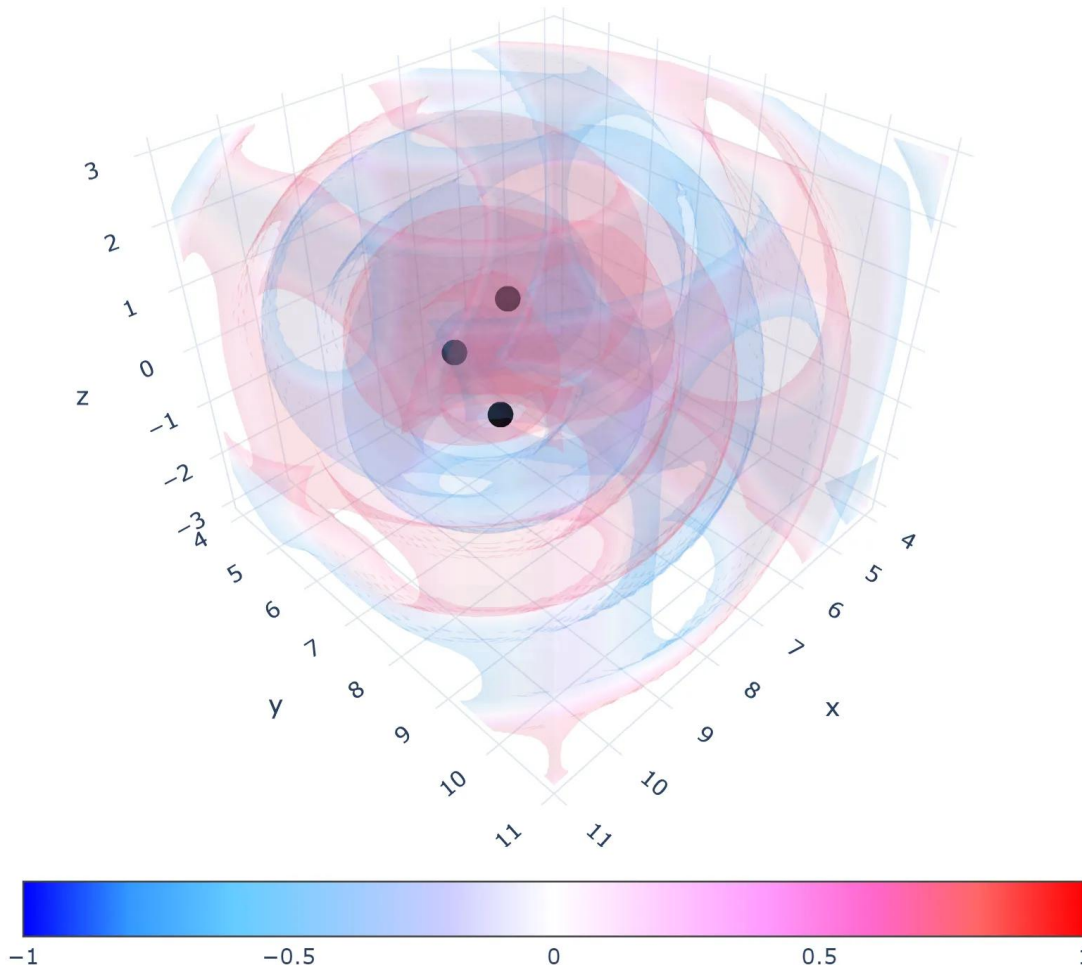


Cadre de l'étude

Simulation | Approximations | Stratégie

Environnement de simulation

Intensité sonore (normalisée) à $t = 0.5703s$



Propriétés

- Onde sphérique progressives
- Atténuation avec la distance
- Milieu :
 - Homogène ($T, P, \rho, c...$)
 - Isotrope
 - Sans réflexion

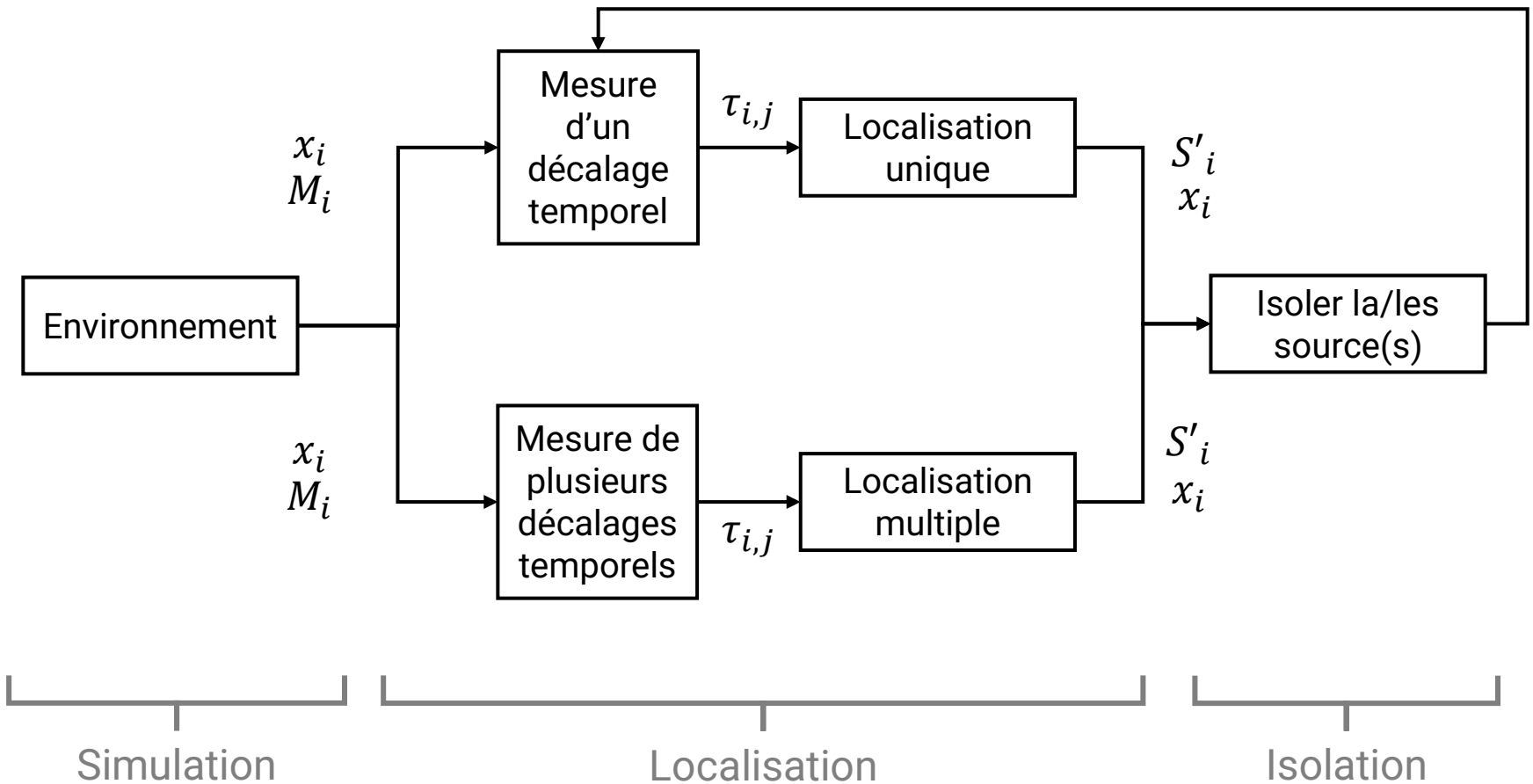
Approximations :

- Microphones idéaux
- Sources
 - fixes
 - indépendantes
 - idéaux

Contenu :

- Musiques variées

Plan de résolution

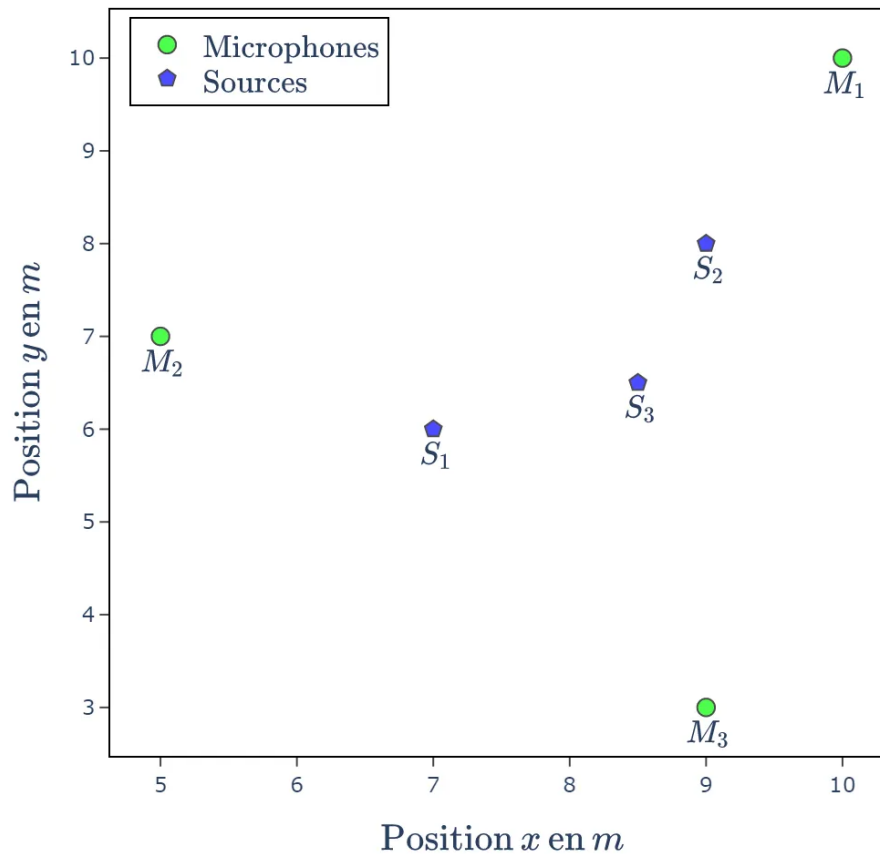


Localisation

Corrélation | Trilatération | Minimisation

Localisation

Exemple d'une configuration illustrative



Informations connues :

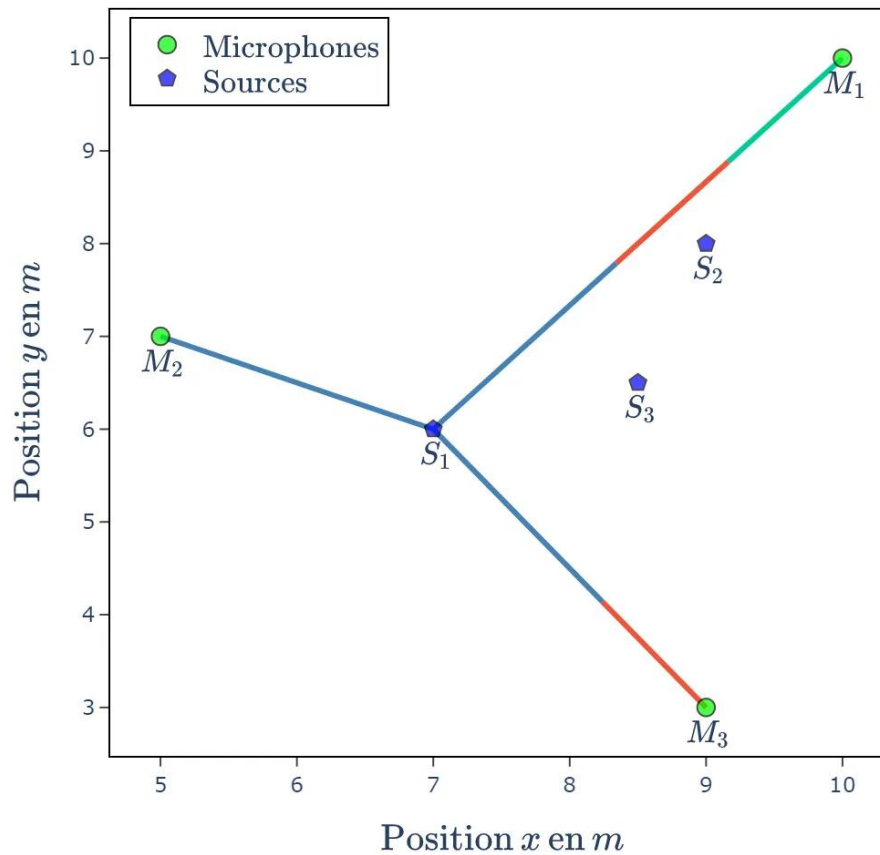
Positions (x, y) et enregistrements des microphones M_1, M_2 et M_3

Information à déterminer :

Le nombre de sources et leurs positions (x, y)

Localisation

Décalages temporels



Pour la source S_1

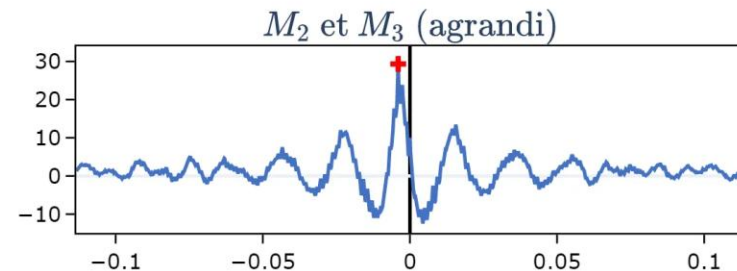
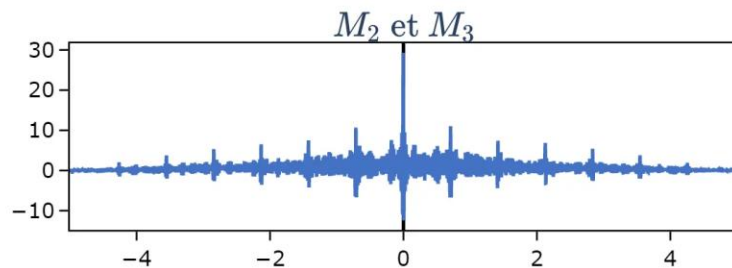
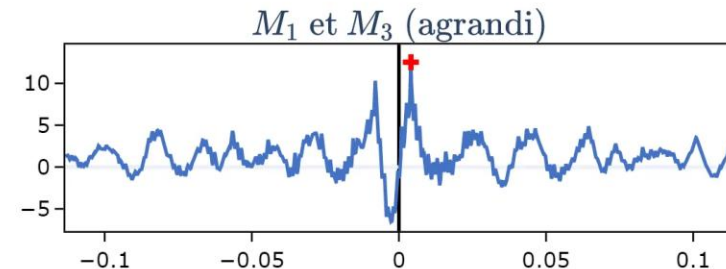
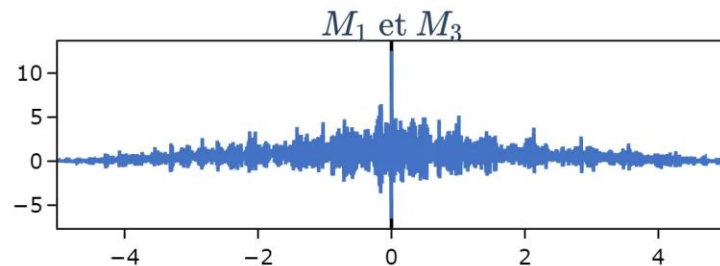
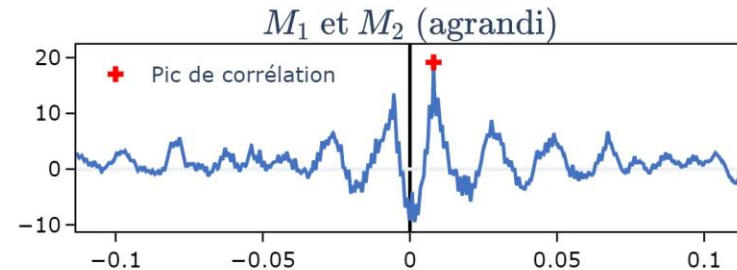
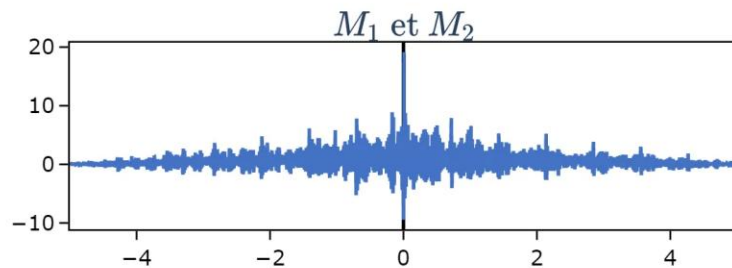
● : $c \cdot \tau_{2,3}$

● : $c \cdot \tau_{2,3}$

Corrélations croisées – Méthode *simple*

Déterminer le décalage temporel

Corrélations croisées entre les enregistrements des microphones



Temps en s

Temps en s

Corrélation croisée - Pics

Déterminer plusieurs décalages temporels

Méthode *pics*

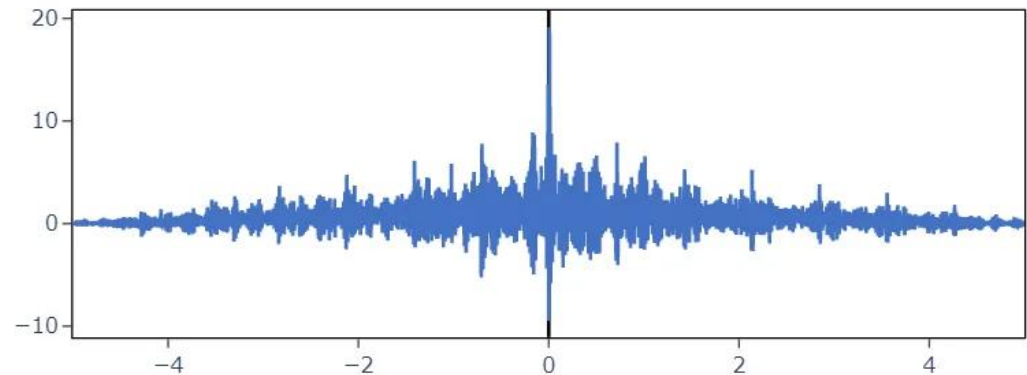
k pics de corrélation entre l'enregistrement de M_i et M_j :

$$T_{i,j} = \{\tau_1, \tau_2, \dots, \tau_k\}$$

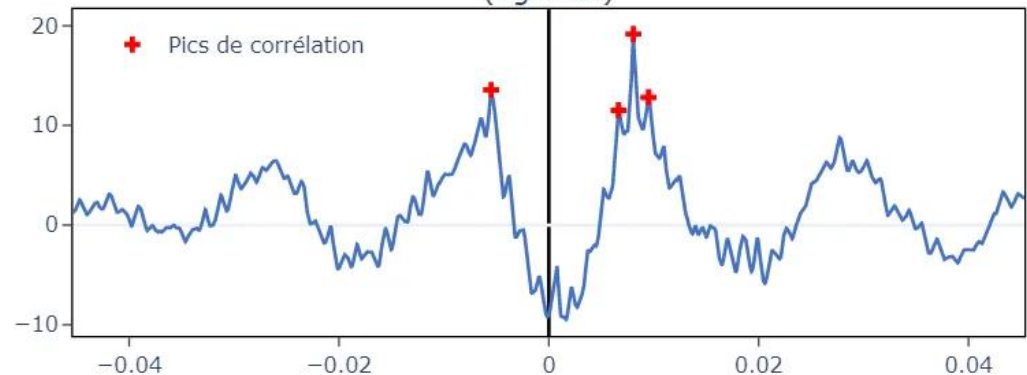
Ensemble de couples de décalages temporels candidats :

$$C = T_{1,2} \times T_{2,3} \times T_{3,1}$$

Corrélation croisée entre les enregistrements des microphones M_1 et M_2



(agrandi)



Temps en s

Corrélations croisées - Spectrogrammes

Déterminer plusieurs décalages temporels

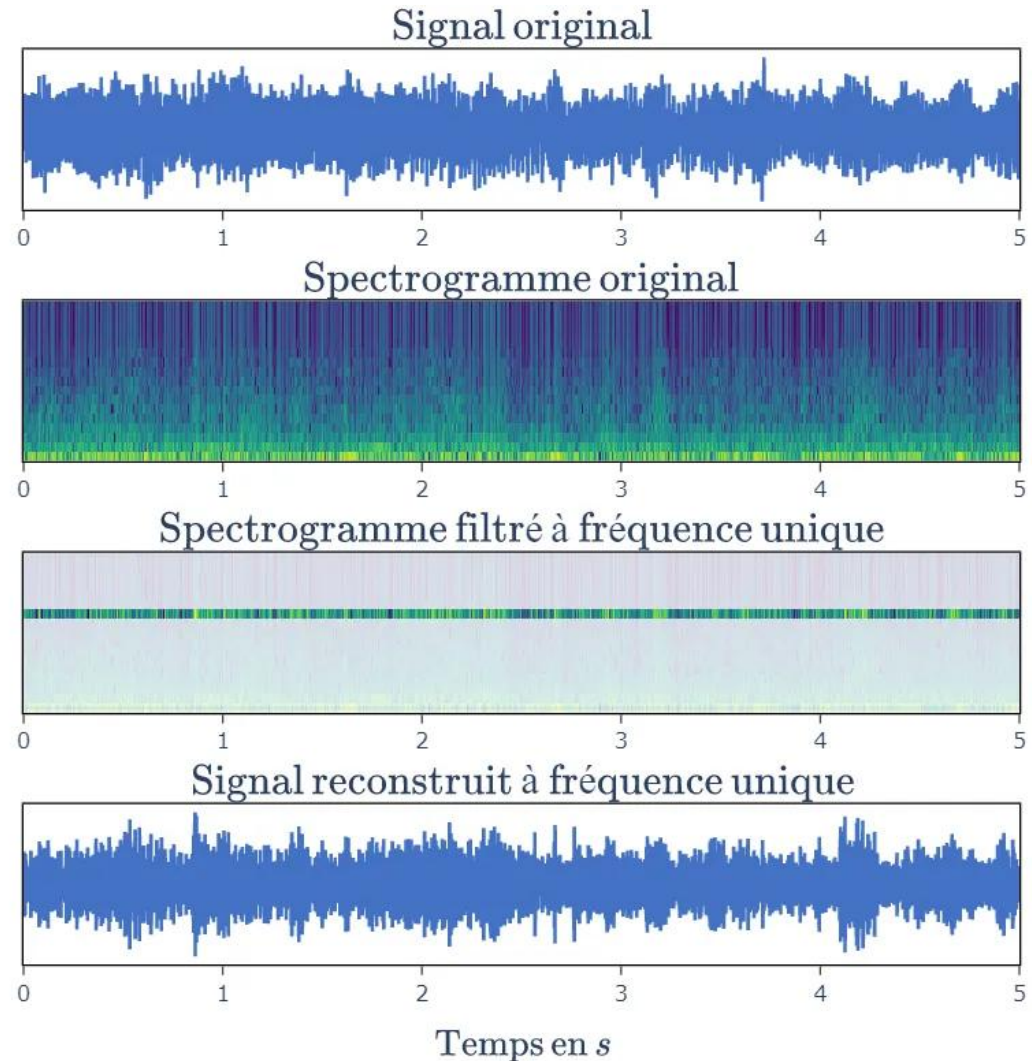
Méthode *spectrogramme*

Décalages temporels candidats pour un signal à fréquence unique f :

$$c_f = (\tau_{1,2}, \tau_{2,3}, \tau_{3,1})$$

Ensemble de décalages temporels candidats :

$$\mathcal{C} = \{c_f \mid f \in F\}$$



Recherche d'une source

Transformer les décalages temporels en coordonnées

Minimisation

Fonction à minimiser :

$$f: \mathbb{R}^n \rightarrow \mathbb{R}^+$$

$$X \mapsto \sum_{\substack{i,j \in \llbracket 1; n \rrbracket \\ i < j}} |d(M_j, X) - d(M_i, X) - c \cdot \tau_{i,j}|$$

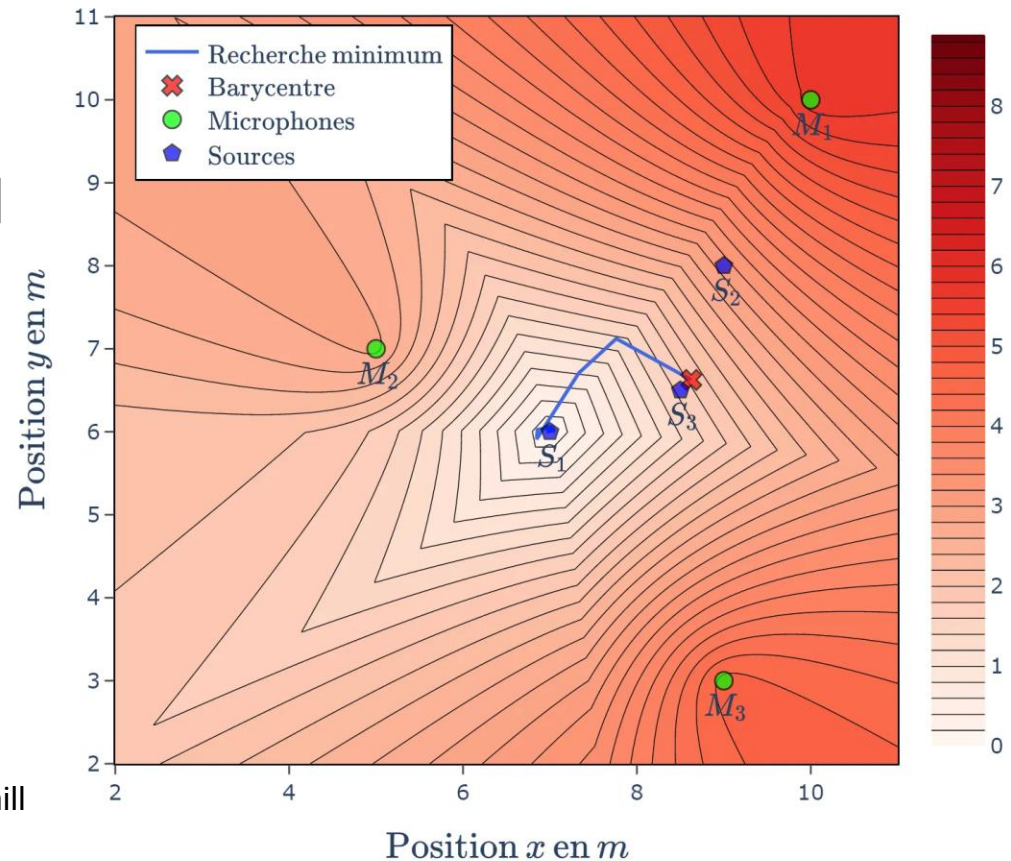
Caractéristiques de f :

f continue, non-dérivable sur $\mathbb{R}^n$
 f présente des « fond de vallée » [1]

L'algorithme de Nelder-Mead est donc adapté [1]

[2] Press, W. H. et al. (2007). "Section 10.5. Downhill Simplex Method in Multidimensions" *Numerical Recipes: The Art of Scientific Computing* (Annexe 1.2)

Erreurs à minimiser en m



Trilatération d'une source

Transformer les décalages temporels en coordonnées

Trilatération

Solution mathématique

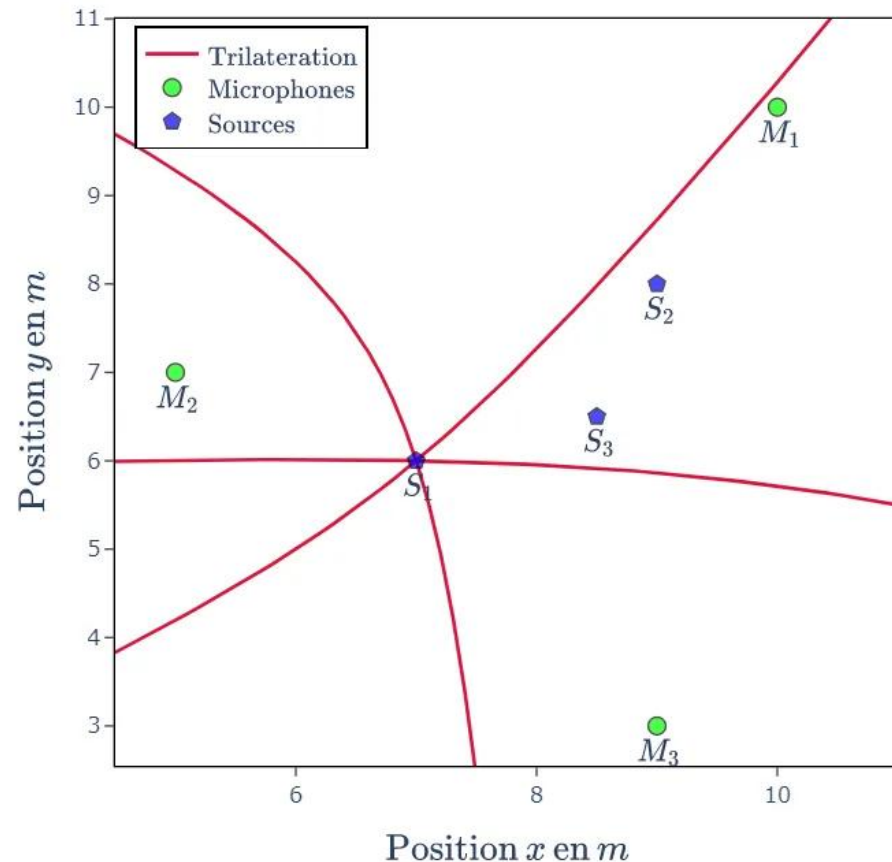
Equation cartésienne :

Coordonnées (x, y) possibles engendrant un décalage temporel $\tau_{i,j}$ entre M_i et M_j sont donné par :

$$d(M_i, (x, y)) - d(M_j, (x, y)) = c \cdot \tau_{i,j}$$

Problème majeur :

La solution peut être non définie en cas d'erreur sur $\tau_{i,j}$



Coordonnées candidates

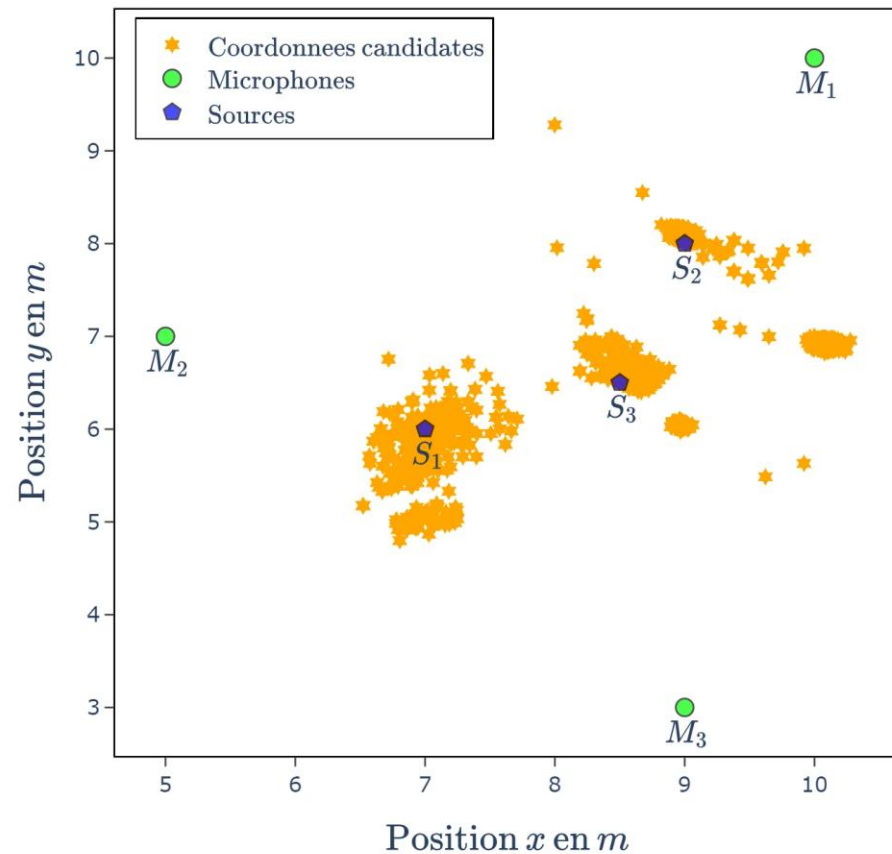
Transformer les candidats en coordonnées

Candidats pour localiser des sources

2000 candidats - Spectrogramme 128 + Pics 5 (Minimisation)

Résultats

- Coordonnées candidates(x, y) éliminées si $f(x, y) > \varepsilon$ ($\varepsilon = 1m$)
- Regroupements de candidats autour des sources



Partitionnement des candidats

Transformer les candidats en coordonnées

Regroupements candidats pour localiser des sources

2000 candidats - Spectrogramme 128 + Pics 5 (Minimisation)

Partitionnement

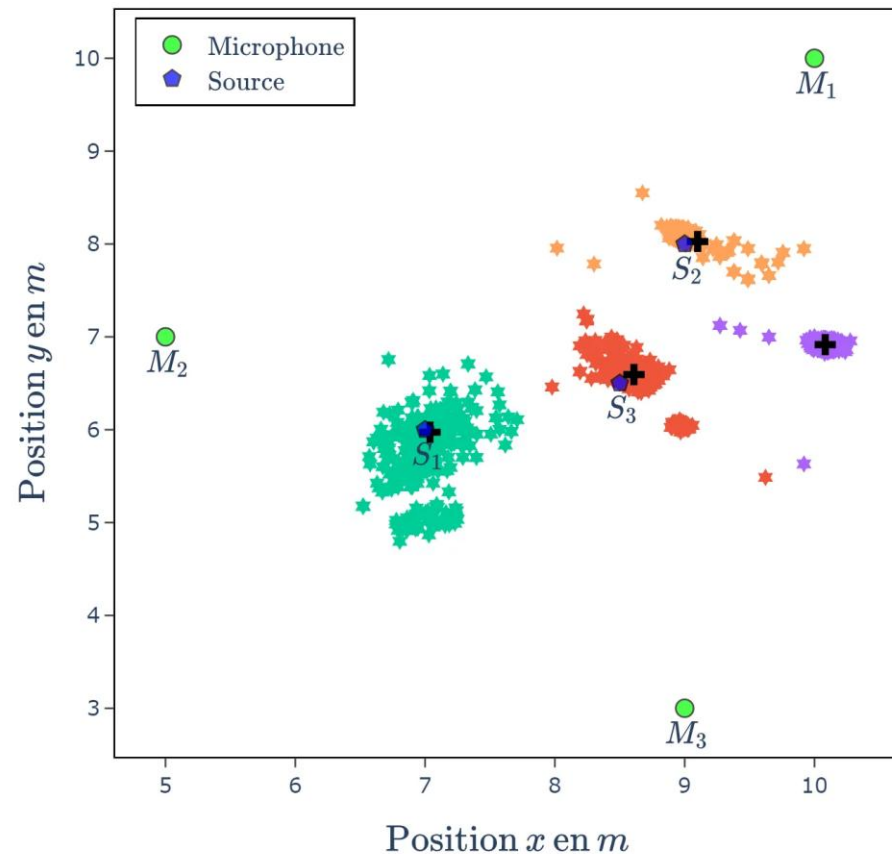
Etapes :

- Application de l'algorithme de partitionnement
- Calculer les barycentres
- Classer les barycentres

Choix de partitionnement :

- Nombre de sources inconnu
- Partitions caractérisées par leur densité

L'algorithme de MEAN-SHIFT est donc adapté^[2]

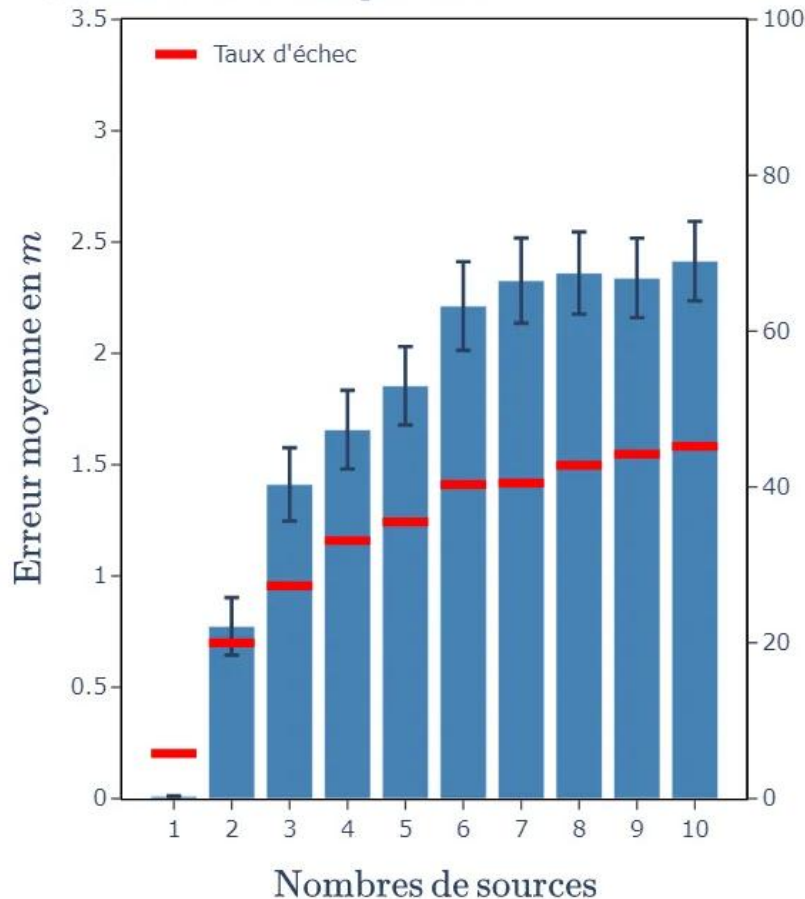


[3] Carreira-Perpiñán, Miguel. (2015). A review of mean-shift algorithms for clustering. (Annexe 1.3)

Comparaison méthodes *simples*

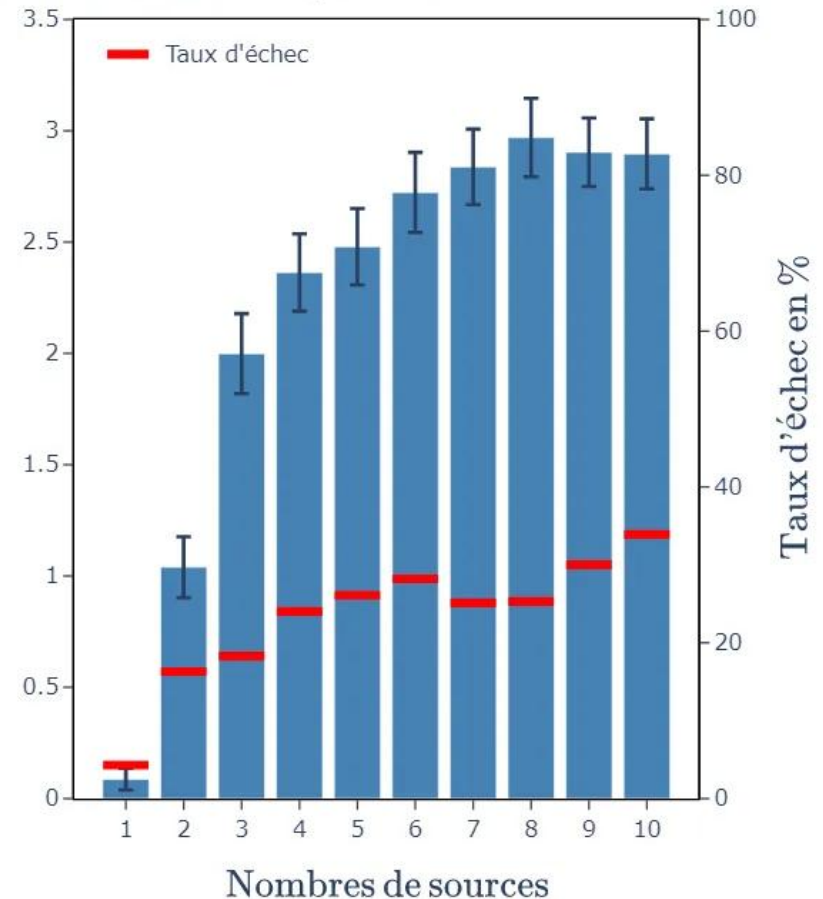
Erreur moyenne de la methode simple en fonction du nombre de sources (trilatération)

1000 mesures aléatoires par barre



Erreur moyenne de la methode simple en fonction du nombre de sources (minimisation)

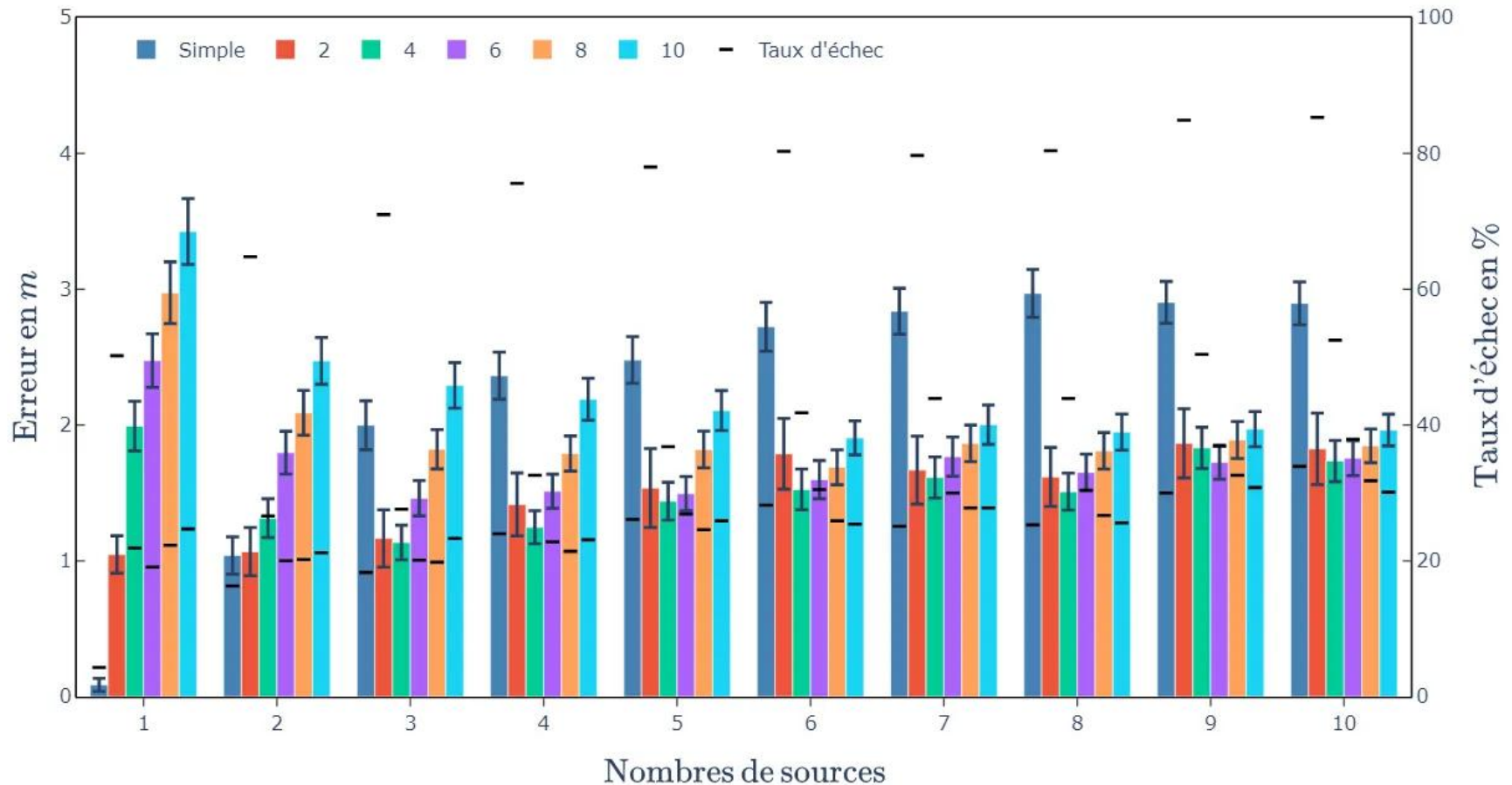
1000 mesures aléatoires par barre



Comparaison des méthodes *pics*

Erreur moyenne des méthodes pics en fonction du nombre de sources (optimisation)

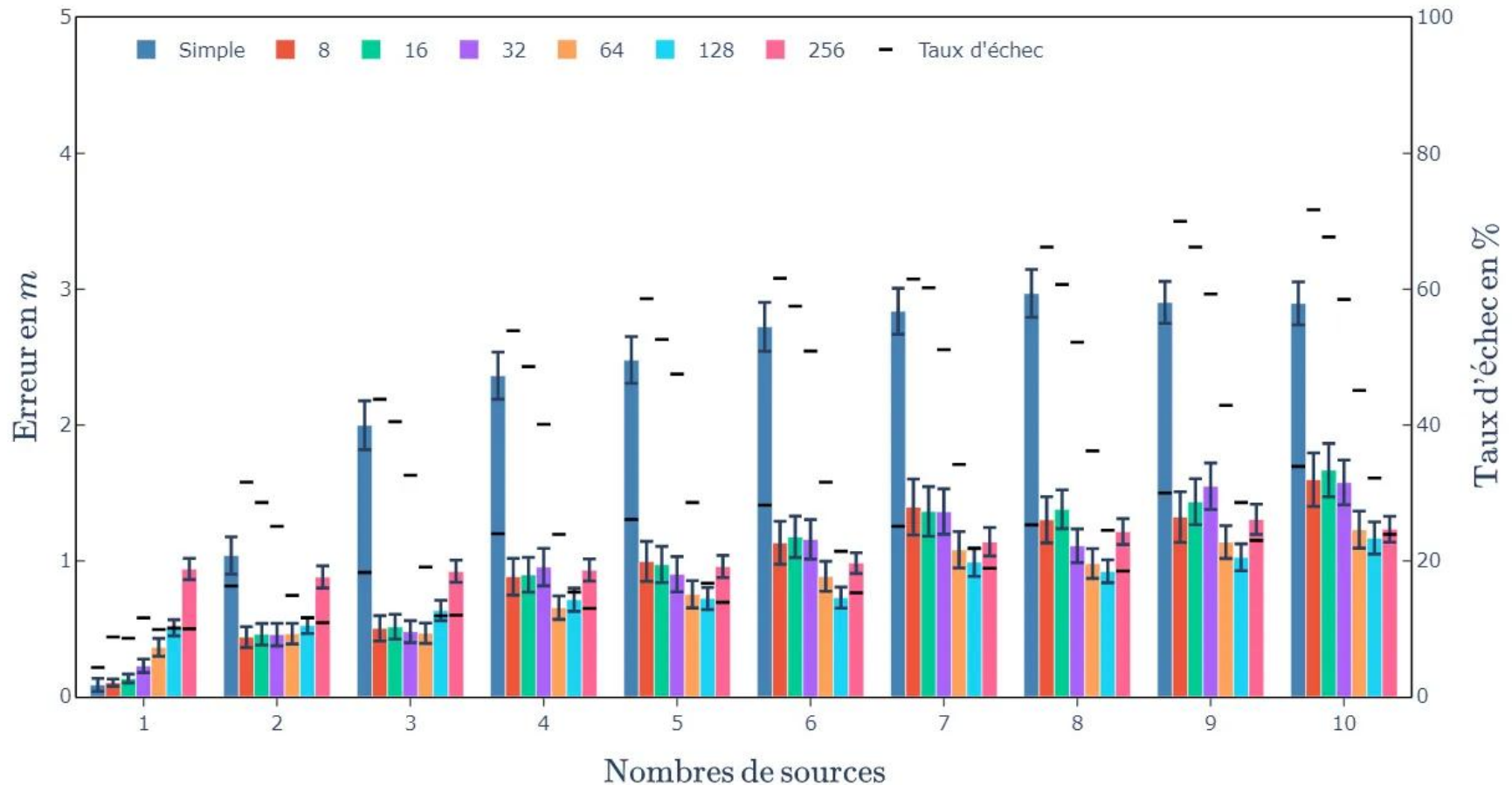
1000 mesures aléatoires par barre



Comparaison méthodes spectrogramme

Erreur moyenne des méthodes spectrogramme en fonction du nombre de sources (optimisation)

1000 mesures aléatoires par barre



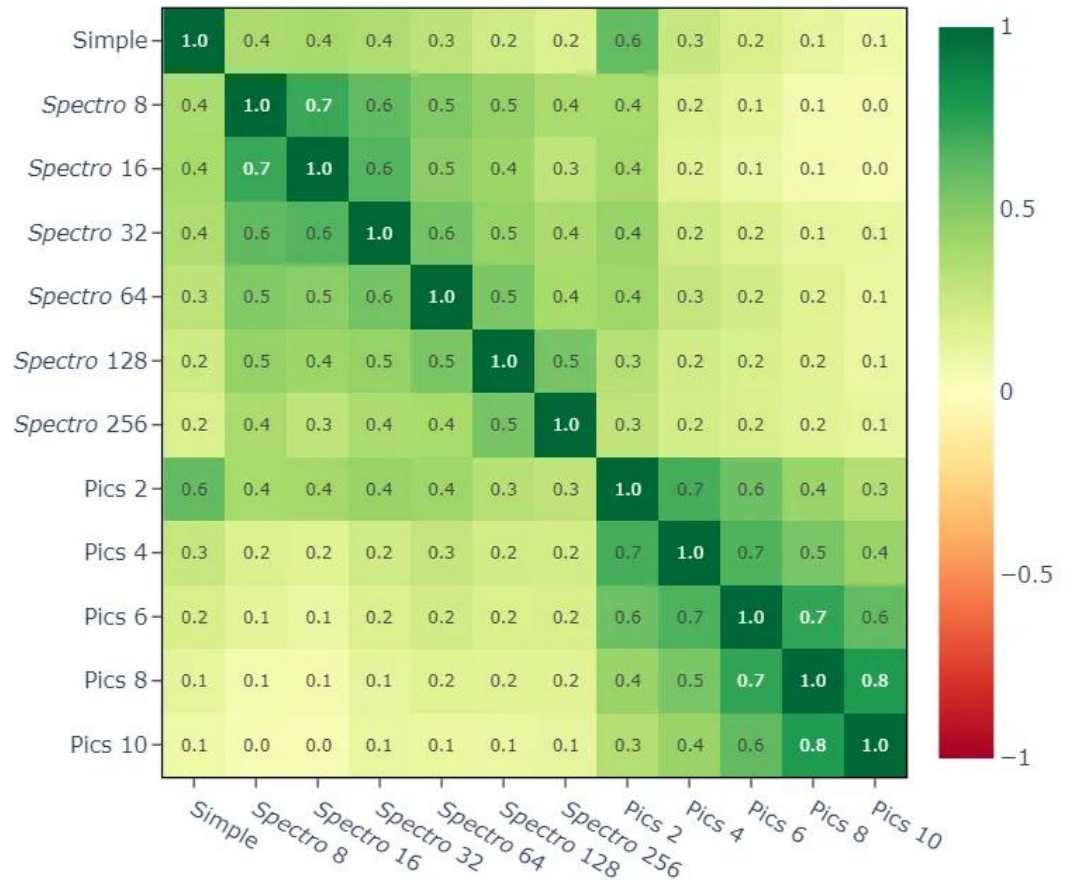
Premiers résultats | Corrélation des résultats



Méthode spectrogramme 128

S	Erreur en m	Erreur relative
1	$0,51 \pm 0,03 m$	$4,3\% \pm 0,3 \%$
2	$0,52 \pm 0,03 m$	$4,1\% \pm 0,3 \%$
3	$0,63 \pm 0,05 m$	$4,8\% \pm 0,4 \%$
4	$0,71 \pm 0,05 m$	$5,8\% \pm 0,4 \%$
5	$0,72 \pm 0,05 m$	$5,6\% \pm 0,4 \%$
6	$0,73 \pm 0,05 m$	$6,2\% \pm 0,5 \%$
7	$0,99 \pm 0,06 m$	$8,2\% \pm 0,5 \%$
8	$0,92 \pm 0,05 m$	$7,1\% \pm 0,4 \%$
9	$1,0 \pm 0,06 m$	$8,1\% \pm 0,6 \%$
10	$1,2 \pm 0,07 m$	$9,5\% \pm 0,6 \%$

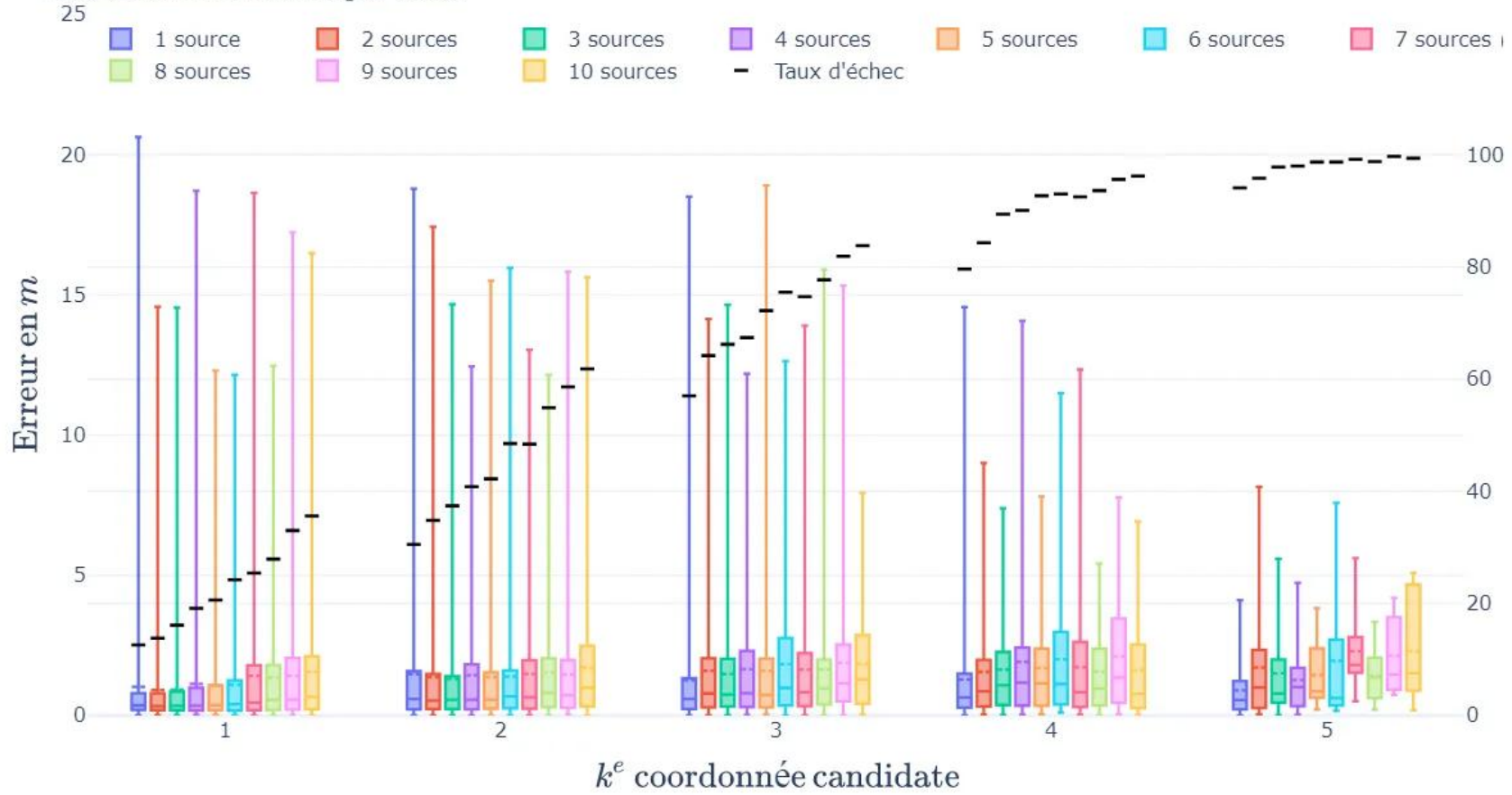
Matrice de corrélation des erreurs des différentes méthodes (par optimisation)



Résultats finaux

Erreur des candidats de la méthode spectrogramme 128
par rapport au nombre de sources présentes

1000 mesures aléatoires par boîte



Isolation

Séparation | Réseaux de neurones

Deux mesures de l'erreur

Mesure d'erreur

Erreur moyenne entre le signal original (x_i) de la source i et le signal reconstruit (x'_i) :

$$\varepsilon(x_i, x'_i) = \frac{1}{n} \sum_{k=1}^n |x_i(k, S) - x'_i(k, S)| \in \mathbb{R}^+$$

- Mesure « pure » d'erreur
- Ne prends pas en compte la réelle perception par l'humain du son
- Sera comparé à l'erreur d'un signal aléatoire de loi normale $\mathcal{N}(0, 1)$

Coefficient de corrélation de PEARSON

Corrélation entre le signal original (x_i) de la source i et le signal reconstruit (x'_i) :

$$\rho(x_i, x'_i) = \frac{\sum_{k=1}^n (x'_i(k, S) - \overline{x'_i}) (x_i(k, S) - \overline{x_i})}{\sigma_{x'_i} \cdot \sigma_{x_i}} \in [-1, 1]$$

- Prends plus en compte la réelle perception par l'humain du son

Séparation aveugle de sources

Méthodes classiques de résolution

Séparation de sources

Alignement des enregistrements

- Calcul de la distance entre la source et chaque microphone
- Réaligner les enregistrements pour les synchroniser
- Perte d'information sur les extrémités du signal

Séparation

Algorithmes usuels :

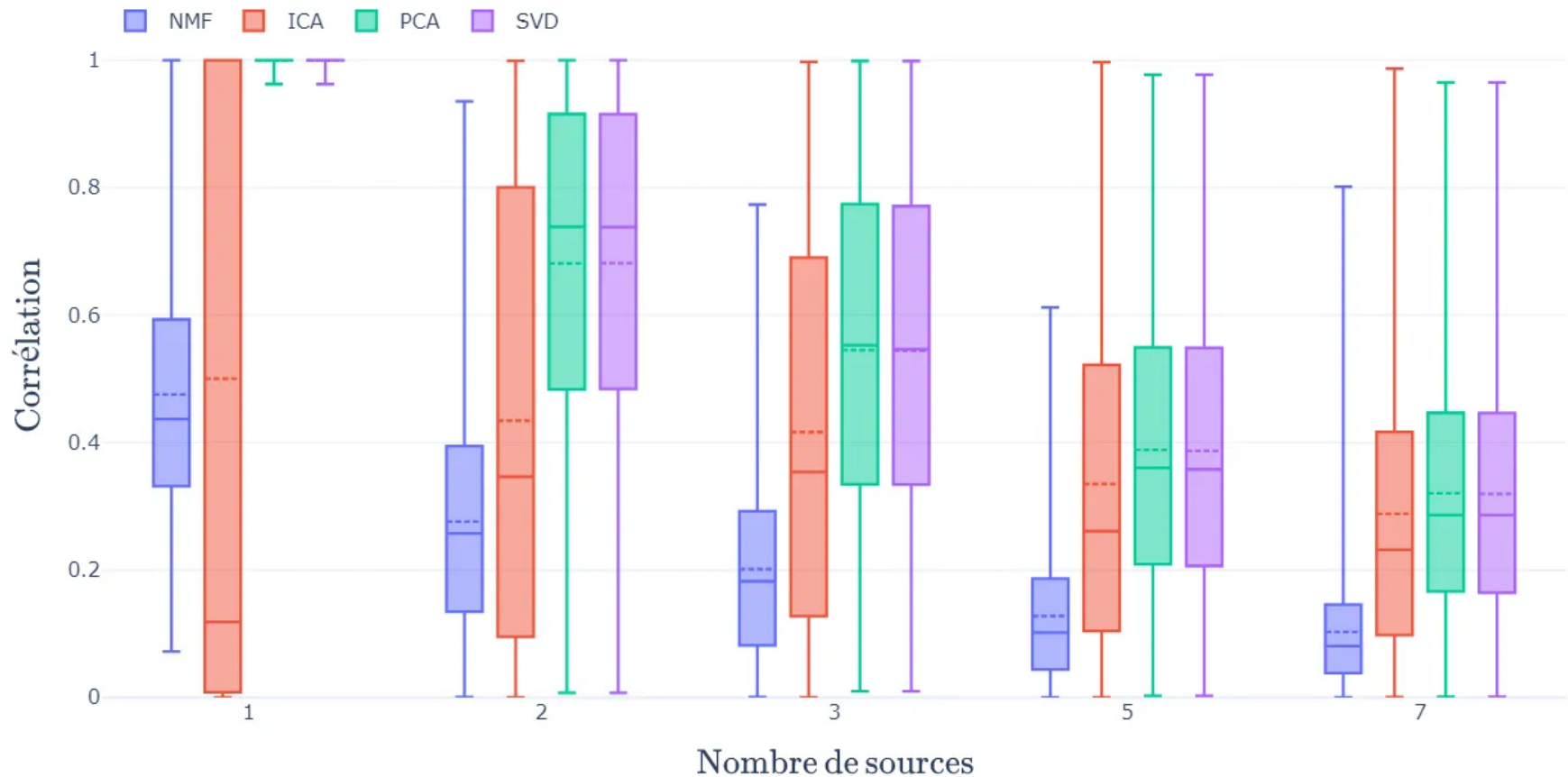
- Analyse en composantes principales (PCA)
- Décomposition en valeurs singulières (SVD)
- Analyse en composantes indépendantes (ICA)
- Factorisation par matrices non négatives (NMF)



Comparaison des algorithmes usuels

Corrélation entre le signal original et le signal isolé

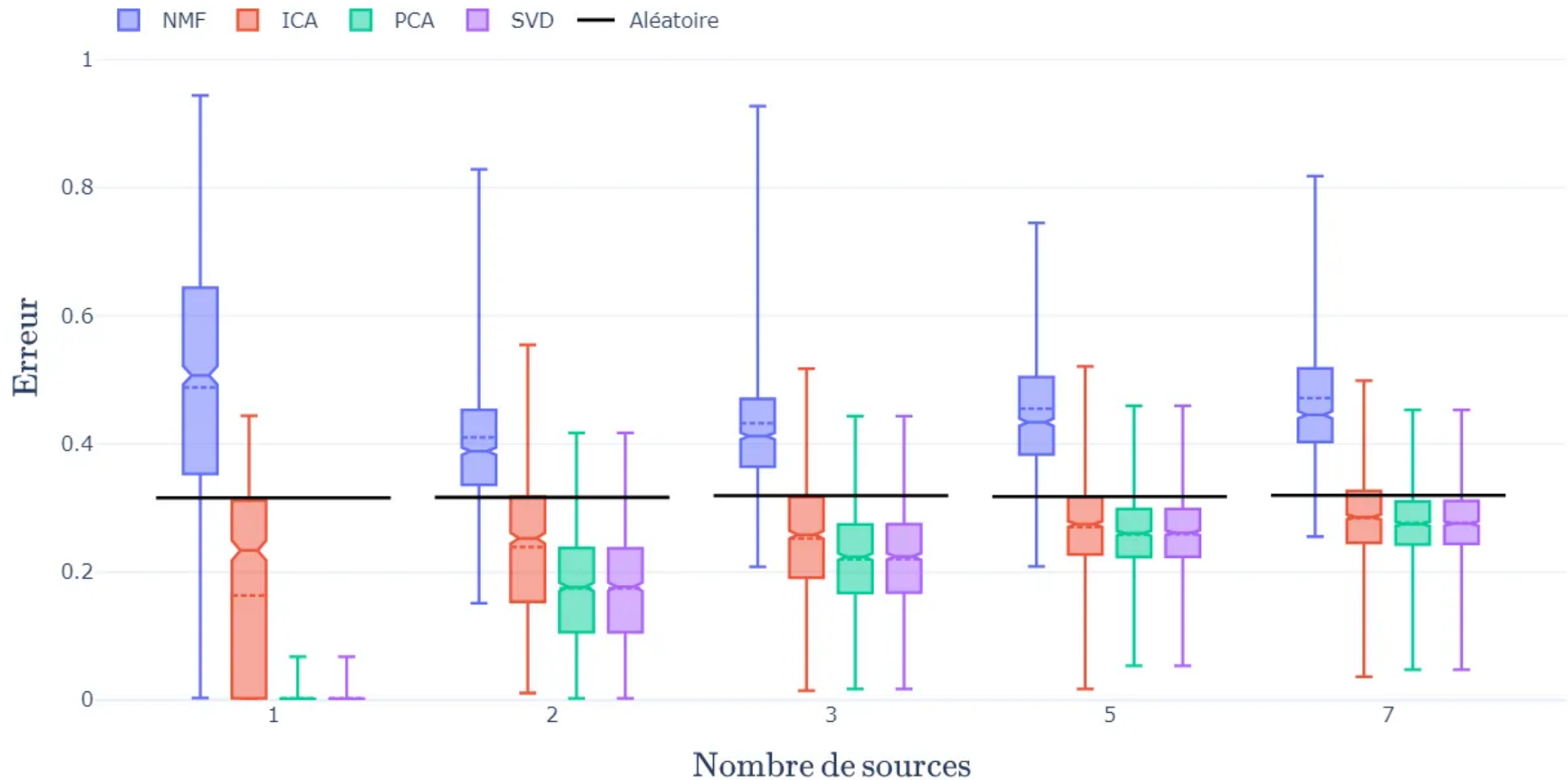
1000 mesures aléatoires par boîte



Comparaison des algorithmes usuels

Erreur entre le signal original et le signal isolé

1000 mesures aléatoires par boîte



Problématique

Améliorer la séparation

Séparation de sources

Problématiques

- Problème compliqué
- Sources parfois totalement inaudibles
- Mélanges non linéaires (déphasage)

Une solution

Réseau de neurone artificiel

- Pourrait imiter le comportement du cerveau humain
- Adapté pour cette tâche

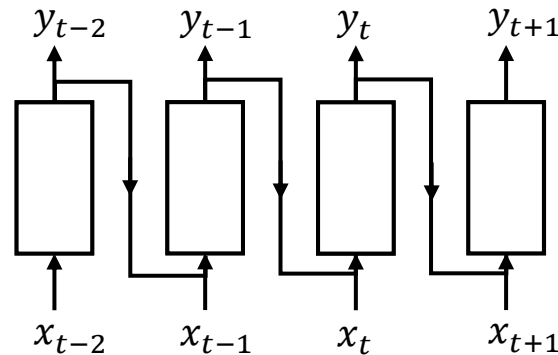


Réseau de neurones récurrent

Idée du réseau de neurones récurrent

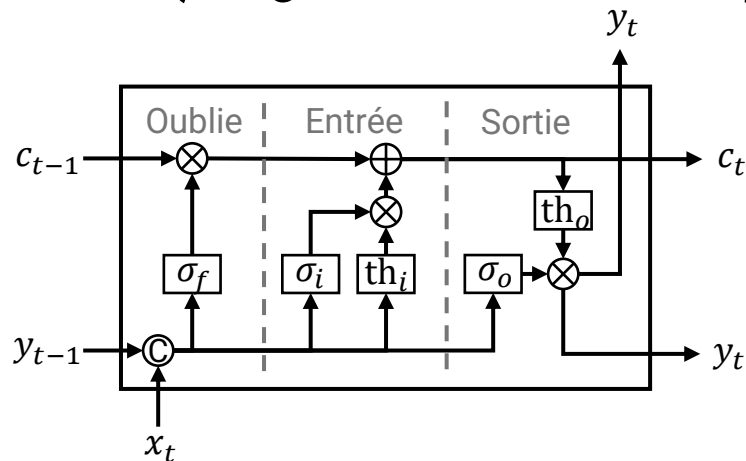


Réseau de neurones



Réseau de neurones récurrent

LSTM (*Long short-term memory*)



$$C: y_{t-1}, x_t \mapsto [y_{t-1}, x_t]$$

$$\sigma_a: X \mapsto \sigma(W_a \cdot X + b_a)$$

$$\text{th}_a: X \mapsto \tanh(W_a \cdot X + b_a)$$

Modèles et entraînement

Paramètres

Type :	Architecture (nombre d'unité)		Entrée :
- LSTM	Une couche :	Deux couches ^[3] :	- 3 signaux bruts
- GRU ^[3]	- 64	- 64 ; 64	- 3 spectrogrammes ^[4]
- LSTM Bidirectionnel	- 128	- 128 ; 128	
- GRU Bidirectionnel	- 256 ^[3]		

Entraînement

Configuration

- Erreur quadratique moyenne (MSE)
- *Adaptive Moment Estimation* (Adam)
- 150 heures de musiques, échantillonnage 44100Hz
- 12 jours d'entraînement^[5]

[4] Khandelwal, S., Lecouteux, B., Besacier., L. (2016). Comparing GRU and LSTM for automatic speech recognition. [Research Report] (4.3) (Annexe 1.4)

[5] Sören, B., Marcel, A., Sebastian, L., Klaus-Robert, M., Wojciech, S. (2018). Interpreting and Explaining Deep Neural Networks for Classification of Audio Signals. (Annexe 1.5)

[6] Emission : 913g eqCO₂ (39,4 kWh consommé 23,2g CO₂eq/kWh) source : electricitymaps.com (Annexe 1.6)

Résultats

Comparaisons des méthodes

Entrée :

Spectrogrammes
significativement moins
bons^[6]

Architecture :

Aucune différence significative.
Réseaux à 256 unités
légèrement meilleurs^[7]

Types :

Aucune différence significative.
GRU légèrement meilleur^[8]

Meilleur modèle

- GRU
- 256 unités récurrentes
- Signal brut

[7] Comparaison des types d'entrées (Annexe 1.7)

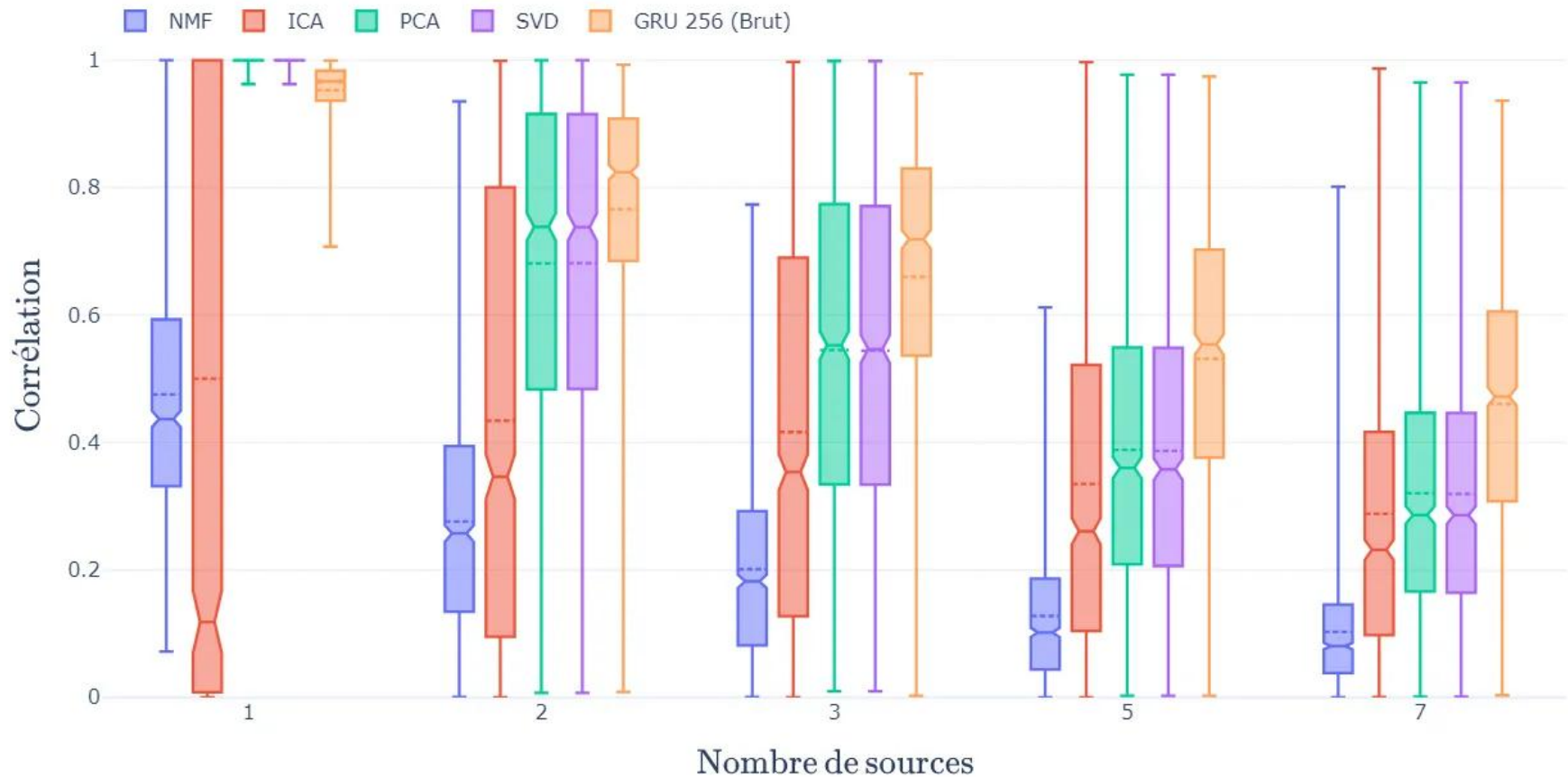
[5] Comparaison des architectures (Annexe 1.8)

[9] Comparaison des types de réseaux (Annexe 1.9)

Comparaison avec les algorithmes usuels

Corrélation entre le signal original et le signal isolé

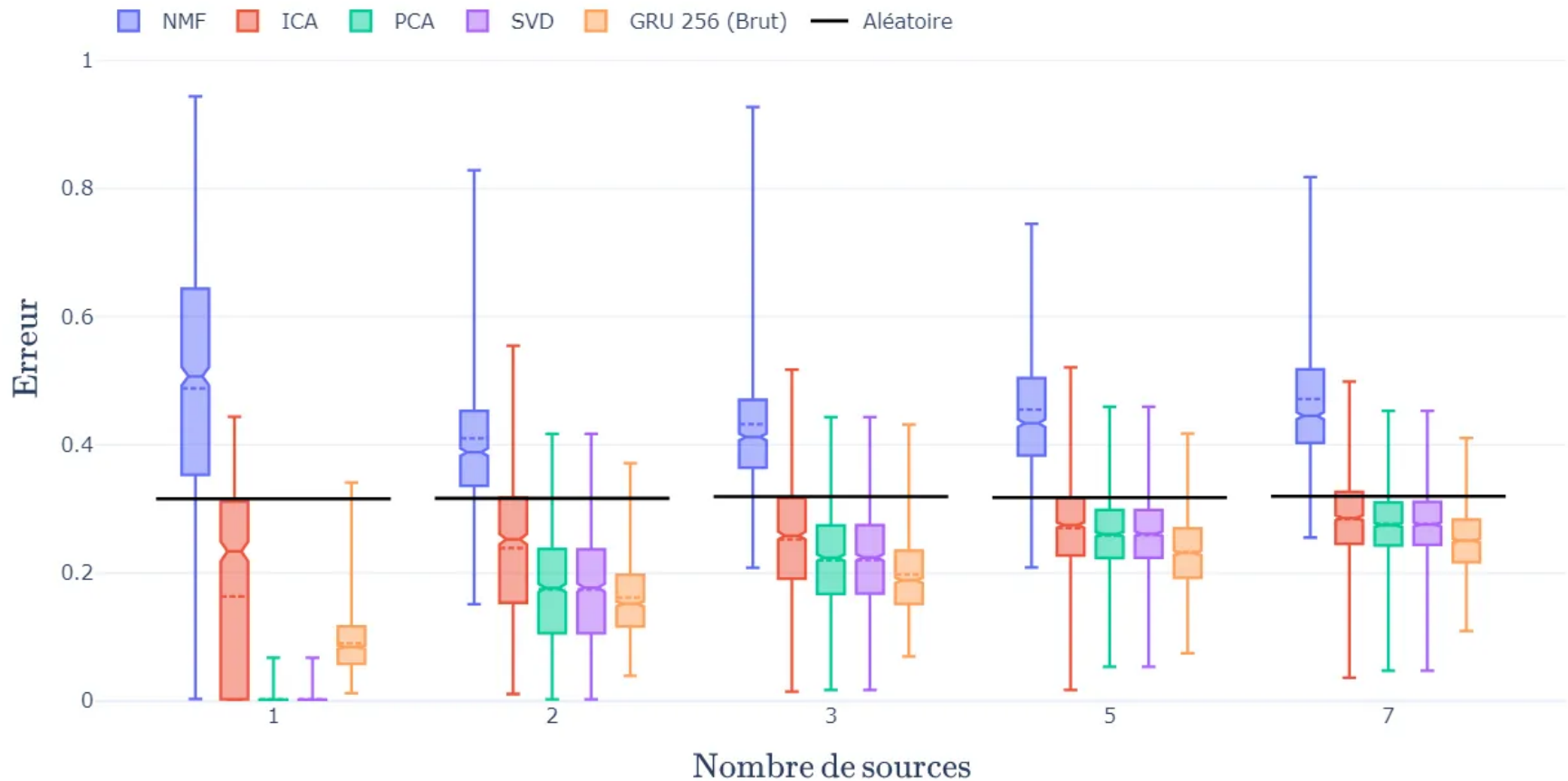
1000 mesures aléatoires par boîte



Comparaison avec les algorithmes usuels

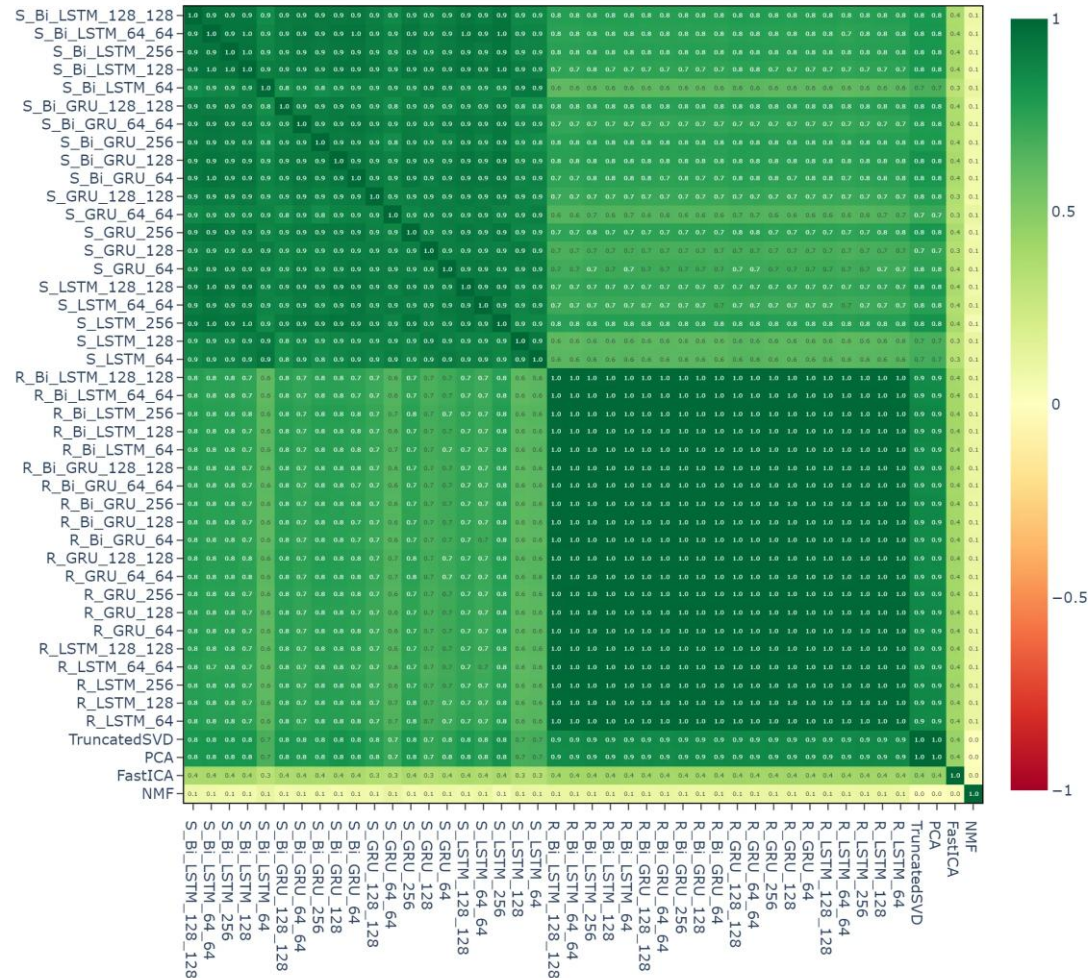
Erreur entre le signal original et le signal isolé

1000 mesures aléatoires par boîte



Matrice de corrélation des erreurs

Matrice de corrélation des erreurs des différents modèles

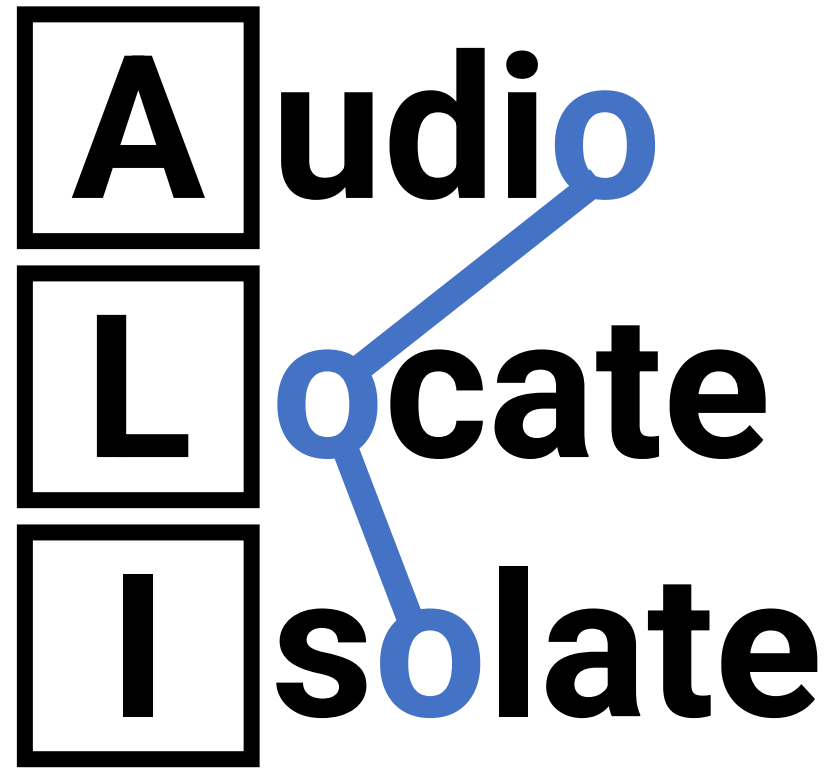


Conclusion

ÉTUDE THÉORIQUE DE LA LOCALISATION ET DE LA SÉPARATION INFORMATIQUE DE SOURCES SONORES.

Comment convertir des signaux sonores en informations de localisation et les transformer informatiquement pour isoler les sources sonores ?

Audio
Locate
Isolate



TIPE | Louis H. | 00000

Annexe 1

Détails | Données

Données

- 1 Atténuation sonore avec la distance
- 2 Choix de l'algorithme NELDER-MEAD
- 3 Choix de l'algorithme de MEAN-SHIFT
- 4 Performances des GRU
- 5 Choix signal brut ou spectrogramme
- 6 Emission de CO₂
- 7 Comparaison des types d'entrées
- 8 Comparaison architecture
- 9 Comparaison type
- 10 Taille des données analysées

Diapositive

- 41
- 42
- 43
- 44
- 45
- 46
- 47
- 49
- 51
- 53



Annexe 2

Code source | Python

Code source

	Diapositive	Listing
- audio_locate_isolate		
- isolate.py	56	13
- locate.py	66	19
- neural_network.py	83	30
- utils.py	107	46
- chart		
- 0_presentation.ipynb	113	49
- 3_illustrate_locate.ipynb	117	51
- 4_test_locate.ipynb	151	74
- 4_test_locate_stats.py	164	86
- 5_test_isolate.ipynb	182	95
- 5_test_isolate.py	200	107
- charts_utils.py	209	113
- environnement.py	211	114

Annexe 3

Tables des valeurs

Tables de résultats

- 18
- 19
- 20
- 21
- 22
- 32-33

Diapositive

- 228
- 229
- 230
- 231
- 233
- 234



Annexe 1

Détails | Données



Atténuation sonore avec la distance

Calcul de coefficient d'atténuation

On a la puissance sonore $P(r) = \frac{4\pi r^2}{\rho c T} \int_0^T p(t, r)^2 dt$ en W avec p la pression en fonction du temps

Or : $\forall r > r_S, P(r) = P(\text{conservation de la puissance sonore}), \text{ donc } P(r_S) = P(r)$

On pose : $p(t, r) = \alpha(r) \cdot p(t, r_S)$ avec α le facteur d'atténuation avec la distance

$$\text{Donc : } P(r) = \frac{r^2}{r_S^2} \alpha^2(r) P(r_S)$$

$$\text{Donc : } \boxed{\alpha(r) = \frac{r_S}{r}}$$

On en déduit que : $s(t, r_S) = \frac{p(t, r_S)}{p_m}$ avec p_m la pression maximale mesurée par le microphone.

$$\text{Donc : } \boxed{s(t, r) = \frac{r_S}{r} \cdot \frac{p(t, r_S)}{p_m}}$$

Ce résultat est utilisé dans `environnement.py` dans la fonction `distance_gain`

La pression maximale se base sur celle d'un microphone Blue Yeti, $p_m = 20\text{Pa}$ ^[10]

[14] Source : logitechg.com (Max SPL: 120 dB)

Choix de l'algorithme de NELDER-MEAD

Une explication intuitive de l'algorithme de NELDER-MEAD tirée de *Numerical Recipes* ^[1]
(Traduit) :

La méthode du simplexe (ou simplexe de NELDER-MEAD) effectue alors une série d'étapes, la plupart consistant simplement à déplacer le point du simplexe où la fonction atteint sa valeur maximale (« point le plus haut ») à travers la face opposée du simplexe, vers un point plus bas. Ces étapes sont appelées réflexions, et elles sont conçues pour conserver le volume du simplexe (et donc préserver sa non-dégénérescence). Lorsque cela est possible, la méthode étend le simplexe dans une direction ou une autre afin de faire de plus grands pas. **Lorsqu'elle atteint le fond d'une vallée, la méthode se contracte dans la direction transverse et tente de s'insinuer le long de la vallée.** Dans les situations où le simplexe tente de « passer par le chas d'une aiguille », il se contracte dans toutes les directions, se resserrant autour de son point le plus bas (le meilleur).

[2] Press, W. H. et al. (2007). "Section 10.5. Downhill Simplex Method in Multidimensions" *Numerical Recipes: The Art of Scientific Computing*, ISBN 978-0-521-88068-8.

Choix de l'algorithme de MEAN-SHIFT

Avantages et inconvénients l'algorithme MEAN-SHIFT tirée de “**A review of mean-shift algorithms for clustering**”^[2] (Traduit) :

Les avantages des algorithmes mean-shift proviennent de la nature non paramétrique de l'estimation de densité par noyau (KDE) :

- (1) Ils ne reposent sur aucune hypothèse de modèle (à part l'utilisation d'un noyau spécifique), contrairement, par exemple, aux modèles de mélanges gaussiens ou à K-means.
- (2) Ils sont capables de modéliser des groupes complexes ayant une forme non convexe (ce qui ne signifie pas que toutes les formes sont bien modélisées), contrairement à K-means.
- (3) L'utilisateur n'a qu'un seul paramètre à régler, la bande passante (bandwidth), qui a une signification physique intuitive correspondant à une échelle locale, et ce paramètre détermine automatiquement le nombre de clusters. Cela est souvent plus pratique que de devoir spécifier explicitement le nombre de groupes.
- (4) L'algorithme ne présente pas de minimum local, ce qui signifie que le regroupement obtenu est uniquement déterminé par la bande passante, sans besoin d'exécuter l'algorithme plusieurs fois avec différentes initialisations.
- (5) Les valeurs aberrantes (outliers), qui peuvent poser de gros problèmes pour les mélanges gaussiens et K-means, n'affectent pas excessivement la KDE (à part éventuellement en créant des groupes isolés composés d'un seul point).

[3] Carreira-Perpiñán, Miguel. (2015). A review of mean-shift algorithms for clustering.

Performances des GRU

Performances des GRU sur un problème de reconnaissance vocal tiré de “**Comparing GRU and LSTM for automatic speech recognition.**”^[3] (Traduit) :

Des expériences préliminaires ont été menées pour comparer de petits modèles LSTM et GRU (avec un nombre de paramètres similaire ou inférieur à celui de notre modèle de référence DNN-512-3).

Les résultats expérimentaux obtenus à l’aide d’un modèle de langage trigramme sont présentés dans le tableau 3. Ces résultats montrent que le GRU donne de meilleures performances que le LSTM pour un nombre équivalent de paramètres (le modèle BGRU512-1 surpasse le modèle BLSTM128-1, alors que les deux ont le même nombre de paramètres). On observe également que les GRU plus profonds donnent de meilleurs résultats. Le modèle GRU256-5, qui ne comporte que 1,8 million de paramètres, atteint un taux d’erreur de mots (WER) de 21,32 %, ce qui surpasse le modèle de référence DNN-512-3 (4,8 millions de paramètres, WER = 22,98 %) ainsi que le LSTM ayant une topologie similaire (LSTM256-5, 3 millions de paramètres, WER = 22,98 %).

[4] Khandelwal, S., Lecouteux, B., Besacier., L. (2016). Comparing GRU and LSTM for automatic speech recognition. [Research Report].

Choix signal brut ou spectrogramme

Choix signal brut ou spectrogramme pour des tâches de classification tiré de “**Interpreting and Explaining Deep Neural Networks for Classification of Audio Signals.**”^[4] (Traduit) :

Abstract (extrait)

Dans cet article, deux architectures de réseaux neuronaux sont entraînées sur des données de spectrogrammes et de formes d’onde brutes pour des tâches de classification audio, à partir d’un nouveau jeu de données audio.

Conclusion (extrait)

La classification à partir des formes d’onde brutes a montré que le réseau prend sa décision en se basant sur une fraction relativement petite d’échantillons très pertinents. Une explication possible de cet effet [...] pourrait être que le réseau se concentre principalement sur la forme « globale » du signal en entrée. En effet, des échantillons sélectionnés aléatoirement sont répartis de manière uniforme sur la durée du signal, de sorte que – tant que la proportion d’échantillons manipulés n’est pas trop importante – il reste, dans chaque voisinage local du signal, des échantillons avec l’amplitude d’origine, ce qui permet de conserver la forme initiale du signal. En revanche, une sélection basée sur l’amplitude ou sur la méthode LRP (Layer-wise Relevance Propagation) pourrait altérer le signal d’une manière telle que la forme globale ne soit plus reconnaissable.

[5] Sören, B., Marcel, A., Sebastian, L., Klaus-Robert, M., Wojciech, S. (2018). Interpreting and Explaining Deep Neural Networks for Classification of Audio Signals.

Emissions de CO₂ dues à l'entraînement



AMD Ryzen 5 2600 (80W^[10])

Nvidia GTX 1070 (150W^[11])

Carte mère, RAM, SSD : (70W^[12])

g eqCO ₂ /KWh ^[5]	Date de début	Date de fin	Heures	kWh	g CO ₂ total
19	25/10/2024 11:30	25/10/2024 21:30	10	3	57
21	26/10/2024 11:30	26/10/2024 21:30	10	3	63
21	27/10/2024 10:00	27/10/2024 22:00	12	3,6	75,6
21	28/10/2024 06:45	28/10/2024 22:00	15,25	4,575	96,075
21	29/10/2024 09:00	29/10/2024 22:00	13	3,9	81,9
21	30/10/2024 09:00	30/10/2024 21:00	12	3,6	75,6
21	31/10/2024 09:00	31/10/2024 23:00	14	4,2	88,2
21	01/11/2024 11:00	01/11/2024 21:00	10	3	63
23	02/11/2024 10:00	02/11/2024 21:00	11	3,3	75,9
33	16/11/2024 10:00	16/11/2024 22:00	12	3,6	118,8
33	17/11/2024 10:00	17/11/2024 22:00	12	3,6	118,8
Total			131,25	39,375	913,875

Soit 537km en TGV pour une personne (1,7g/km^[13])

[6] Source : electricitymaps.com

[10] Source : amd.com

[11] Mesure maximale réalisée

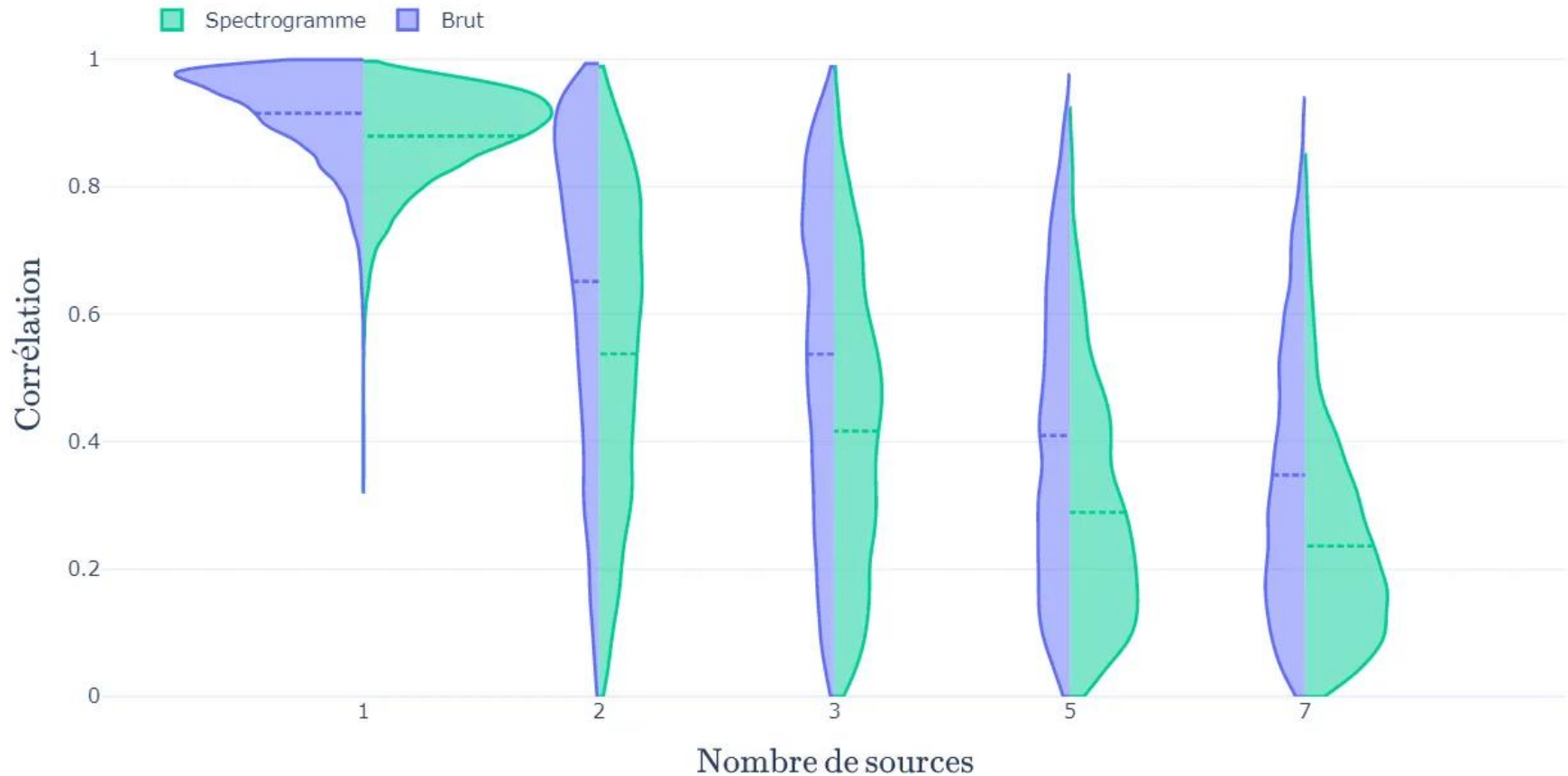
[12] Source : outervision.com

[13] Source : ressources.data.sncf.com

Comparaison des types d'entrées

Corrélation entre le signal original et le signal isolé

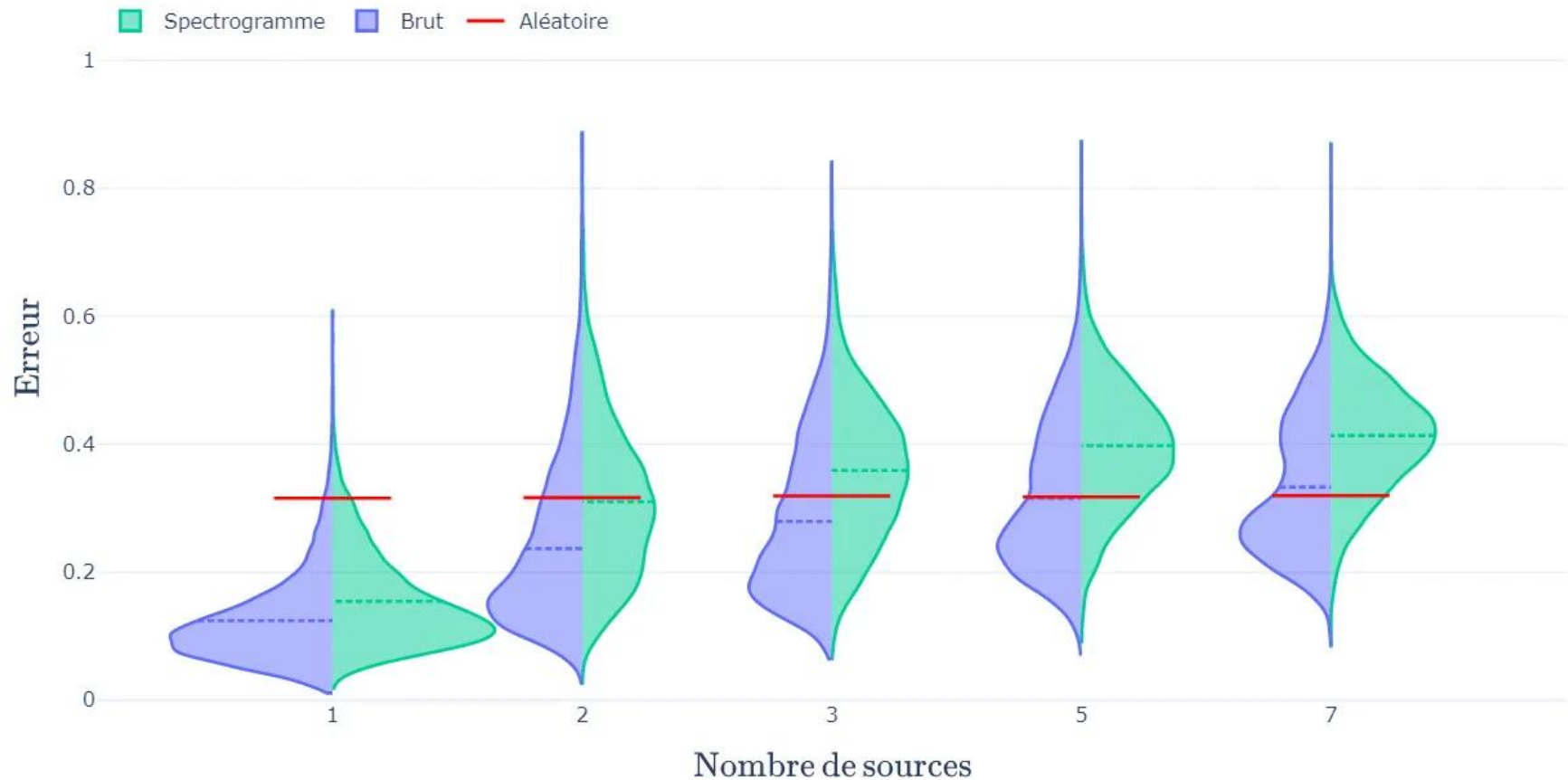
1000 mesures aléatoires par violon



Comparaison des types d'entrées

Erreur entre le signal original et le signal isolé

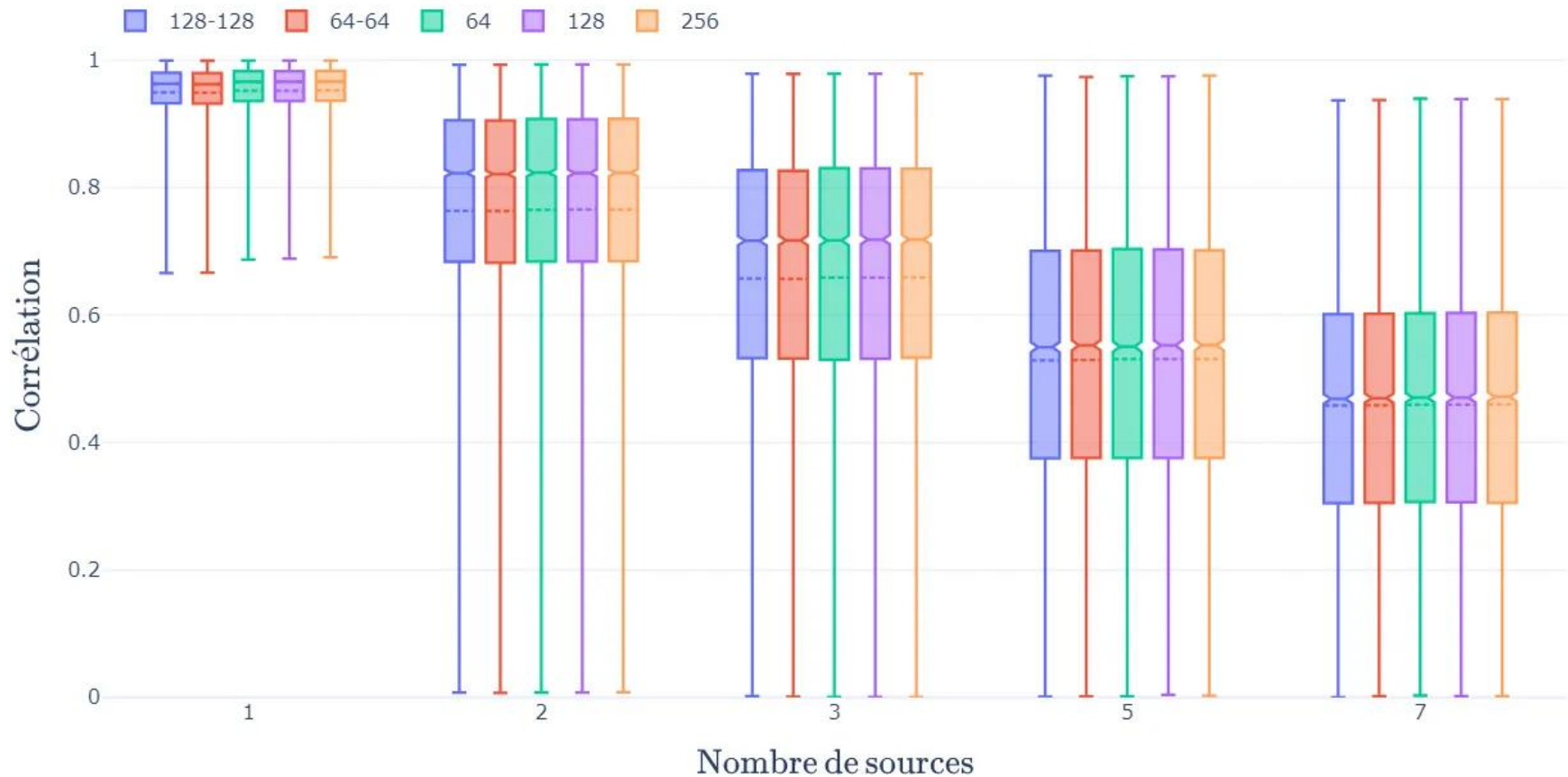
1000 mesures aléatoires par violon



Comparaison des architectures

Corrélation entre le signal original et le signal isolé

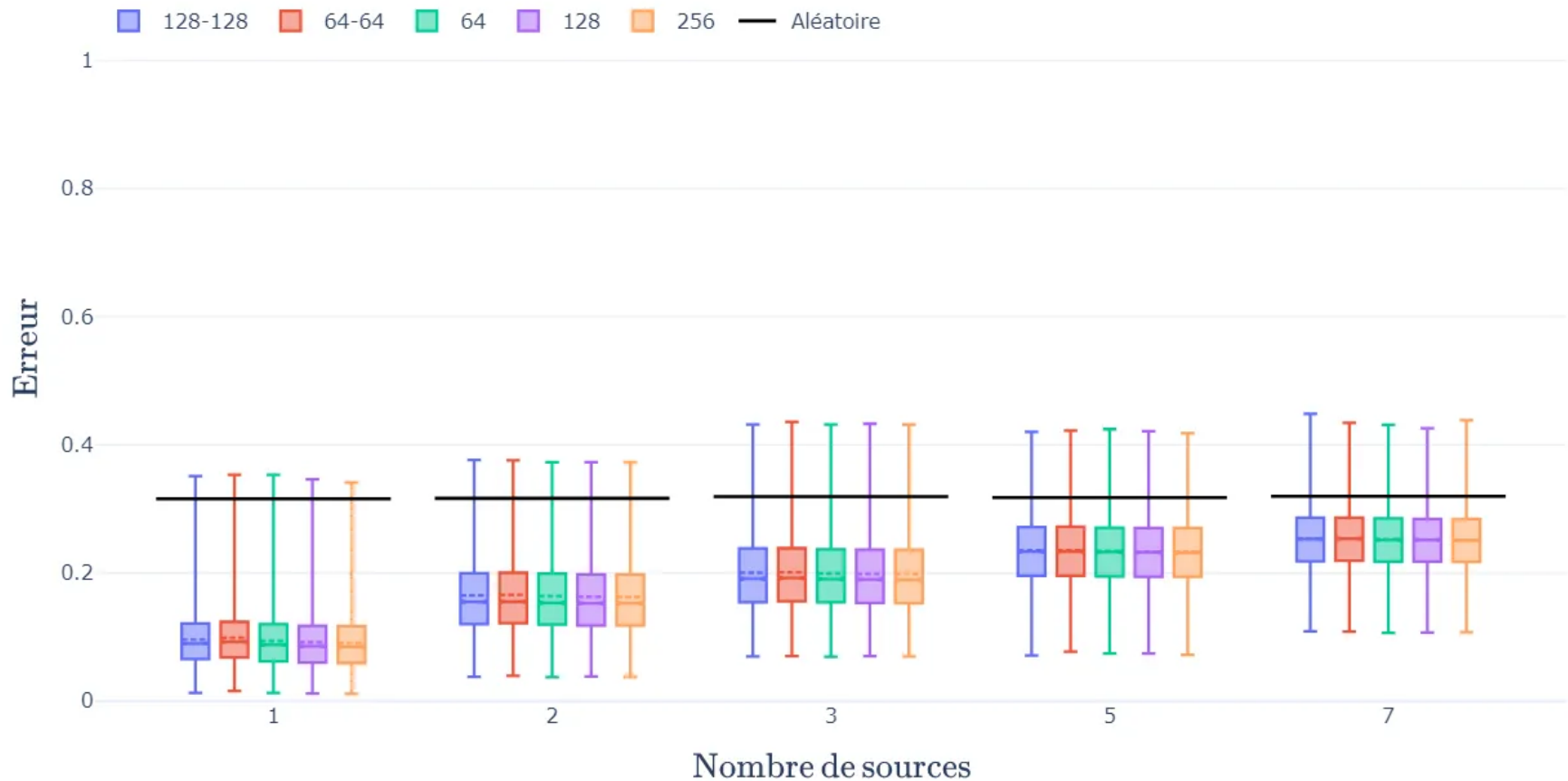
1000 mesures aléatoires par boîte



Comparaison des architectures

Erreur entre le signal original et le signal isolé

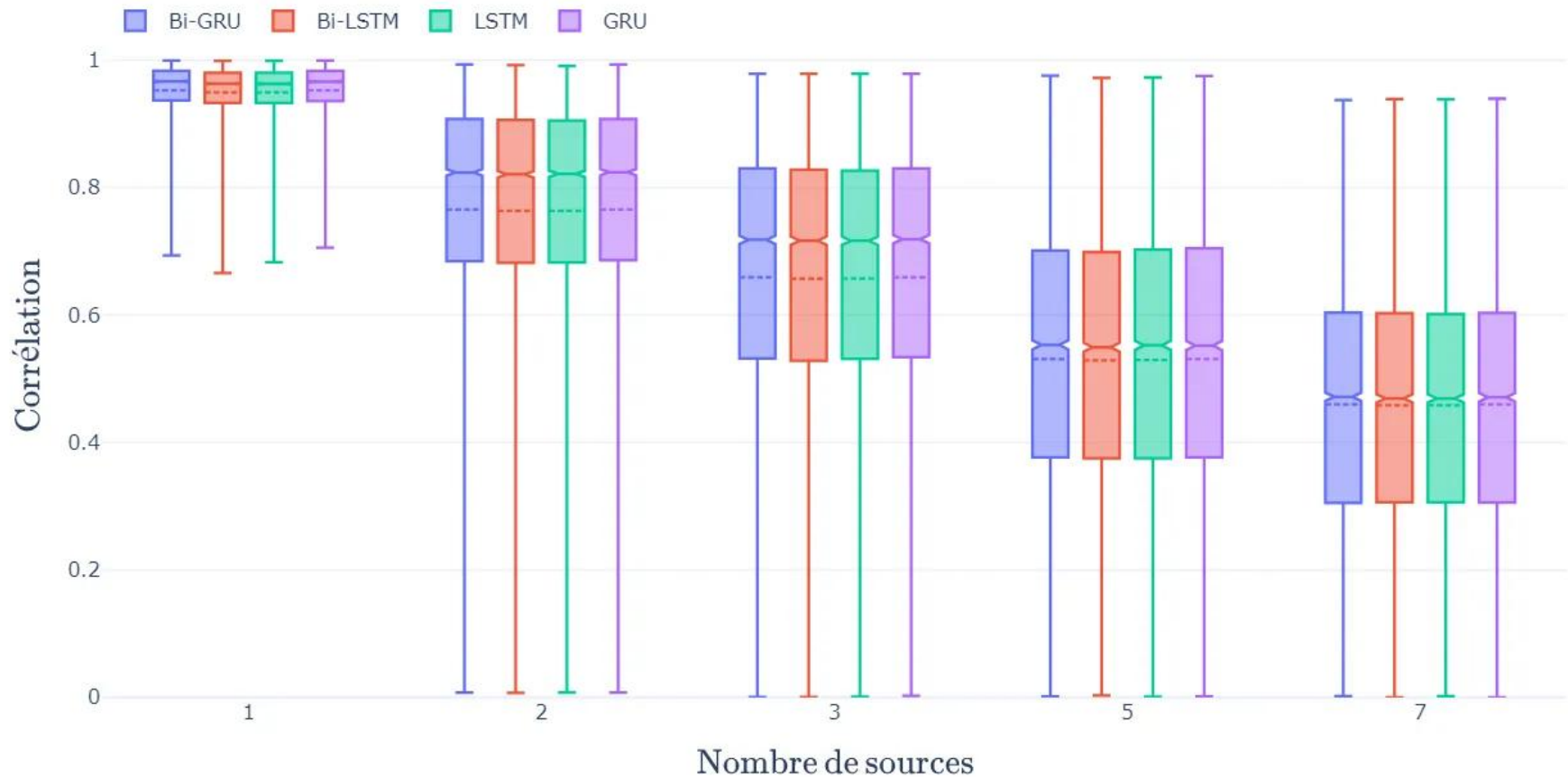
1000 mesures aléatoires par boîte



Comparaison des types de réseaux

Corrélation entre le signal original et le signal isolé

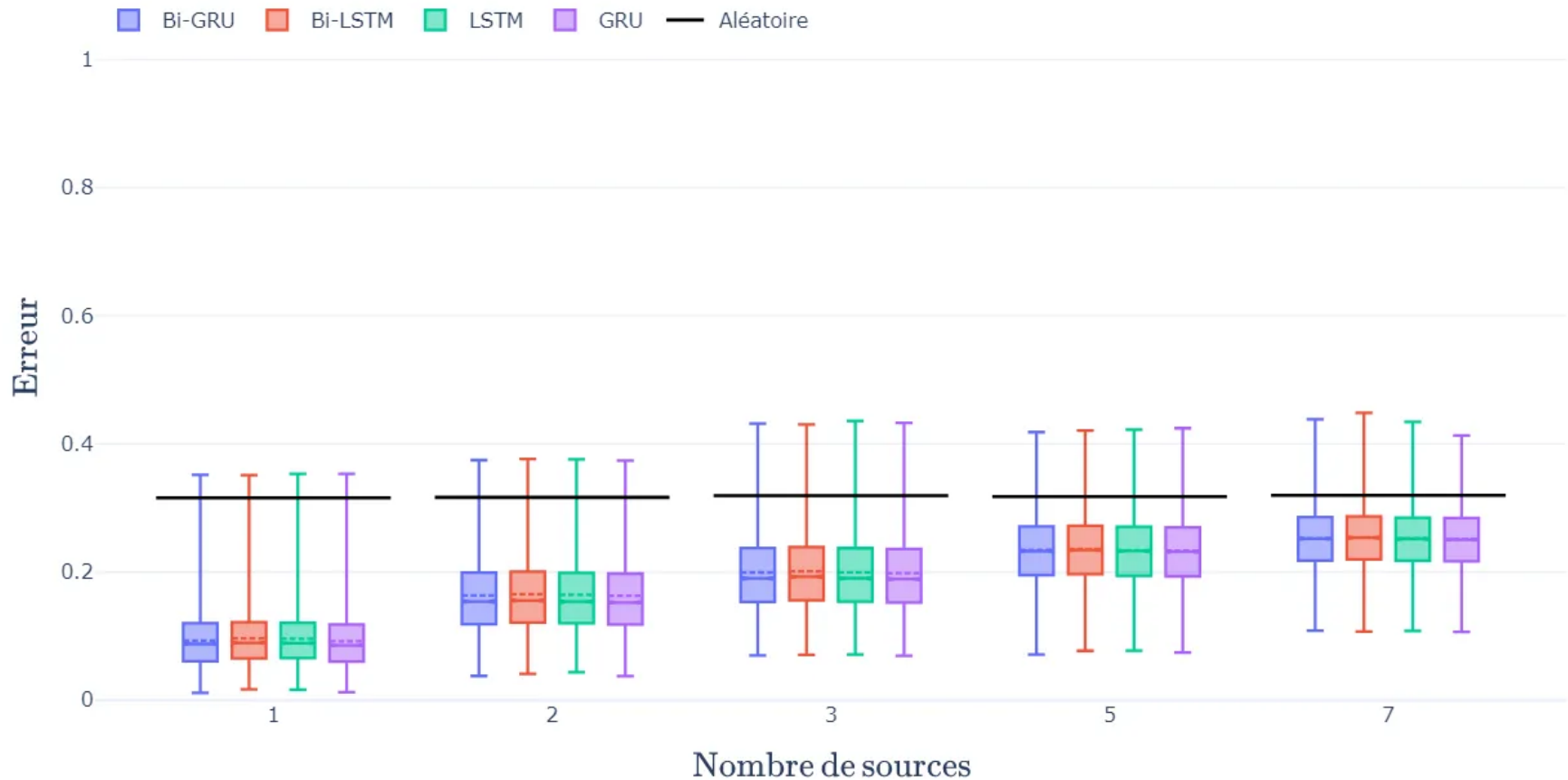
1000 mesures aléatoires par boîte



Comparaison des types de réseaux

Erreur entre le signal original et le signal isolé

1000 mesures aléatoires par boîte



Taille des données analysées

Données	Taille
Statistiques de localisation (dont 10.000 enregistrements)	49,2 Go
Entraînement des modèles d'isolation (40 modèles)	11,42 Go
Statistiques des modèles d'isolation (dont 10.000 enregistrements)	13,2 Go
Total	73,92 Go



Annexe 2

Code source | Python



audio_locate_isolate

Module Python de résolution du TIPE

audio_locate_isolate/**isolate.py**

Méthodes d'isolation des sources sonores


```
"""
```

```
Functions to isolate a source at a specific coordinate
```

```
"""
```

```
import itertools
from math import floor
from typing import Callable

import numpy as np
from sklearn.decomposition import FactorAnalysis
from sklearn.decomposition import FastICA
from sklearn.decomposition import IncrementalPCA
from sklearn.decomposition import MiniBatchNMF
from sklearn.decomposition import NMF
from sklearn.decomposition import PCA
from sklearn.decomposition import SparsePCA
from sklearn.decomposition import TruncatedSVD

from environment import DEFAULT_TEMPERATURE
from environment import define_sound_speed
from environment import distance_gain
from environment import euclidian_distance
from environment import Microphone

from audio_locate_isolate.utils import inverse_spectrogram
from audio_locate_isolate.utils import normalize_array
from audio_locate_isolate.utils import spectrogram

SK_MODEL_NAME = ["FastICA", "NMF", "PCA", "TruncatedSVD"]

SK_MODEL = {
    "FactorAnalysis": FactorAnalysis,
    "FastICA": FastICA,
    "IncrementalPCA": IncrementalPCA,
    "MiniBatchNMF": MiniBatchNMF,
    "NMF": NMF,
    "PCA": PCA,
    "SparsePCA": SparsePCA,
    "TruncatedSVD": TruncatedSVD
}
```

```
SK_NORM = {
    "FactorAnalysis": 0,
    "FastICA": 0,
    "IncrementalPCA": 0,
    "MiniBatchNMF": 1,
    "NMF": 1,
    "PCA": 0,
    "SparsePCA": 0,
    "TruncatedSVD": 0
}
```

```
def align_records(x: float, y: float, z: float,
                 microphones: list[Microphone],
                 records: list[np.ndarray],
                 sample_rate: int,
                 temperature: float = DEFAULT_TEMPERATURE,
                 source_radius: float = 0.1
                 ) -> list[np.ndarray]:
```

```
    """
```

```
    This function aligns many records to be synchronized at a specific coordinate and also adjust gain.
```

```
    :param x: The x coordinate where sources are going to be aligned.
```

```
    :param y: The y coordinate where sources are going to be aligned.
```

```
    :param z: The z coordinate where sources are going to be aligned.
```

```
    :param microphones: List of microphones (corresponding to the record at the same index)
```

```
    :param records: List of records from microphones.
```

```
    :param sample_rate: The sample rate of records.
```

```
    :param temperature: The temperature of the room.
```

```
    :param source_radius: Radius of sources.
```

```
    :return: Records realigned and with the gain adjusted.
```

```
    """
```

```
    assert len(records) == len(microphones), "microphones and records must be the same length"
```

```
    # Create all variables needed
```

```
    dist: list[float] = [euclidian_distance(x, y, z, m.x, m.y, m.z) for m in microphones]
```

```
    reversed_gain = list(map(lambda d: 1 / distance_gain(d, source_radius), dist))
```

```
    time_shift = list(map(lambda d: d / define_sound_speed(temperature), dist))
```

```
    list_shift = list(map(lambda t: floor(t * sample_rate), time_shift))
```

```

# add time shift
shifted_records = [np.concatenate(
    (records[i][s:], np.zeros((s,)))
) for i, s in enumerate(list_shift)
]

shifted_records: list = [s * g for s, g in zip(shifted_records, reversed_gain)]

return shifted_records

```

```

def isolate_sources_spectrogram(x: float, y: float, z: float,
                                microphones: list[Microphone],
                                records: list[np.ndarray],
                                sample_rate: int,
                                error_function: Callable[[list[np.ndarray]], np.ndarray],
                                temperature: float = DEFAULT_TEMPERATURE,
                                spectrogram_length: int = 128,
                                workers: int = 1,
                                source_radius: float = 0.1
                                ) -> np.ndarray:

```

"""

This function isolates sources using the spectrogram method: It takes the spectrogram of the n records and looks to

find différences, if differences are found, values of both n spectrograms are not kept.

Then the function realizes an inverse spectrogram only with kept values.

:param x: The x coordinate of a potential source.

:param y: The y coordinate of a potential source.

:param z: The z coordinate of a potential source.

:param microphones: List of microphones.

:param records: List of records from microphones.

:param sample_rate: The sample rate of all records.

:param temperature: The temperature of the room during the record.

:param spectrogram_length: The length to use for spectrogram.

:param error_function: Function to calculate error.

:param workers: Number of CPU cores to use to calculate the spectrogram.

:param source_radius: Radius of sources.

:return: The potential signal from the source at the given coordinate.

```
"""
```

```
assert len(records) >= 3, "records must contain at least 3 records"  
assert len(microphones) >= 3, "microphones must be at least 3 microphones"  
assert len({len(r) for r in records}) == 1, "All record must be of the same size"
```

```
shifted_records = align_records(x, y, z, microphones, records, sample_rate,  
                                temperature=temperature,  
                                source_radius=source_radius)
```

```
# Calculate spectrogram
```

```
spectrogram_list: list[np.ndarray] = [  
    spectrogram(r, fs=sample_rate, length=spectrogram_length, workers=workers)[2] for r in shifted_records  
]
```

```
result = error_function(spectrogram_list)  
sound = inverse_spectrogram(result, spectrogram_length, workers=workers)
```

```
return sound[:len(records[0])]
```

```
def isolate_source_sklearn(x: float, y: float, z: float,  
                           microphones: list[Microphone],  
                           records: list[np.ndarray],  
                           sample_rate: int,  
                           methode_name: str,  
                           temperature: float = DEFAULT_TEMPERATURE,  
                           source_radius: float = 0.1  
                           ) -> np.ndarray:
```

```
"""
```

```
This function isolates sources using Independent Component Analysis (ICA) method:
```

```
:param x: The x coordinate of a potential source.  
:param y: The y coordinate of a potential source.  
:param z: The z coordinate of a potential source.  
:param microphones: List of microphones.  
:param records: List of records from microphones.  
:param sample_rate: The sample rate of all records.  
:param temperature: The temperature of the room during the record.  
:param source_radius: Radius of sources.  
:param methode_name: Sklearn method name to decompose signals.
```

:return: The potential signal from the source at the given coordinate.
"""

```
assert len(records) >= 3, "records must contain at least 3 records"
assert len(microphones) >= 3, "microphones must be at least 3 microphones"
assert len({len(r) for r in records}) == 1, "All record must be of the same size"
```

```
shifted_records = align_records(x, y, z, microphones, records, sample_rate,
                                temperature=temperature,
                                source_radius=source_radius)
```

```
shifted_records = [r for r in shifted_records]
mx = np.max(np.abs(np.array(shifted_records)))
shifted_records = [r + mx * SK_NORM[methode_name] for r in shifted_records]
```

```
methode = SK_MODEL[methode_name]
```

```
decomp = methode(n_components=len(shifted_records))
res = decomp.fit_transform(np.array(shifted_records).T)
```

```
def choose_best(x1):
    s = np.corrcoef(x1, shifted_records[0])[0][1] + np.corrcoef(x1, shifted_records[1])[0][1] + \
        np.corrcoef(x1, shifted_records[2])[0][1]
```

```
    return float(s)
```

```
res_final = max(
    [res[:, i] - mx * SK_NORM[methode_name] for i in range(len(shifted_records))],
    key=choose_best
)
```

```
return res_final
```

```
def isolate_source_rnn(x: float, y: float, z: float,
                       microphones: list[Microphone],
                       records: list[np.ndarray],
                       sample_rate: int,
                       model,
                       temperature: float = DEFAULT_TEMPERATURE,
```

```

        chunk_size: int = 4410,
        source_radius: float = 0.1):
    """
    This function isolates sources using a recurrent neural network method.
    This function is built for more dimension.
    :param x: The x coordinate of a potential source.
    :param y: The y coordinate of a potential source.
    :param z: The z coordinate of a potential source.
    :param microphones: List of microphones.
    :param records: List of records from microphones.
    :param sample_rate: The sample rate of all records.
    :param model: The LSTM model to use for isolation.
    :param temperature: The temperature of the room during the record.
    :param chunk_size: The input length of the LSTM.
    :param source_radius: Radius of sources.
    :return: The potential signal from the source at the given coordinate.
    """

    assert len(records) >= 3, "records must contain at least 3 records"
    assert len(microphones) >= 3, "microphones must be at least 3 microphones"
    assert len({len(r) for r in records}) == 1, "All record must be of the same size"

    normalize_a, normalize_b = model.get_normalization_param

    n = len(records)
    k = 0
    n_channel_input = 2

    # for _ in range(iter):
    sounds1 = []
    sounds2 = []

    for s in itertools.combinations(list(range(0, n)), n_channel_input):
        shifted_records = align_records(x, y, z,
                                       [microphones[i] for i in s],
                                       [records[i] for i in s],
                                       sample_rate,
                                       temperature=temperature,
                                       source_radius=source_radius)
        # shifted_records = [nr for nr in normalize_array(np.array(shifted_records), normalize_a, normalize_b)]

```

```

        shifted_records = [normalize_array(sr, normalize_a, normalize_b) for sr in shifted_records]
        sounds1.append(shifted_records[0])
        sounds2.append(shifted_records[1])
        k += 1

    out, _, _ = model(sounds1, sounds2, chunk_size=chunk_size, large=True)
    out = [normalize_array(o, -1, 1) for o in out]

    result = np.zeros(records[0].shape)

    for o in out:
        result += o

    result = result / k

    return result

def isolate_source_rnn2(x: float, y: float, z: float,
                        microphones: list[Microphone],
                        records: list[np.ndarray],
                        sample_rate: int,
                        model,
                        temperature: float = DEFAULT_TEMPERATURE,
                        chunk_size: int = 4410,
                        source_radius: float = 0.1):
    """
    Isolate sources using a recurrent neural network with two inputs
    :param x: The x coordinate of a potential source.
    :param y: The y coordinate of a potential source.
    :param z: The z coordinate of a potential source.
    :param microphones: List of microphones.
    :param records: List of records from microphones.
    :param sample_rate: The sample rate of all records.
    :param model: The LSTM model to use for isolation.
    :param temperature: The temperature of the room during the record.
    :param chunk_size: The input length of the LSTM.
    :param source_radius: Radius of sources.
    :return: The potential signal from the source at the given coordinate.

```

```
'''
```

```
assert len(records) >= 3, "records must contain at least 3 records"  
assert len(microphones) >= 3, "microphones must be at least 3 microphones"  
assert len({len(r) for r in records}) == 1, "All record must be of the same size"
```

```
normalize_a, normalize_b = model.get_normalization_param
```

```
shifted_records = align_records(x, y, z,  
                                microphones,  
                                records,  
                                sample_rate,  
                                temperature=temperature,  
                                source_radius=source_radius)
```

```
new_res = np.zeros(records[0].shape)
```

```
for i in range(1):  
    sounds1 = []  
    sounds2 = []  
    k = 0  
    for s1, s2 in itertools.combinations(shifted_records, 2):  
        sounds1.append(normalize_array(s1, normalize_a, normalize_b))  
        sounds2.append(normalize_array(s2, normalize_a, normalize_b))  
        k += 1
```

```
out, _, _ = model(sounds1, sounds2, chunk_size=chunk_size, large=True)  
out = [normalize_array(o, -1, 1) for o in out]
```

```
result = np.zeros(records[0].shape)
```

```
for o in out:  
    result += o
```

```
result = result / k  
new_res = result
```

```
return new_res
```



```

def isolate_source_rnn_triple(x: float, y: float, z: float,
                              microphones: list[Microphone],
                              records: list[np.ndarray],
                              sample_rate: int,
                              model,
                              temperature: float = DEFAULT_TEMPERATURE,
                              chunk_size: int = 4410,
                              source_radius: float = 0.1):
    """
    Isolate sources using a recurrent neural network with 3 inputs
    :param x: The x coordinate of a potential source.
    :param y: The y coordinate of a potential source.
    :param z: The z coordinate of a potential source.
    :param microphones: List of microphones.
    :param records: List of records from microphones.
    :param sample_rate: The sample rate of all records.
    :param model: The LSTM model to use for isolation.
    :param temperature: The temperature of the room during the record.
    :param chunk_size: The input length of the LSTM.
    :param source_radius: Radius of sources.
    :return: The potential signal from the source at the given coordinate.
    """
    normalize_a, normalize_b = model.get_normalization_param
    shifted_records = align_records(x, y, z,
                                    microphones,
                                    records,
                                    sample_rate,
                                    temperature=temperature,
                                    source_radius=source_radius)

    sounds = [normalize_array(s, normalize_a, normalize_b) for s in shifted_records]

    out = model([[sounds[0]], [sounds[1]], [sounds[2]]], chunk_size=chunk_size, large=True)

    return normalize_array(out[0], -1, 1)

```

audio_locate_isolate/**locate.py**

Méthodes d'isolation des sources sonores

```
"""
```

```
Method to locate a/many source(s) of sound using acoustic triangulation (or even with more microphone)
```

```
"""
```

```
import time
```

```
from itertools import product
```

```
from math import floor
```

```
import numpy as np
```

```
from scipy.optimize import minimize
```

```
from scipy.signal import correlate
```

```
from scipy.signal import find_peaks
```

```
from sklearn.cluster import MeanShift
```

```
from audio_locate_isolate.utils import inverse_spectrogram
```

```
from environment import DEFAULT_TEMPERATURE
```

```
from environment import define_sound_speed
```

```
from environment import euclidian_distance
```

```
from environment import Microphone
```

```
from audio_locate_isolate.utils import spectrogram
```

```
from math import sqrt
```

```
x_candidate = [
```

```
    lambda x1, x2, y1, y2, t1, t2: (...)
```

```
]
```

```
SCORE = {i: 0 for i in range(16)}
```

```
def signal_time_shift(signal1: np.ndarray, signal2: np.ndarray, unit: str = "s", rate: int = None) -> int | float:
```

```
    """
```

```
Determine the time shift between the first and the second signal using cross-correlation.
```

```
It can return the result in second (unit='s') or in list index (unit='index')
```

```
combien de temps d'avance à le signal 1 sur le signal 2
```

```
    """
```

```
corr: np.ndarray = correlate(signal1, signal2, method="fft", mode="full")
```

```
n = len(corr)
```

```

if unit == "index":
    return floor(float(np.argmax(corr)) - n / 2)

elif unit == "s":
    return (float(np.argmax(corr)) - n / 2) * (1 / rate)

else:
    raise Exception("Unsupported unit.")

```

```

def labels_barycenter(coordinates: np.ndarray, labels: np.ndarray) -> dict:
    """
    This function calculate the barycenter of a labeled dataset.
    :param coordinates: Coordinates of points.
    :param labels: Labels of points.
    :return: Dict of the form : Label -> Barycenter
    """
    label_groups = {}

    for coord, label in zip(coordinates, labels):
        if label not in label_groups:
            label_groups[label] = []
        label_groups[label].append(coord)

    bc = {label: np.mean(coords, axis=0) for label, coords in label_groups.items()}

    return bc

```

```

def trilateration(center_mic: tuple,
                  mic1: tuple,
                  mic2: tuple,
                  distance_shift_c1: float,
                  distance_shift_c2: float) -> tuple[float, float]:
    """
    This function uses trilateration equations to find the coordinate of a point based on latency with microphones.
    :param center_mic: Center point of the trilateration. (microphone 0)
    :param mic1: microphone 1 used to trilaterate
    :param mic2: microphone 2 used to trilaterate

```

```

:param distance_shift_c1: "distance shift" between m0 and m1
:param distance_shift_c2: "distance shift" between m0 and m2
:return: The coordinate of the source. Exception if it had not been found.
"""

```

```

x1, y1 = (mic1[0] - center_mic[0], mic1[1] - center_mic[1])
x2, y2 = (mic2[0] - center_mic[0], mic2[1] - center_mic[1])
t1 = distance_shift_c1
t2 = distance_shift_c2

```

```

candidates = set()

```

```

y_candidate = [
    lambda x: (-t1 * sqrt(
        (-t1 ** 2 + x1 ** 2 + y1 ** 2) * (-t1 ** 2 + 4 * x ** 2 - 4 * x * x1 + x1 ** 2 + y1 ** 2)) * (
            t1 - y1) * (t1 + y1) + y1 * (t1 ** 2 - y1 ** 2) * (
                t1 ** 2 + 2 * x * x1 - x1 ** 2 - y1 ** 2)) / (
                2 * (t1 - y1) * (t1 + y1) * (t1 ** 2 - y1 ** 2)),
    lambda x: (t1 * sqrt(
        (-t1 ** 2 + x1 ** 2 + y1 ** 2) * (-t1 ** 2 + 4 * x ** 2 - 4 * x * x1 + x1 ** 2 + y1 ** 2)) * (
            t1 - y1) * (t1 + y1) + y1 * (t1 ** 2 - y1 ** 2) * (
                t1 ** 2 + 2 * x * x1 - x1 ** 2 - y1 ** 2)) / (
                2 * (t1 - y1) * (t1 + y1) * (t1 ** 2 - y1 ** 2)),
    lambda x: (-t2 * sqrt(
        (-t2 ** 2 + x2 ** 2 + y2 ** 2) * (-t2 ** 2 + 4 * x ** 2 - 4 * x * x2 + x2 ** 2 + y2 ** 2)) * (
            t2 - y2) * (t2 + y2) + y2 * (t2 ** 2 - y2 ** 2) * (
                t2 ** 2 + 2 * x * x2 - x2 ** 2 - y2 ** 2)) / (
                2 * (t2 - y2) * (t2 + y2) * (t2 ** 2 - y2 ** 2)),
    lambda x: (t2 * sqrt(
        (-t2 ** 2 + x2 ** 2 + y2 ** 2) * (-t2 ** 2 + 4 * x ** 2 - 4 * x * x2 + x2 ** 2 + y2 ** 2)) * (
            t2 - y2) * (t2 + y2) + y2 * (t2 ** 2 - y2 ** 2) * (
                t2 ** 2 + 2 * x * x2 - x2 ** 2 - y2 ** 2)) / (
                2 * (t2 - y2) * (t2 + y2) * (t2 ** 2 - y2 ** 2)),
]

```

```

for fy in y_candidate:
    for fx in x_candidate:
        try:
            x = fx(x1, x2, y1, y2, t1, t2)

```

```

        candidate = (x + center_mic[0], fy(x) + center_mic[1])

        candidates.add(candidate)
    except Exception as e:
        _ = e

def candidate_compare(cand):
    """
    Function to compare many candidates
    :param cand: candidate
    :return: The error on this candidate
    """
    cand = cand[:2]
    cd1 = abs(euclidian_distance(*center_mic, 0, *cand, 0) - euclidian_distance(*mic1, 0, *cand, 0) - t1)
    cd2 = abs(euclidian_distance(*center_mic, 0, *cand, 0) - euclidian_distance(*mic2, 0, *cand, 0) - t2)
    return cd1 + cd2

c = min(candidates, key=candidate_compare, default=None)

return c

def possible_coordinate(microphones: list[Microphone],
                        distance_shift: list[tuple | list],
                        start_point: np.ndarray | None,
                        max_error: float,
                        method: str = "optimize"
                        ) -> list[tuple[float, float, float]]:
    """
    Find many possible coordinate for a set of candidate.
    :param microphones: List of microphones used.
    :param distance_shift: List of measured "distance shift" in records
    :param start_point: The starting point of the search
    :param max_error: The max tolerated error in m
    :param method: Methode to use ('optimize' or 'trilateration')
    :return:
    """
    # Create the error function
    n = len(distance_shift[0])

```

```

dim = len(start_point)

t_range = [(i, j) for j in range(n) for i in range(n) if j < i]

coords = []

if method == "optimize":
    def error_function(m: np.array, selected_ts: tuple) -> float:
        if dim == 3:
            x, y, z = m
        elif dim == 2:
            x, y = m
            z = 0
        else:
            raise Exception("Dimension unsupported.")

        error = 0
        k = 0
        for i, j in t_range:
            di = euclidian_distance(x, y, z, *microphones[i].get_coord)
            dj = euclidian_distance(x, y, z, *microphones[j].get_coord)
            delta = (di - dj)

            error += abs(delta - selected_ts[k])
            k += 1

        return error / k

    # For each timeshift minimize the error for x, y, z coordinates
    min_ts = {(i, j): euclidian_distance(*microphones[i].get_coord, *microphones[j].get_coord)
              for i in range(n) for j
              in range(n) if j < i
              }

    for elt in distance_shift:
        # remove impossible distance shift
        if any(map(lambda c: abs(c[0]) > min_ts[c[1]], zip(elt, t_range))):
            continue

```

```

res = minimize(error_function, start_point, args=(elt,), method="Nelder-Mead", tol=0.001)

if error_function(res.x, elt) < max_error:
    if dim == 3:
        x1, y1, z1 = res.x
    elif dim == 2:
        x1, y1 = res.x
        z1 = 0.0
    else:
        raise Exception("Dimension unsupported.")

    coords.append([x1, y1, z1])

elif method == "trilateration":
    min_ts = {(i, j): euclidian_distance(*microphones[i].get_coord, *microphones[j].get_coord)
              for i in range(n) for j
              in range(n) if j < i
              }

    for elt in distance_shift:
        # remove impossible distance shift
        if any(map(lambda c: abs(c[0]) > min_ts[c[1]], zip(elt, t_range))):
            continue

    mic_time_shift = {(i, j): elt[k] for k, (i, j) in enumerate(t_range)}

    ds1 = mic_time_shift.get((0, 1), - mic_time_shift.get((1, 0), 0.0))
    ds2 = mic_time_shift.get((0, 2), - mic_time_shift.get((2, 0), 0.0))

    res = trilateration(microphones[0].get_coord[:2],
                       microphones[1].get_coord[:2],
                       microphones[2].get_coord[:2],
                       ds1,
                       ds2)

    if res is not None:
        x1, y1 = res
        z1 = 0.0

```



```

        coords.append([x1, y1, z1])
    else:
        raise Exception("Unsupported method.")

# If there are no candidate
    if len(coords) == 0:
        return []

coords = np.array(coords)

# Clustering to find barycentre of each cluster corresponding to potential source coordinate
result: list[tuple[float, float, float] | tuple[float, float]] = []

clustering = MeanShift()
clustering.fit(coords)

labels = clustering.labels_
cluster_centers = labels_barycenter(coords, labels)

unique, counts = np.unique(labels, return_counts=True)
dict_labels = dict(zip(unique, counts))

for i, center in sorted(*cluster_centers.items(), key=lambda c: dict_labels[c[0]], reverse=True):
    if dict_labels[i] > 1:
        if dim == 3:
            xc, yc, zc = center
            result.append((xc, yc, zc))
        elif dim == 2:
            xc, yc, _ = center
            result.append((xc, yc))

return result

def locate_source(microphones: list[Microphone],
                  records: list[np.ndarray],
                  sample_rate: int,
                  start_point: np.ndarray | None,
                  temperature: float = DEFAULT_TEMPERATURE,

```

```

        method: str = "optimize") -> tuple[float, float, float] | tuple[float, float]:
    """
    This function tries to find the coordinate of the source.
    It first determines the time shift between all signals and then uses a minimization algorithm to find the
    coordinate
    of the source.
    :param microphones: List of microphones.
    :param records: List of records.
    :param sample_rate: Sampling rate of the two signals.
    :param start_point: The starting point of the search of sources
    :param temperature: Temperature of space where the mesure is realized.
    :param method: Methode to use to find coordinate. ('optimize' or 'trilateration')
    """
    assert len(records) >= 3, "records must contain at least 3 records"
    assert len(records) == len(microphones), "microphones and records must be the same length"

    n = len(records)
    sound_speed = define_sound_speed(temperature)

    mic_time_shift: dict[tuple[int, int], float] = {}

    # Get all time shifts for every microphone
    t_range = [(i, j) for j in range(n) for i in range(n) if j < i]
    for i, j in t_range:
        ts: float = signal_time_shift(records[i], records[j], rate=sample_rate, unit="s")
        mic_time_shift[(i, j)] = ts * sound_speed

    if method == "optimize":
        dim = len(start_point)

        def error_function(m: np.ndarray) -> float:
            if dim == 3:
                x, y, z = m
            elif dim == 2:
                x, y = m
                z = 0
            else:
                raise Exception("Dimension unsupported.")

```

```

error = 0
k = 0
for i, j in t_range:
    di = euclidian_distance(x, y, z, *microphones[i].get_coord)
    dj = euclidian_distance(x, y, z, *microphones[j].get_coord)
    delta = (di - dj)

    error += abs(delta - mic_time_shift[(i, j)])
    k += 1

return error / k

# find the coordinate of the source using minimization algorithm
res = minimize(error_function, start_point, tol=0.001, method="Nelder-Mead")

if len(start_point) == 3:
    x1, y1, z1 = res.x
    return float(x1), float(y1), float(z1)

elif len(start_point) == 2:
    x1, y1 = res.x
    return float(x1), float(y1), 0.0
else:
    raise Exception("Dimension unsupported.")

elif method == "trilateration":
    ds1 = mic_time_shift.get((0, 1), - mic_time_shift.get((1, 0), None))
    ds2 = mic_time_shift.get((0, 2), - mic_time_shift.get((2, 0), None))

    res = trilateration(microphones[0].get_coord[:2],
                        microphones[1].get_coord[:2],
                        microphones[2].get_coord[:2],
                        ds1,
                        ds2)

    if res is None:
        return tuple()

    else:

```

```

        return res + (0,)

    else:
        raise Exception("Method unsupported.")

def locate_many_sources_spikes(microphones: list[Microphone],
                               records: list[np.ndarray],
                               sample_rate: int,
                               n_peaks: int,
                               start_point: np.ndarray,
                               max_error: float = 1.0,
                               temperature: float = DEFAULT_TEMPERATURE,
                               method: str = "optimize"
                               ):
    """
    This function locate many sources using the spikes method.
    :param microphones: List of microphones.
    :param records: List of records.
    :param sample_rate: Sampling rate of the two signals.
    :param start_point: The starting point of the search of sources
    :param temperature: Temperature of space where the mesure is realized.
    :param method: Methode to use to find coordinate. ('optimize' or 'trilateration')
    :param n_peaks: Number of peaks to use.
    :param max_error: Max tolerated error
    :return: A list of possible coordinates.
    """
    # Prepare data
    n = len(records)
    t_range = [(i, j) for j in range(n) for i in range(n) if j < i]

    initial_distance_shift = []

    # Get all 'n_tries' peaks in records
    for (i, j) in t_range:
        corr: np.ndarray = correlate(records[i], records[j], method="fft", mode="full")
        peaks, _ = find_peaks(corr)
        peaks: np.ndarray = peaks
        c: np.ndarray = np.argpartition(corr[peaks], -n_peaks)[-n_peaks:]

```

```

ds = np.round(peaks[c] - len(corr) / 2) * 1 / sample_rate * define_sound_speed(temperature)

initial_distance_shift.append(ds)

list_distance_shift = [*product(*initial_distance_shift)]

return possible_coordinate(microphones, list_distance_shift, start_point, max_error, method=method)

def locate_many_sources_spectrogram(microphones: list[Microphone],
                                     records: list[np.ndarray],
                                     sample_rate: int,
                                     start_point: np.ndarray,
                                     length: int = 16,
                                     max_error: float = 1.0,
                                     temperature: float = DEFAULT_TEMPERATURE,
                                     method: str = "optimize") -> list[tuple[float, float, float]]:
    """
    This function locate many sources using the spectrogram method.
    :param microphones: List of microphones.
    :param records: List of records.
    :param sample_rate: Sampling rate of the two signals.
    :param start_point: The starting point of the search of sources
    :param temperature: Temperature of space where the mesure is realized.
    :param method: Methode to use to find coordinate. ('optimize' or 'trilateration')
    :param length: Length of the spectrogram to use.
    :param max_error: Max tolerated error
    :return: A list of possible coordinates.
    """
    n = len(records[0])
    records = np.array(records)
    pad_length = (length - (n % length)) % length
    records = np.pad(records, ((0, 0), (0, pad_length)))
    segmented_records = records.reshape(len(records), -1, length)

    spectrogram = np.fft.rfft(segmented_records, axis=2)

    indices = np.arange(length // 2 + 1)
    modified_spectrograms = np.zeros(

```

```

(len(records), length // 2 + 1, (n // length + 1), length // 2 + 1),
dtype=np.complex128)

for k in range(len(records)):
    modified_spectrograms[k, indices, :, indices] = spectrogram[k, :, indices]

reconstructed_segments = np.fft.irfft(modified_spectrograms, n=length, axis=3)
reconstructed_signals = reconstructed_segments.transpose(1, 0, 2, 3).reshape(length // 2 + 1, len(records), -1)
signal_list = np.swapaxes(reconstructed_signals, 0, 1)

# get timeshift
i_to_d = 1 / sample_rate * define_sound_speed(temperature)
freq = np.fft.rfftfreq(length, 1 / sample_rate)

list_distance_shift = [
    [signal_time_shift(signal_list[i][fi], signal_list[j][fi], "index") * i_to_d
     for j in range(len(records))
     for i in range(len(records)) if j < i] for fi, _ in enumerate(freq)
]

return possible_coordinate(microphones, list_distance_shift, start_point, max_error, method=method)

def locate_many_sources_both(microphones: list[Microphone],
                             records: list[np.ndarray],
                             sample_rate: int,
                             n_tries: int,
                             start_point: np.ndarray,
                             length: int = 16,
                             max_error: float = 1.0,
                             temperature: float = DEFAULT_TEMPERATURE,
                             method: str = "optimize"
                             ) -> list[tuple[float, float, float] | tuple[float, float]]:
    """
    This function locate many sources using both the spectrogram and peaks method.
    :param microphones: List of microphones.
    :param records: List of records.
    :param sample_rate: Sampling rate of the two signals.
    :param start_point: The starting point of the search of sources

```

```

:param temperature: Temperature of space where the mesure is realized.
:param method: Methode to use to find coordinate. ('optimize' or 'trilateration')
:param length: Length of the spectrogram to use.
:param n_tries: Numbers of spikes to use
:param max_error: Max tolerated error
:return: A list of possible coordinates.
"""

t, freq, _ = spectrogram(records[0], sample_rate, length)
n = len(records)
mask = np.array([freq for _ in range(len(t))]).T

spectrogram_list = [spectrogram(r, sample_rate, length, workers=12)[2] for r in records]

signal_list = [
    [
        inverse_spectrogram(np.where(mask != f, 0, spectrogram_list[i]), length, workers=12) for f in freq
    ] for i in range(n)
]

t_range = [(i, j) for j in range(n) for i in range(n) if j < i]

list_distance_shift = []

for sig1, sig2, sig3 in zip(*signal_list):
    k = 0
    sig = [sig1, sig2, sig3]

    init_ds = []

    for (i, j) in t_range:
        corr: np.ndarray = correlate(sig[i], sig[j], method="fft", mode="full")
        peaks, _ = find_peaks(corr)
        peaks: np.ndarray = peaks
        c: np.ndarray = np.argmaxpartition(corr[peaks], -n_tries)[-n_tries:]

        ds = np.round(peaks[c] - len(corr) / 2) * 1 / sample_rate * define_sound_speed(temperature)

        init_ds.append(ds)

```

```

        k += 1

    list_distance_shift += [*product(*init_ds)]

    return possible_coordinate(microphones, list_distance_shift, start_point, max_error, method=method)

if __name__ == '__main__':
    import os
    from environnement import Environment
    from environnement import Microphone
    from environnement import MicrophoneTensor
    from environnement import Sound
    from environnement import Source

    PATH_EVAL: str = "C:/Users/Utilisateur/Documents/Python/ADVANCED/GeoSoundTIPE/sound_data/test/"
    RATE = 44100
    sounds_list_eval = [Sound(f"{PATH_EVAL}{f}", rate=RATE) for f in os.listdir(PATH_EVAL)]

    m1 = Microphone("m1", 10, 10, 0)
    m2 = Microphone("m2", 5, 7, 0)
    m3 = Microphone("m3", 9, 3, 0)

    s1 = Source(sounds_list_eval[0], "s1", 7, 6, 0)
    s2 = Source(sounds_list_eval[1], "s2", 9, 8, 0)
    s3 = Source(sounds_list_eval[2], "s3", 8.5, 6.5, 0)

    tens3d = MicrophoneTensor(
        "3D",
        np.linspace(-1.5, 1.5, 50),
        np.linspace(-1.5, 1.5, 50),
        np.linspace(-1.5, 1.5, 50)
    )

    tens2d = MicrophoneTensor(
        "2D",
        np.linspace(-10, 10, 500),
        np.linspace(-10, 10, 500),

```



```

    np.linspace(0, 0, 1)
)
environment = Environment([s1, s2, s3], [m1, m2, m3], rate=RATE, microphone_tensors=[tens3d, tens2d])

lr = environment.listen_from_microphone(["m1", "m2", "m3"], 60, 65, 59, True)[1]

print("----- TEST -----")

print("\nlocate_source")

t = time.time()
print(locate_source([m1, m2, m3], lr, RATE, np.array([8.5, 6.5]), method="trilateration"))
print(time.time() - t)

t = time.time()
print(locate_source([m1, m2, m3], lr, RATE, np.array([8.5, 6.5]), method="optimize"))
print(time.time() - t)
print("\nlocate_many_sources_spectrogram")

t = time.time()
print(locate_many_sources_spectrogram([m1, m2, m3], lr, RATE, np.array([8.5, 6.5]), max_error=0.5, length=8,
                                     method="trilateration")
)
print(time.time() - t)

t = time.time()
print(locate_many_sources_spectrogram([m1, m2, m3], lr, RATE, np.array([8.5, 6.5]), max_error=0.5, length=8,
                                     method="optimize"))

print(time.time() - t)

print("\nlocate_many_sources_spikes")

t = time.time()
print(locate_many_sources_spikes([m1, m2, m3], lr, RATE, 10, np.array([8.8, 6.3]), max_error=0.34,
                               method="trilateration"))
print(time.time() - t)

```

```
t = time.time()
print(locate_many_sources_spikes([m1, m2, m3], lr, RATE, 10, np.array([8.8, 6.3]), max_error=0.34,
                                method="optimize"))
print(time.time() - t)

print("\nlocate_many_sources_both")

t = time.time()
print(locate_many_sources_both([m1, m2, m3], lr, RATE, 5, np.array([8.5, 6.5]), 16, max_error=0.34,
                              method="trilateration"))
print(time.time() - t)

t = time.time()
print(locate_many_sources_both([m1, m2, m3], lr, RATE, 5, np.array([8.5, 6.5]), 16, max_error=0.34,
                              method="optimize"))
print(time.time() - t)

print(SCORE)
```

audio_locate_isolate/**neural_network.py**

Réseaux de neurones pour isolation

```
"""
```

Neural networks used to isolate audio signal

```
"""
```

```
__import__("os").environ["KERAS_BACKEND"] = "jax"  
__import__("os").environ["XLA_PYTHON_CLIENT_PREALLOCATE"] = "false"
```

```
from copy import deepcopy  
from datetime import datetime  
from math import floor  
import os  
import random  
from threading import main_thread  
from threading import Thread  
import tkinter as tk  
import time  
from typing import Any  
from typing import Callable  
import warnings  
  
from keras.api.callbacks import LearningRateScheduler  
from keras.api.layers import Bidirectional  
from keras.api.layers import Dense  
from keras.api.layers import Input  
from keras.api.layers import GRU  
from keras.api.layers import LSTM  
from keras.api.models import Model  
from keras.api.models import load_model  
from keras.api.optimizers import Adam  
import matplotlib  
import matplotlib.animation as animation  
from matplotlib.backends.backend_tkagg import FigureCanvasTkAgg  
import matplotlib.pyplot as plt  
import numpy as np  
  
from environment import DEFAULT_TEMPERATURE  
from environment import define_sound_speed  
from environment import distance_gain  
from environment import euclidian_distance  
from environment import Environment
```

```

from environnement import Microphone
from environnement import Source
from environnement import Sound

from audio_locate_isolate.isolate import isolate_source_rnn_triple
from audio_locate_isolate.utils import absolute_error
from audio_locate_isolate.utils import env_test_isolation
from audio_locate_isolate.utils import float_array
from audio_locate_isolate.utils import inverse_spectrogram
from audio_locate_isolate.utils import normalize_array
from audio_locate_isolate.utils import relative_error
from audio_locate_isolate.utils import spectrogram

EVALUATE: bool = False
loaded_data: dict = {}
graph_results = []
saving_thread_queue: list[tuple[str, dict | float_array]] | list = []

```

```

class IsolateModel:
    def __init__(self,
        neural_network: Model,
        input_number: int = 2,
        output_number: int = 3,
        norm_a: float = -1,
        norm_b: float = 1,
        use_dropout: float | None = 0.1,
        lr: float = 0.001):
        self.opt = Adam

        self.neural_network = neural_network

        self.input_number = input_number
        self.output_number = output_number

        self.norm_a = norm_a
        self.norm_b = norm_b

        self.use_dropout = use_dropout

```

```

self.lr = lr

def save_model(self, path: str) -> None:
    """
    Save the model in a file '.keras' file
    :param path: Path and filename (with .keras at the end)
    :return: None
    """
    self.neural_network.save(path)

def load_model(self, path: str) -> None:
    """
    Load a model from a '.keras' file
    :param path: Path to a '.keras' file
    :return: None
    """
    new_network = load_model(path)
    self.neural_network.set_weights(new_network.get_weights())

@property
def get_normalization_param(self) -> tuple[float, float]:
    """
    Returns the lower and upper bound for normalization of inputs.
    :return: Normalization bounds.
    """
    return self.norm_a, self.norm_b

def edit_dataset(self,
                 shifted_records,
                 result,
                 only_input: bool = False) -> tuple[list[float_array], list[float_array]]:
    """
    This function edits the dataset to fit with the model.
    :param shifted_records: Shifted records given as input for the model.
    :param result: The sound that the model has to isolate.
    :param only_input: To transform only an input
    :return: X_train and Y_train
    """
    _ = self

```

```

        return [np.array([])], [np.array([])]

def summary(self) -> None:
    self.neural_network.summary()

def get_weights(self):
    return self.neural_network.get_weights()

def set_weights(self, weights) -> None:
    self.neural_network.set_weights(weights)

def train_model(self,
                x_train: float_array,
                y_train: float_array,
                batch_size=8,
                epochs=1,
                *args, **kwargs) -> None:
    self.neural_network.fit(x_train, y_train, *args, batch_size=batch_size, epochs=epochs, **kwargs)

class RawIsolateModel(IsolateModel):
    def __init__(self, layers: list, use_dropout: float | None = 0.1, lr: float = 0.001):
        self.opt = Adam

        optimizer = self.opt(learning_rate=lr)
        inputs = Input(shape=(None, 3,))
        x = None

        for i, layer in enumerate(layers):
            if i == 0:
                x = layer(inputs)

            else:
                x = layer(x)

        y = Dense(3)(x)

        self.neural_network = Model(inputs=inputs, outputs=y)

```

```

self.neural_network.compile(loss='mse',
                             optimizer=optimizer,
                             metrics=["accuracy", "mse"],
                             run_eagerly=False,
                             jit_compile=True,
                             )

super().__init__(neural_network=self.neural_network,
                 input_number=3,
                 output_number=1,
                 norm_a=-1,
                 norm_b=1,
                 use_dropout=use_dropout,
                 lr=lr)

def __call__(self,
             sounds: list[list[float_array]],
             chunk_size: int = None,
             large: bool = False,
             *args, **kwargs) -> list[float_array]:
    """
    Call the neural network for audio isolation
    :param sounds: List of audio signals
    :param chunk_size: Output length of the model
    :param args: *
    :param kwargs: **
    :return: Return a signal (isolate)
    """
    n = len(sounds[0][0])
    chunk_size: int = n if chunk_size is None else chunk_size
    sound_amount = len(sounds[0])
    x = []
    x_index = []

    # create chunks
    for k, s_all in enumerate(zip(*sounds)):
        for i in range(0, n, chunk_size):
            d = np.array([normalize_array(s[i:min(i + chunk_size, n)], self.norm_a, self.norm_b) for s in
s_all]).T

```



```

        x.append(d)
        x_index.append(k)

# get the output of the neural network
    if not large:
        output = self.neural_network.call(np.array(x), *args, training=False, **kwargs)
    else:
        output = self.neural_network.predict(np.array(x), verbose=False, *args, **kwargs)

# transform chunk data into one sigle signal
    y = [np.array([]) for _ in range(sound_amount)]

    for i, line in enumerate(output):
        ind = x_index[i]
        y[ind] = np.append(y[ind], np.reshape(line[:, 0], (chunk_size,)))

    return y

def edit_dataset(self,
                 shifted_records: list[float_array],
                 result: list[float_array],
                 only_input: bool = False) -> tuple[list[float_array], list[float_array]]:

    data_x = np.array([normalize_array(r, self.norm_a, self.norm_b) for r in shifted_records]).T
    if not only_input:
        data_y = np.reshape(
            normalize_array(np.array(result), self.norm_a, self.norm_b),
            (len(result[0]), self.output_number)
        )
    else:
        return [data_x], []

    return [data_x], [data_y]

class SpectrogramIsolateModel(IsolateModel):
    def __init__(self,
                 layers: list,
                 rate: float = 44100,

```

```

        length: int = 128,
        use_dropout: float | None = 0.1,
        lr: float = 0.001):
    self.rate = rate
    self.length = length

    self.opt = Adam

    optimizer = self.opt(learning_rate=lr)
    inputs = Input(shape=(None, 3 * 1 * (length // 2 + 1),))
    x = None

    for i, layer in enumerate(layers):
        if i == 0:
            x = layer(inputs)

        else:
            x = layer(x)

    y = Dense(1 * (length // 2 + 1))(x)

    self.neural_network = Model(inputs=inputs, outputs=y)

    self.neural_network.compile(loss='mse',
                                optimizer=optimizer,
                                metrics=["accuracy", "mse"],
                                run_eagerly=False,
                                jit_compile=True,
                                )

    super().__init__(neural_network=self.neural_network,
                     input_number=3,
                     output_number=1,
                     norm_a=-1,
                     norm_b=1,
                     use_dropout=use_dropout,
                     lr=lr)

def edit_dataset(self,

```

```

        shifted_records: list[float_array],
        result: list[float_array],
        only_input: bool = False) -> tuple:
fft_sr = [spectrogram(sr, self.rate, self.length)[2] for sr in shifted_records]
res = [spectrogram(r, self.rate, self.length)[2] for r in result]

x = []
for lines in zip(*[s.T for s in fft_sr]):
    x.append(
        np.concatenate((*(
            # [np.abs(l) / np.max(np.abs(l)) if np.max(np.abs(l)) > 0 else np.zeros((len(l),)) for l in
lines] +
            [np.angle(l) / np.pi for l in lines])),))
    )

if not only_input:
    y = []
    for lines in zip(*[s.T for s in res]):
        y.append(
            np.concatenate((*(
                # [np.abs(l) / np.max(np.abs(l)) if np.max(np.abs(l)) > 0 else np.zeros((len(l),)) for l in
lines] +
                [np.angle(l) / np.pi for l in lines])),))
        )
    return [np.array(x)], [np.array(y)]

else:
    return [np.array(x)], []

def __call__(self,
    sounds: list[list[float_array]],
    chunk_size=None,
    large: bool = False,
    *args,
    **kwargs) -> list[float_array]:

x = []
for ls in zip(*sounds):
    inp, _ = self.edit_dataset(list(ls), [], True)

```

```

        x.append(inp[0])

    if not large:
        output = self.neural_network.call(np.array(x), *args, training=False, **kwargs)
    else:
        output = self.neural_network.predict(np.array(x), *args, verbose=False, **kwargs)

    y = []

    for out, inp in zip(output, sounds[0]):
        y_fft = []

        spectro = spectrogram(inp, self.rate, self.length)[2].T

        for line, amp in zip(out, spectro):
            amp = np.abs(amp)
            y_fft.append(
                amp * np.exp(1j * line * np.pi)
            )

        y_fft = np.array(y_fft).T

        y.append(inverse_spectrogram(np.array(y_fft), self.length)[:len(sounds[0][0])])
    return y

def align_records(x: float, y: float, z: float,
                 microphones: list[Microphone],
                 records: list[np.ndarray],
                 sample_rate: int,
                 temperature: float = DEFAULT_TEMPERATURE,
                 source_radius: float = 0.1
                 ) -> list[np.ndarray]:
    assert len(records) == len(microphones), "microphones and records must be the same length"

    # Create all variables needed
    dist: list[float] = [euclidian_distance(x, y, z, m.x, m.y, m.z) for m in microphones]
    reversed_gain = list(map(lambda d: 1 / distance_gain(d, source_radius), dist))
    time_shift = list(map(lambda d: d / define_sound_speed(temperature), dist))

```

```
list_shift = list(map(lambda t: floor(t * sample_rate), time_shift))
```

```
# add time shift
shifted_records = [np.concatenate(
    (records[i][s:], np.zeros((s,)))
) for i, s in enumerate(list_shift)]
]
```

```
shifted_records: list = [s * g for s, g in zip(shifted_records, reversed_gain)]
```

```
return shifted_records
```

```
def func_generate_train_env(sounds_list: list[Sound],
```

```
    mics_amount: int = 10,
    mic_coordinate_interval: tuple[float, float] = (-25, 25),
    min_mics_distance: float = 2.0,

    source_amount: tuple[int, int] = (1, 5),
    source_coordinate_interval: tuple[float, float] = (-25, 25),
    min_sources_distance: float = 1.0,
```

```
    plan: bool = True,
    temperature: float = DEFAULT_TEMPERATURE) -> Callable[[], Environment]:
```

```
    """
```

```
Generate a function that will create a new environment with certain properties.
```

```
:param sounds_list: List of sound to use in the environment.
```

```
:param mics_amount: The number of microphones needed
```

```
:param mic_coordinate_interval: An [a, b] coordinate interval for microphones.
```

```
:param min_mics_distance: Minimum distance between two microphones
```

```
:param source_amount: An [a, b] amount of sources.
```

```
:param source_coordinate_interval: An [a, b] coordinate interval for sources
```

```
:param min_sources_distance: The minimum distance between two sources.
```

```
:param temperature: The temperature of the environment
```

```
:param plan: If the environment is a 2D or 3D environment.
```

```
:return: A function that generates environment.
```

```
    """
```

```
min_mic_coordinate, max_mic_coordinate = mic_coordinate_interval
```

```
min_source_coordinate, max_source_coordinate = source_coordinate_interval
min_source_amount, max_source_amount = source_amount
```

```
def generator():
    """
    This function generates a new environment.
    :return: A new environment.
    """
    # Create microphones
    mics = []

    for k in range(mics_amount):
        x1, y1, z1 = np.random.uniform(min_mic_coordinate, max_mic_coordinate, (3,))
        z1 = 0 if plan else z1

        # Check the minimum distance between microphones
        while any(map(lambda m: euclidian_distance(x1, y1, z1, *m.get_coord) < min_mics_distance, mics)):
            x1, y1, z1 = np.random.uniform(min_mic_coordinate, max_mic_coordinate, (3,))
            z1 = 0 if plan else z1

        mics.append(Microphone(f"m{k}", x1, y1, z1))

    # Create sources
    sources = []
    coordinates = []

    selected_source_amount = random.randint(min_source_amount, max_source_amount)

    for k in range(selected_source_amount):
        x, y, z = np.random.uniform(min_source_coordinate, max_source_coordinate, (3,))
        z = 0 if plan else z

        # Check the minimum distance between sources and microphones
        while (any(map(lambda c: euclidian_distance(x, y, z, *c) < min_sources_distance, coordinates)) and
              any(map(lambda m: euclidian_distance(x, y, z, *m.get_coord) < min_mics_distance, mics))):
            x, y, z = np.random.uniform(min_source_coordinate, max_source_coordinate, (3,))
            z = 0 if plan else z

        sources.append(Source(
```

```

        sound=random.choice(sounds_list),
        name=f"s{k}",
        x=x, y=y, z=z
    ))
    coordinates.append((x, y, z))

```

Create the environment

```

env = Environment(
    sources=sources,
    microphones=mics,
    rate=None,
    microphone_tensors=[],
    temperature=temperature
)

```

```

    return env

```

```

return generator

```

```

def func_dataset_generator(data_length: int,
                           sound_start_interval: tuple[float, float] = (1.0, 60.0),
                           compress: tuple[bool, bool] = (True, True),
                           noise: tuple[float, float] = (0.0025, 0.0)
                           ) -> Callable[
    [Environment, IsolateModel], tuple[list[float_array], list[float_array]]]:
    """

```

This function returns a generatr that generate a piece of dataset

```

:param data_length: Length of the dataset to generate
:param sound_start_interval: Interval of time for the dataset
:param compress: If the audio is compressed
:param noise: True to add noise in the signal
:return: A dataset generator.
    """

```

```

min_sound_start, max_sound_start = sound_start_interval

```

```

def generator(env: Environment, model: IsolateModel):
    input_number = model.input_number

```

```

start = random.uniform(min_sound_start, max_sound_start)
end = (start + data_length / env.get_rate * 2)  # * 2 because it takes the center of the data after

```

```

selected_mics = [m for m in random.sample(list(env.microphones.values()), input_number)]

```

```

_, records = env.listen_from_microphone([m.name for m in selected_mics], start, end, start - 1,
                                         attenuate=True,
                                         compress=compress[0],
                                         noise=noise[0]
                                         )

```

```

# Select a source to isolate

```

```

selected = random.choice(list(env.sources.values()))
x, y, z = selected.get_coord

```

```

shifted_records = align_records(x, y, z, selected_mics, records, env.get_rate, env.temperature)
ltu = len(shifted_records[0])
shifted_records = [sr[:ltu // 2] for sr in shifted_records]

```

```

_, result = env.listen_from_microphone((x, y, z),
                                       start, end, start - 1,
                                       attenuate=False,
                                       specific_sound=[selected.name],
                                       compress=compress[1],
                                       noise=noise[1])

```

```

result = result[0]
result = result[:ltu // 2]

```

```

return shifted_records, [result]

```

```

return generator

```

```

def train_isolate_model(model,

```

```

    generate_env_func: Callable[[], Environment],
    dataset_generator_func: Callable[[Environment, IsolateModel], tuple[Any, Any, list[Any]]],
    global_epoch: int,

```



```

    data_amount: int,
    data_diversity: int,

    batch_size: int = 1024,
    epochs: int = 1,

    evaluation_function_test: Callable[[IsolateModel, int], None] = None, # K

    ge_shift: int = 0,
    change_lr_step: tuple[tuple[int, int, float]] = ((-1, 100, 0.001), (99, 10000, 0.0001)),

    *args,
    **kwargs
) -> None:
    """
    This function is the main training loop for all model.
    :param model: Model to train.
    :param generate_env_func: Function to generate an environment.
    :param dataset_generator_func: Function to generate a dataset from an environment.
    :param global_epoch: Total number of epoch to make
    :param data_amount: Amount of data for each epoch
    :param data_diversity: Amount of different environment for each epoch
    :param batch_size: Batch size for training
    :param epochs: Number of epaoch in each training loop
    :param evaluation_function_test: Function to evaluate the model at each global epoch
    :param ge_shift: Number of the first epoch.
    :param change_lr_step: Learning rate description
    :param args: *
    :param kwargs: **
    :return: None
    """
    for ge in range(ge_shift + 1, global_epoch):
        print(f"===== Global epoch {ge:7d}/{global_epoch} =====")

        def change_lr(*a, **k) -> float | None:
            _ = a, k
            for a, b, lr in change_lr_step:
                if a <= ge < b:
                    return lr

```

```

        return None

    callback = LearningRateScheduler(change_lr)

    x_train = []
    y_train = []

    for _ in range(data_diversity):
        env = generate_env_func()

        for amount in range(data_amount // data_diversity):
            s, r = dataset_generator_func(env, model)

            x, y = model.edit_dataset(s, r)

            x_train += x
            y_train += y

    x_train = np.array(x_train)
    y_train = np.array(y_train)

    model.train_model(x_train, y_train, batch_size=batch_size, epochs=epochs, callbacks=[callback], *args,
**kwargs)

    if evaluation_function_test is not None:
        evaluation_function_test(model, ge)

def reload_train_and_evaluate(name: str,
                             path: str,
                             save_epoch: int,
                             sounds_list_eval: list[Sound],
                             rate: int = 44100,
                             ) -> tuple[Callable[[IsolateModel, int], None], int, list[float_array]]:
    """
    This function creates the evaluation environment for the model.
    And also load past information about the model (create it if the model is new).
    :param name: The name of the file where the data about the model and its training is stored.

```

```

:param path: The paths of the file where the data about the model and its training is stored.
:param save_epoch: How often model weights are saved.
:param sounds_list_eval: Liste of sound to use to evaluate model.
:param rate: Rate of records.
:return: The evaluation function, the last epoch saved, the last model saved
"""

```

```

global loaded_data

```

```

(r1, r2, r3), (real1, real2, real3), mics = env_test_isolation(sounds_list_eval)

```

```

# Create the result history file (if it already exists, load it)

```

```

if os.path.exists(f'{path}{name}.npy'):
    print("Loading existing data.")
    train_results_dict = np.load(f'{path}{name}.npy', allow_pickle=True).item()
else:
    train_results_dict = {"xm": [], "weights": []}

```

```

    np.save(f'{path}{name}.npy', deepcopy(train_results_dict))

```

```

loaded_data = np.load(f'{path}{name}.npy', allow_pickle=True).item()

```

```

# Model weights = None is no previous model saved

```

```

w = train_results_dict["weights"][-1] if len(train_results_dict["weights"]) > 0 else None

```

```

def evaluation(model: IsolateModel, n: int) -> None:

```

```

    """

```

```

    This function is used to save the performance of the model.

```

```

    :param model: Model to save.

```

```

    :param n: Epoch.

```

```

    :return: None

```

```

    """

```

```

    # Save the new model

```

```

    loaded_data["xm"].append(n)

```

```

    loaded_data["weights"].append(deepcopy(model.get_weights()))

```

```

    # Then add new and previous data to the queue to be saved

```

```

    if n % save_epoch == 0:

```

```

        saving_thread_queue.append((f'{path}{name}.npy', loaded_data))

```

Use neural network to predict the isolation of many sources

if EVALUATE:

```
o1 = isolate_source_rnn_triple(9, 5, 0, mics, [r1, r2, r3], rate, model)
o2 = isolate_source_rnn_triple(3, 3, 0, mics, [r1, r2, r3], rate, model)
o3 = isolate_source_rnn_triple(6, 2, 0, mics, [r1, r2, r3], rate, model)
```

```
a1 = absolute_error(signal=o1, original=real1, raw=True) / 0.31
a2 = absolute_error(signal=o2, original=real2, raw=True) / 0.31
a3 = absolute_error(signal=o3, original=real3, raw=True) / 0.31
q1 = relative_error(signal=o1, original=real1, raw=True)
q2 = relative_error(signal=o2, original=real2, raw=True)
q3 = relative_error(signal=o3, original=real3, raw=True)
```

```
p = lambda sample: f"{np.average(sample):.4f} ± {(np.std(sample, ddof=1) / len(sample) ** 0.5):.4f}"
```

```
graph_results.append(
    (
        np.average((a1 + a2 + a3) / 3),
        min(map(lambda x: np.average(x), (a1, a2, a3))),
        max(map(lambda x: np.average(x), (a1, a2, a3)))
    ),
    (
        np.average((q1 + q2 + q3) / 3),
        min(map(lambda x: np.average(x), (q1, q2, q3))),
        max(map(lambda x: np.average(x), (q1, q2, q3)))
    )
)
```

```
print(f"Average absolute error:      {p(a1)} | {p(a2)} | {p(a3)}")
print(f"Average relative error:     {p(q1)} | {p(q2)} | {p(q3)}")
```

Get the previous data

```
# data = np.load(f"{path}train_results {name}.npy", allow_pickle=True).item()
```

Del to avoid memory saturation (in case Python does not do it)

```
del a1, a2, a3, o1, o2, o3, q1, q2, q3
```

```
return evaluation, max(train_results_dict.get("xm", 0), default=0), w
```

```

def save_and_plot_thread(gui: bool = True) -> None:
    """
    This function, run as a thread function, saves data placed in queue
    :return: None
    """

def save_files() -> None:
    """
    This function saves data placed in saving queue.
    :return: None
    """
    # If there is data to save in the queue
    if len(saving_thread_queue) > 0:
        # Save data
        print(f"Started saving training data ({len(saving_thread_queue) - 1} remaining).")
        t = time.time()
        path, data = saving_thread_queue.pop()
        np.save(path, data)
        print(f"Training saved at {datetime.now()} in {time.time() - t:.2f} secondes.")

def update_func(i: int) -> tuple:
    # save new data
    save_files()

    if len(graph_results) < 2:
        return i,

    if not main_thread().is_alive():
        raise StopIteration

    x = list(range(len(graph_results)))

    y1 = list(map(lambda t: t[0][0], graph_results))
    y1_b = list(map(lambda t: t[0][1], graph_results))
    y1_t = list(map(lambda t: t[0][2], graph_results))

    y2 = list(map(lambda t: t[1][0], graph_results))
    y2_b = list(map(lambda t: t[1][1], graph_results))

```



```

        window.mainloop()
    except Exception as error:
        warnings.warn("-" * 76)
        warnings.warn(f"Error: {mod_name} - {error}")
        warnings.warn("-" * 76)

while main_thread().is_alive():
    time.sleep(0.1)
    save_files()

def max_batch_size(n: int, input_length: int) -> int:
    """
    This function calculate the optimal batch size depending on the number of parameters in the model
    :param n: Number of parameter of the model
    :param input_length: Input length of the RNN
    :return: Max batch size.
    """
    f = lambda p: (1 * 10 ** 4) * p + (2.25 * 10 ** 9)

    mbs_44100 = (f(n) / f(100_000)) * (input_length / (44100 // 2))

    return floor(10 / mbs_44100)

MODELS = {
    # Raw models
    "R LSTM 64": RawIsolateModel([LSTM(64, return_sequences=True)]),
    "R LSTM 128": RawIsolateModel([LSTM(128, return_sequences=True)]),
    "R LSTM 256": RawIsolateModel([LSTM(256, return_sequences=True)]),
    "R LSTM 64-64": RawIsolateModel([LSTM(64, return_sequences=True), LSTM(64, return_sequences=True)]),
    "R LSTM 128-128": RawIsolateModel([LSTM(128, return_sequences=True), LSTM(128, return_sequences=True)]),

    "R GRU 64": RawIsolateModel([GRU(64, return_sequences=True)]),
    "R GRU 128": RawIsolateModel([GRU(128, return_sequences=True)]),
    "R GRU 256": RawIsolateModel([GRU(256, return_sequences=True)]),
    "R GRU 64-64": RawIsolateModel([GRU(64, return_sequences=True), GRU(64, return_sequences=True)]),
    "R GRU 128-128": RawIsolateModel([GRU(128, return_sequences=True), GRU(128, return_sequences=True)]),

```

```

"R Bi-GRU 64": RawIsolateModel([Bidirectional(GRU(64, return_sequences=True))]),
"R Bi-GRU 128": RawIsolateModel([Bidirectional(GRU(128, return_sequences=True))]),
"R Bi-GRU 256": RawIsolateModel([Bidirectional(GRU(256, return_sequences=True))]),
"R Bi-GRU 64-64": RawIsolateModel(
    [Bidirectional(GRU(64, return_sequences=True)), Bidirectional(GRU(64, return_sequences=True))]
),
"R Bi-GRU 128-128": RawIsolateModel(
    [Bidirectional(GRU(128, return_sequences=True)), Bidirectional(GRU(128, return_sequences=True))]
),
"R Bi-LSTM 64": RawIsolateModel([Bidirectional(LSTM(64, return_sequences=True))]),
"R Bi-LSTM 128": RawIsolateModel([Bidirectional(LSTM(128, return_sequences=True))]),
"R Bi-LSTM 256": RawIsolateModel([Bidirectional(LSTM(256, return_sequences=True))]),
"R Bi-LSTM 64-64": RawIsolateModel(
    [Bidirectional(LSTM(64, return_sequences=True)), Bidirectional(LSTM(64, return_sequences=True))]
),
"R Bi-LSTM 128-128": RawIsolateModel(
    [Bidirectional(LSTM(128, return_sequences=True)), Bidirectional(LSTM(128, return_sequences=True))]
),

```

Spectrogram models

```

"S LSTM 64": SpectrogramIsolateModel([LSTM(64, return_sequences=True)]),
"S LSTM 128": SpectrogramIsolateModel([LSTM(128, return_sequences=True)]),
"S LSTM 256": SpectrogramIsolateModel([LSTM(256, return_sequences=True)]),
"S LSTM 64-64": SpectrogramIsolateModel([LSTM(64, return_sequences=True), LSTM(64, return_sequences=True)]),
"S LSTM 128-128": SpectrogramIsolateModel([LSTM(128, return_sequences=True), LSTM(128, return_sequences=True)]),

"S GRU 64": SpectrogramIsolateModel([GRU(64, return_sequences=True)]),
"S GRU 128": SpectrogramIsolateModel([GRU(128, return_sequences=True)]),
"S GRU 256": SpectrogramIsolateModel([GRU(256, return_sequences=True)]),
"S GRU 64-64": SpectrogramIsolateModel([GRU(64, return_sequences=True), GRU(64, return_sequences=True)]),
"S GRU 128-128": SpectrogramIsolateModel([GRU(128, return_sequences=True), GRU(128, return_sequences=True)]),

"S Bi-GRU 64": SpectrogramIsolateModel([Bidirectional(GRU(64, return_sequences=True))]),
"S Bi-GRU 128": SpectrogramIsolateModel([Bidirectional(GRU(128, return_sequences=True))]),
"S Bi-GRU 256": SpectrogramIsolateModel([Bidirectional(GRU(256, return_sequences=True))]),
"S Bi-GRU 64-64": SpectrogramIsolateModel(
    [Bidirectional(GRU(64, return_sequences=True)), Bidirectional(GRU(64, return_sequences=True))]
),

```



```

"S Bi-GRU 128-128": SpectrogramIsolateModel(
    [Bidirectional(GRU(128, return_sequences=True)), Bidirectional(GRU(128, return_sequences=True))]
),

"S Bi-LSTM 64": SpectrogramIsolateModel([Bidirectional(LSTM(64, return_sequences=True))]),
"S Bi-LSTM 128": SpectrogramIsolateModel([Bidirectional(LSTM(128, return_sequences=True))]),
"S Bi-LSTM 256": SpectrogramIsolateModel([Bidirectional(LSTM(256, return_sequences=True))]),
"S Bi-LSTM 64-64": SpectrogramIsolateModel(
    [Bidirectional(LSTM(64, return_sequences=True)), Bidirectional(LSTM(64, return_sequences=True))]
),
"S Bi-LSTM 128-128": SpectrogramIsolateModel(
    [Bidirectional(LSTM(128, return_sequences=True)), Bidirectional(LSTM(128, return_sequences=True))]
),
}

if __name__ == '__main__':
    RATE = 44100
    D = 2 # 2
    LEN = RATE // 2

    PATH_TRAIN: str = __file__.replace("\\", "/").replace("audio_locate_isolate/neural_networks.py", "sound_data/")
    PATH_EVAL: str = __file__.replace("\\", "/").replace("audio_locate_isolate/neural_networks.py",
"sound_data/test/")

    sounds_list_wav = [Sound(f"{PATH_TRAIN}{f}", rate=RATE) for f in os.listdir(PATH_TRAIN) if "test" not in f]
    sounds_list_eval_wav = [Sound(f"{PATH_EVAL}{f}", rate=RATE) for f in os.listdir(PATH_EVAL)]

    models_names = list(MODELS.keys())

    thread0 = Thread(target=save_and_plot_thread, daemon=True, kwargs={"gui": EVALUATE})
    thread0.start()

    for number, mod_name in enumerate(models_names):
        graph_results = []

        print("\n\n\n" + "-" * 76 * 2 + "\n\n\n" + "=" * 76)
        print(f"Training {number + 1}/{len(models_names)}: {mod_name}")
        print("=" * 76)
        mod: IsolateModel = MODELS[mod_name]

```

```

mod.summary()

max_bs = max_batch_size(mod.neural_network.count_params(), LEN) * 5
print(f"Max batch_size: {max_bs}")

env_func = func_generate_train_env(sounds_list_wav, source_amount=(3, 3))
dataset_func = func_dataset_generator(LEN)

eval_func, last_epoch, last_weights = reload_train_and_evaluate(name=mod_name,
                                                                path="res/",
                                                                save_epoch=5,
                                                                sounds_list_eval=sounds_list_eval_wav, )

if last_weights is not None:
    mod.set_weights(last_weights)

# Start saving and charting thread
print(f"Starting from: {last_epoch}")

try:
    train_isolate_model(mod, env_func, dataset_func, 251, 10 * D, 5 * D,
                        batch_size=max_bs,
                        epochs=10, # 5
                        ge_shift=last_epoch,
                        evaluation_function_test=eval_func)
except Exception as err:
    warnings.warn("-" * 76)
    warnings.warn(f"Error: {mod_name} - {err}")
    warnings.warn("-" * 76)

print("Train finished")
thread0.join()

```

audio_locate_isolate/**utils.py**

Fonctions utilitaires

```
"""
```

```
All utils functions for audio_locate_isolate
```

```
"""
```

```
from copy import deepcopy
```

```
import numpy as np
```

```
from scipy.fft import irfft
```

```
from scipy.fft import get_workers
```

```
from scipy.fft import rfft
```

```
from scipy.fft import rfftfreq
```

```
from environment import Environment
```

```
from environment import Microphone
```

```
from environment import Source
```

```
from environment import Sound
```

```
float_array = np.ndarray # Type for float array
```

```
def normalize_array(arr: float_array, a: float, b: float):
```

```
    """
```

```
    Normalize an array : from [min(arr), max(arr)] to [a, b].
```

```
    :param arr: An array to normalize.
```

```
    :param a: Minimum value.
```

```
    :param b: Maximum value.
```

```
    :return: Normalized array.
```

```
    """
```

```
    try:
```

```
        arr_min = np.min(arr)
```

```
        arr_max = np.max(arr)
```

```
    except Exception as e:
```

```
        print(e)
```

```
        print(arr)
```

```
        quit()
```

```
    # Avoid dividing by 0
```

```
    if arr_min == arr_max:
```

```
        return np.full_like(arr, a)
```

```
    normalized_arr = (arr - arr_min) / (arr_max - arr_min)
```

```
scaled_arr = a + (normalized_arr * (b - a))
```

```
return scaled_arr
```

```
def spectrogram(x: np.array, fs: float, length: int, workers: int = None) -> tuple[np.ndarray, np.ndarray, np.ndarray]:  
    """
```

This function makes the real spectrogram of a signal : it splits the signal into chunks of length 'length', and then realizes the real fft on each chunk and returns all real fft.

Note : This function adds a window of Hamming to avoid border problems.

:param x: The signal to transform into spectrogram.

:param fs: The sample rate of the signal.

:param length: The length of chunks in the spectrogram.

:param workers: Number of CPU cores to use to calculate spectrogram.

:return: The spectrogram of the signal.

"""

Add 0 at the end to fit the size

```
x = deepcopy(x)
```

```
if len(x) % length != 0:
```

```
    x = np.concatenate((x, np.zeros((length - len(x) % length,))))
```

```
# hamming_window = np.hamming(length)
```

```
parts = np.array(  
    [x[i * length:(i + 1) * length] for i in range(len(x) // length)]  
)
```

```
y = rfft(parts, n=length, workers=get_workers() if workers is None else workers).T
```

```
t = np.array([length * i / fs for i in range(len(x) // length)])
```

```
f = rfftfreq(length, 1 / fs)
```

```
return t, f, y
```

```
def inverse_spectrogram(y: np.ndarray, length: int, workers: int = None) -> np.ndarray:  
    """
```

This function if the reverse of the 'spectrogram' function: it transforms a spectrogram into a signal.

:param y: The spectrogram.

```

:param length: The length of chunks used.
:param workers: Number of workers to use to calculate spectrogram.
:return: The original signal.
"""

```

```

# hamming_window = np.hamming(length)

```

```

r: np.ndarray = irfft(y.T, n=length, workers=get_workers() if workers is None else workers)
x = np.array([l for l in r]).flatten()

```

```

return x

```

```

def absolute_error(signal: float_array, original: float_array, raw: bool = False) -> float_array | float:
    e = np.abs(normalize_array(signal, -1, 1) - normalize_array(original, -1, 1))
    if raw:
        return e

    else:
        return e / len(signal)

```

```

def relative_error(signal: float_array, original: float_array, raw: bool = False) -> float_array | float:
    e = np.abs((signal - original) / (np.abs(original) + 1))

    if raw:
        return e

    else:
        return e / len(signal)

```

```

def env_test_isolation(sound_list: list[Sound], length: float = 5.0, rate = 44100) :
    m1 = Microphone("m1", 0, 10, 0)
    m2 = Microphone("m2", 0, 0, 0)
    m3 = Microphone("m3", 5, 8.66, 0)

    s1 = Source(sound_list[0], "s1", 9, 5, 0)
    s2 = Source(sound_list[1], "s2", 3, 3, 0)
    s3 = Source(sound_list[2], "s3", 6, 2, 0)

```

```

env = Environment([s1, s2, s3], [m1, m2, m3], rate=rate, microphone_tensors=[])

start = 50.0
end = start + length

_, (r1, r2, r3) = env.listen_from_microphone(["m1", "m2", "m3"], start, end, start - 1,
                                             attenuate=True,
                                             compress=True,
                                             noise=0.0025
                                             )
_, (real1,) = env.listen_from_microphone((9, 5, 0), start, end, start - 1,
                                         specific_sound=["s1"],
                                         attenuate=False)
_, (real2,) = env.listen_from_microphone((3, 3, 0), start, end, start - 1,
                                         specific_sound=["s2"],
                                         attenuate=False)
_, (real3,) = env.listen_from_microphone((6, 2, 0), start, end, start - 1,
                                         specific_sound=["s3"],
                                         attenuate=False)

return (r1, r2, r3), (real1, real2, real3), [m1, m2, m3]

```

chart

Etudes scientifiques | Graphiques

chart/**0_presentation.ipynb**

Illustration de la simulation

```

import os

from environnement import Environment
from environnement import Microphone
from environnement import MicrophoneTensor
from environnement import Sound
from environnement import Source

import numpy as np
import plotly.graph_objects as go
import plotly.io as pio

pio.renderers.default = "browser"
PATH_EVAL: str = "C:/Users/Utilisateur/Documents/Python/ADVANCED/GeoSoundTIPE/sound_data/test/"
import random

RATE = 44100
sounds_list_eval = [Sound(lambda x: np.cos(x * 1000 + random.randint(0, 10))), rate=RATE, start=0, end=10) for f in
    os.listdir(PATH_EVAL)]

m1 = Microphone("m1", 10, 10, 0)
m2 = Microphone("m2", 5, 7, 0)
m3 = Microphone("m3", 9, 3, 0)

s1 = Source(sounds_list_eval[0], "s1", 7, 6, 0)
s2 = Source(sounds_list_eval[1], "s2", 9, 8, 0)
s3 = Source(sounds_list_eval[2], "s3", 8.5, 6.5, 0)

tens3d = MicrophoneTensor(
    "3D",
    np.linspace(4, 11, 50),
    np.linspace(4, 11, 50),
    np.linspace(-3, 3, 50)
)

tens2d = MicrophoneTensor(
    "2D",
    np.linspace(-10, 10, 500),
    np.linspace(-10, 10, 500),

```

```

    np.linspace(0, 0, 1)
)

environment = Environment([s1, s2, s3], [m1, m2, m3], rate=RATE, microphone_tensors=[tens3d, tens2d])

temps, lr = environment.listen_from_microphone(["m1", "m2", "m3"], 60, 65, 59, True)
temps = temps - 60
t = environment.render_frame_tenor("3D", render_time=25150 / RATE, attenuate=False)

volume = go.Volume(
    x=environment.microphone_tensors["3D"].mesh_x.flatten(),
    y=environment.microphone_tensors["3D"].mesh_y.flatten(),
    z=environment.microphone_tensors["3D"].mesh_z.flatten(),
    value=t.flatten(),
    isomin=-0.75,
    isomax=0.75,
    cmax=1,
    cmin=-1,
    opacity=0.1,
    opacityscale=[[0, 1], [0.499999999, 1], [0.5, 0], [0.50000001, 1], [1, 1]],
    colorscale='picnic',
    colorbar={'orientation': 'h', 'y': -0.15}
)

scatter = go.Scatter3d(
    x=[s.x for s in environment.sources.values()],
    y=[s.y for s in environment.sources.values()],
    z=[s.z for s in environment.sources.values()],
    mode="markers",
    marker=dict(color=["black" for _ in range(3)], ),
    showlegend=False,
    name="Sources",
)

fig = go.Figure(data=[scatter, volume])
fig.update_layout(
    template='plotly_white',
    legend=dict(
        yanchor="top",

```

```

        y=0.99,
        xanchor="left",
        x=0.025,
        bordercolor="black",
        borderwidth=1),
    scene_camera=dict(
        up=dict(x=0, y=0, z=1),
        center=dict(x=0, y=0, z=-0.10),
        eye=dict(x=1.275, y=1.275, z=1.275)
    ),
    margin=dict(
        l=0,
        r=0,
        b=0,
        t=0,
    ),
    autosize=True, #
    title=dict(text="$\\Large{\\text{Intensité sonore (normalisée) à } t = 0.5703s}$", automargin=True,
    yref='paper'),
    xaxis=dict(visible=False, showgrid=False, showline=False),
    yaxis=dict(visible=False, showgrid=False, showline=False),
)

# fig.for_each_coloraxis(lambda x: x['colorbar'].update({'orientation':'h', 'thickness':20, 'y': -1.0}))
fig.write_image("results/0 fig1 lr.webp", scale=1.3 * 2, height=600, width=600)
fig.write_image("results/0 fig1 hr.png", scale=13, height=600, width=600)

```

chart/3_illustrate_locate.ipynb

Illustration des méthodes de localisation

```

import os

from environnement import Environment
from environnement import Microphone
from environnement import MicrophoneTensor
from environnement import Sound
from environnement import Source
from environnement import euclidian_distance

import numpy as np
import plotly.graph_objects as go
import plotly.io as pio
from plotly.subplots import make_subplots
from scipy import signal

from chart_utils import cercle_circonscrit

PATH_EVAL: str = "C:/Users/Utilisateur/Documents/Python/ADVANCED/GeoSoundTIPE/sound_data/test/"

RATE = 44100
sounds_list_eval = [Sound(f"{PATH_EVAL}{f}", rate=RATE) for f in os.listdir(PATH_EVAL)]

m1 = Microphone("m1", 10, 10, 0)
m2 = Microphone("m2", 5, 7, 0)
m3 = Microphone("m3", 9, 3, 0)

s1 = Source(sounds_list_eval[0], "s1", 7, 6, 0)
s2 = Source(sounds_list_eval[1], "s2", 9, 8, 0)
s3 = Source(sounds_list_eval[2], "s3", 8.5, 6.5, 0)

tens3d = MicrophoneTensor(
    "3D",
    np.linspace(-1.5, 1.5, 50),
    np.linspace(-1.5, 1.5, 50),
    np.linspace(-1.5, 1.5, 50)
)

tens2d = MicrophoneTensor(
    "2D",

```

```

    np.linspace(-10, 10, 500),
    np.linspace(-10, 10, 500),
    np.linspace(0, 0, 1)
)

environment = Environment([s1, s2, s3], [m1, m2, m3], rate=RATE, microphone_tensors=[tens3d, tens2d])

temps, lr = environment.listen_from_microphone(["m1", "m2", "m3"], 60, 65, 59, True)
temps = temps - 60

#####
###
# # Illustrate simulation

pio.renderers.default = "browser"

O = cercle_circonscrit(*[m.get_coord[0:2] for m in [m1, m2, m3]])

mx = [m.get_coord[0] for m in [m1, m2, m3]]
my = [m.get_coord[1] for m in [m1, m2, m3]]

sx = [s.get_coord[0] for s in [s1, s2, s3]]
sy = [s.get_coord[1] for s in [s1, s2, s3]]

fig = go.Figure(data=[go.Scatter(x=mx,
                                y=my,
                                mode='markers+text',
                                marker_line_color=["black" for _ in range(3)],
                                marker=dict(color=["lime" for _ in range(3)],
                                              size=[10 for _ in range(3)]),
                                marker_symbol=["circle" for _ in range(3)],
                                showlegend=True,
                                name="$\\Large{\\text{Microphones}}$",
                                text=["$\\Large{M_1}$", "$\\Large{M_2}$", "$\\Large{M_3}$"],
                                textposition="bottom center"
                                ),
                    go.Scatter(x=sx,
                                y=sy,

```

```

mode='markers+text',
marker_line_color=["black" for _ in range(3)],
marker=dict(color=["blue" for _ in range(3)],
            size=[10 for _ in range(3)]),
marker_symbol=['pentagon' for _ in range(3)],
showlegend=True,
name="$\\Large{\\text{Sources}}$",
text=["$\\Large{S_1}$", "$\\Large{S_2}$", "$\\Large{S_3}$"],
textposition="bottom center"
),

])

# fig.show(config={'scrollZoom': True})

# Save image compressed
fig.update_xaxes(scaleanchor='x',
                linewidth=1,
                linecolor='black',
                mirror=True,
                ticks='outside',
                showline=True,
                showgrid=False,
                scaleratio=1
                )
fig.update_yaxes(scaleanchor='y',
                linewidth=1,
                linecolor='black',
                mirror=True,
                ticks='outside',
                showline=True,
                showgrid=False,
                scaleratio=1)

fig.update_layout(
    template='plotly_white',
    legend=dict(
        yanchor="top",
        y=0.99,
        xanchor="left",

```



```

        x=0.025,
        bordercolor="black",
        borderwidth=1,
    ),
    xaxis={"title": {"text": r"$\Large{\text{Position } x \text{ en } m}$"},
    yaxis={"title": {"text": r"$\Large{\text{Position } y \text{ en } m}$"},
)

fig.write_image("results/3 fig1 lr.webp", scale=1.3 * 1.5, height=600, width=600)
fig.write_image("results/3 fig1 hr.png", scale=13, height=600, width=600)

#####
###
# # Illustrate timeshift

lags = signal.correlation_lags(len(lr[0]), len(lr[1])) / RATE

fig = make_subplots(rows=3, cols=2,
                    subplot_titles=tuple(
                        ["$\large{M_{d}} \text{ et } M_{d} \text{ %s}$" % (j + 1, i + 1,
                            "(agrandi)" if k == 1 else "") for
                        (i, j, k) in [(i, j, k) for j in range(3) for i in range(3) if j < i for k in range(2)]]),
                    vertical_spacing=0.125,
                    )

# Correlation

for i, s in enumerate([signal.correlate(lr[0], lr[1]), signal.correlate(lr[0], lr[2]), signal.correlate(lr[1],
lr[2])]):
    fig.add_trace(go.Scatter(
        x=lags,
        y=s,
        line=dict(color='#4472C4'),
        showlegend=False
    ),
    row=i + 1,
    col=1
)

```

```

# Correlation centred
C = len(lags) // 2 # center
WS = 5000 # window size

for i, s in enumerate([signal.correlate(lr[0], lr[1]), signal.correlate(lr[0], lr[2]), signal.correlate(lr[1],
lr[2])]):
    fig.add_trace(
        go.Scatter(
            x=lags[C - WS:C + WS],
            y=s[C - WS:C + WS],
            line=dict(color='#4472C4'),
            showlegend=False,
        ),
        row=i + 1,
        col=2)

    fig.add_trace(go.Scatter(
        mode='markers',
        marker_symbol="cross",
        x=[lags[np.argmax(s)]],
        y=[s[np.argmax(s)]],
        showlegend=True if i == 0 else False,
        marker=dict(color="red", size=7.5),
        name="Pic de corrélation"
    ),
        row=i + 1,
        col=2)

for c in range(1, 2 + 1):
    for r in range(1, 3 + 1):
        fig.update_xaxes(linewidth=1,
                        color="black",
                        linecolor='black',
                        mirror=True,
                        ticks='outside',
                        showline=True,
                        showgrid=False,
                        zerolinecolor="black",
                        row=r,

```

```

        col=c)
fig.update_yaxes(linewidth=1,
                 color="black",
                 linecolor='black',
                 mirror=True,
                 ticks='outside',
                 showline=True,
                 showgrid=False,
                 row=r,
                 col=c)

fig.update_layout(yaxis=dict(scaleanchor='y', scaleratio=1, showgrid=False),
                  xaxis=dict(scaleanchor='x', scaleratio=1, showgrid=False),
                  showlegend=True,
                  title_text="$\\Large{\\text{Corrélations croisées entre les enregistrements des microphones}}$",
                  template='plotly_white',
                  margin=dict(l=20, r=20),
                  legend=dict(
                      yanchor="top",
                      y=0.995,
                      xanchor="left",
                      x=0.5525,
                      bordercolor="black",
                      borderwidth=1),
                  )

fig.update_xaxes(title={"text": r"$\\Large{\\text{Temps en } s }$",
                      row=3,
                      col=1})
fig.update_xaxes(title={"text": r"$\\Large{\\text{Temps en } s }$",
                      row=3,
                      col=2})

# fig.show()

fig.write_image("results/3 fig2 lr.webp", scale=1.3 * 2, height=600, width=900)
fig.write_image("results/3 fig2 hr.png", scale=13, height=600, width=900)

```

```
#####
```

```

###
# # Illustration peaks method

lags = signal.correlation_lags(len(lr[0]), len(lr[1])) / RATE
C = len(lags) // 2 # center
WS = 2000 # window size
s = signal.correlate(lr[0], lr[1])
n_peaks = 4

fig = make_subplots(rows=2, cols=1,
                    subplot_titles=("", "(agrandi)"),
                    vertical_spacing=0.125,
                    )

fig.add_trace(go.Scatter(
    x=lags,
    y=s,
    line=dict(color='#4472C4'),
    showlegend=False
),
    row=1,
    col=1
)

fig.add_trace(
    go.Scatter(
        x=lags[C - WS:C + WS],
        y=s[C - WS:C + WS],
        line=dict(color='#4472C4'),
        showlegend=False,
    ),
    row=2,
    col=1)

peaks, _ = signal.find_peaks(s, distance=50)
peaks: np.ndarray = peaks
c: np.ndarray = np.argpartition(s[peaks], -n_peaks)[-n_peaks:]

fig.add_trace(go.Scatter(

```

```

mode='markers',
marker_symbol="cross",
x=lags[peaks[c]],
y=s[peaks[c]],
showlegend=True,
name="Pics de corrélation",
marker=dict(color="red", size=7.5),
),
row=2,
col=1
)

# fig.show(config={'scrollZoom': True})

# Save image compressed
for i in range(2):
    fig.update_xaxes(linewidth=1,
                    linecolor='black',
                    mirror=True,
                    ticks='outside',
                    showline=True,
                    showgrid=False,
                    row=i + 1,
                    zerolinecolor="black",
                    col=1)
    fig.update_yaxes(linewidth=1,
                    linecolor='black',
                    mirror=True,
                    ticks='outside',
                    showline=True,
                    showgrid=False,
                    row=i + 1,
                    col=1)

fig.update_xaxes(title={"text": r"$\Large{\text{Temps en } s}$"},
                row=2,
                col=1)

fig.update_layout(template='plotly_white',

```

```

        title={
            "text": "$\\Large{\\text{Corrélation croisée entre les enregistrements}} \\\\
\\Large{\\text{des microphones } M_1 \\text{ et } M_2}\\$"
        },
        yaxis=dict(scaleanchor='y', scaleratio=1, showgrid=False),
        xaxis=dict(scaleanchor='x', scaleratio=1, showgrid=False),
        margin=dict(l=20, r=20),
        legend=dict(
            yanchor="top",
            y=0.425,
            xanchor="left",
            x=0.025,
            borderwidth=1
        ),
    )

# fig.show(config={'scrollZoom': True})

fig.write_image("results/3 fig3 lr.webp", scale=1.3, height=600, width=600)
fig.write_image("results/3 fig3 hr.png", scale=13, height=600, width=600)

#####
###
# # Illustration spectrogram method

from audio_locate_isolate.utils import spectrogram, inverse_spectrogram

LENGTH = 32
fig = make_subplots(rows=4, cols=1,
                    subplot_titles=["$\\Large{\\text{%s}}\\$" % s for s in (
                        "Signal original", "Spectrogramme original", "Spectrogramme filtré à fréquence unique",
                        "Signal reconstruit à fréquence unique")
                    ],
                    vertical_spacing=0.1,
                    )

# Signal original
fig.add_trace(
    go.Scatter(

```

```

        x=temps,
        y=lr[0],
        line=dict(color='#4472C4'),
        showlegend=False
    ),
    row=1,
    col=1
)

# Spectrogram original

t, f, s = spectrogram(lr[0], length=LENGTH, fs=RATE)

fig.add_trace(
    go.Heatmap(
        x=t,
        y=f,
        z=np.log(np.abs(s) + 0.001),
        colorscale="Viridis",
        showscale=False
    ),
    row=2,
    col=1
)

# Spectrogram isolé

fig.add_trace(
    go.Heatmap(
        x=t,
        y=f,
        z=np.log(np.abs(s) + 0.001),
        opacity=0.20,
        colorscale="Viridis",
        showscale=False
    ),
    row=3,

```

```

        col=1,
    )

    isolate_s = s
    isolate_s[np.meshgrid(t, f)[1] != f[10]] = np.nan

    fig.add_trace(
        go.Heatmap(
            x=t,
            y=f,
            z=np.log(np.abs(isolate_s) + 0.001),
            colorscale="Viridis",
            showscale=False
        ),
        row=3,
        col=1,
    )

# Signal isolé

    isolate_s = s
    isolate_s[np.meshgrid(t, f)[1] != f[10]] = 0

    fig.add_trace(
        go.Scatter(
            x=temps,
            y=inverse_spectrogram(isolate_s, LENGTH),
            line=dict(color='#4472C4'),
            showlegend=False
        ),
        row=4,
        col=1,
    )

    for i in range(4):
        fig.update_xaxes(
            linewidth=1,
            linecolor='black',
            mirror=True,
            ticks='outside',

```



```

        showline=True,
        showgrid=False,
        row=i + 1,
        col=1)
fig.update_yaxes(linewidth=1,
                 linecolor='black',
                 mirror=True,
                 showticklabels=False,
                 showline=True,
                 showgrid=False,
                 row=i + 1,
                 col=1)

fig.update_layout(template='plotly_white',
                 yaxis=dict(scaleanchor='y', scaleratio=1, showgrid=False),
                 xaxis=dict(scaleanchor='x', scaleratio=1, showgrid=False),
                 margin=dict(l=20, r=20, t=25, b=20),
                 legend=dict(
                     yanchor="top",
                     y=0.425,
                     xanchor="left",
                     x=0.025,
                     borderwidth=1
                 ),
                 )
fig.update_coloraxes(showscale=False)

fig.update_xaxes(title={"text": r"$\Large{\text{Temps en } s }$"},
                 row=4,
                 col=1)

# fig.show()

fig.write_image("results/3 fig4 lr.webp", scale=1.3, height=600, width=600)
fig.write_image("results/3 fig4 hr.png", scale=13, height=600, width=600)

```

```

#####
###

```

```

from audio_locate_isolate.utils import spectrogram, inverse_spectrogram

lags = signal.correlation_lags(len(lr[0]), len(lr[1])) / RATE
C = len(lags) // 2 # center
WS = 500 # window size

fig = make_subplots(
    rows=2, cols=2,
    specs=[[{}], {}],
    [{"colspan": 2}, None]],
    subplot_titles=(
        "$\\large{\\text{Signaux reconstruits à fréquence unique } \\quad \\text{de } M_1 \\text{ pour toutes les fréquences}}$",
        "$\\large{\\text{Signaux reconstruits à fréquence unique } \\quad \\text{de } M_2 \\text{ pour toutes les fréquences}}$",
        "$\\large{\\text{Corrélation croisée entre signaux reconstruits de même fréquence (Normalisé individuellement) }}$",
        vertical_spacing=0.1,
    )

t, freq, s0 = spectrogram(lr[0], length=LENGTH, fs=RATE)
_, _, s1 = spectrogram(lr[1], length=LENGTH, fs=RATE)
mask = np.array([freq for _ in range(len(t))]).T
l_s0 = [inverse_spectrogram(np.where(mask != f, 0, s0), LENGTH, workers=12) for f in freq[2:]]
l_s1 = [inverse_spectrogram(np.where(mask != f, 0, s1), LENGTH, workers=12) for f in freq[2:]]

for l in l_s0:
    fig.add_trace(
        go.Scatter(
            x=temps,
            y=l,
            line=dict(color='#4472C4'),
            showlegend=False,
            opacity=0.25
        ),
        row=1,
        col=1
    )

```

```

for l in l_s1:
    fig.add_trace(
        go.Scatter(
            x=temps,
            y=l,
            line=dict(color='#4472C4'),
            showlegend=False,
            opacity=0.5
        ),
        row=1,
        col=2
    )

```

```

min_c = float("+inf")
max_c = float("-inf")

```

```

for a, b in zip(l_s0, l_s1):
    s = signal.correlate(a, b)
    norm = np.max(np.abs(s))

    fig.add_trace(
        go.Scatter(
            x=lags[C - WS:C + WS],
            y=s[C - WS:C + WS] / norm,
            line=dict(color='#4472C4'),
            showlegend=False,
            opacity=0.25
        ),
        row=2,
        col=1)

```

```

leg = True

```

```

for a, b in zip(l_s0, l_s1):
    s = signal.correlate(a, b)
    norm = np.max(np.abs(s))

    fig.add_trace(go.Scatter(
        mode='markers',

```

```

        marker_symbol="cross",
        x=[lags[np.argmax(s)]],
        y=[s[np.argmax(s)] / norm],
        showlegend=leg,
        marker=dict(color="red", size=7.5),
        name="$\\large{\\text{Pics de corrélation}}$"
    ),
    row=2,
    col=1)
leg = False

for (i, j) in [(1, 1), (1, 2), (2, 1)]:
    fig.update_xaxes(linewidth=1,
                     linecolor='black',
                     mirror=True,
                     ticks='inside',
                     showline=True,
                     showgrid=False,
                     zerolinecolor="black",
                     row=i,
                     col=j)
    fig.update_yaxes(linewidth=1,
                     linecolor='black',
                     mirror=True,
                     showticklabels=False if (i, j) != (2, 1) else True,
                     showline=True,
                     showgrid=False,
                     row=i,
                     col=j)

fig.update_layout(template='plotly_white',
                  yaxis=dict(scaleanchor='y', scaleratio=1, showgrid=False),
                  xaxis=dict(scaleanchor='x', scaleratio=1, showgrid=False),
                  margin=dict(l=20, r=20, t=50, b=20),
                  legend=dict(
                      yanchor="top",
                      y=0.425,
                      xanchor="left",
                      x=0.0185,

```

```

        bordercolor="black",
        borderwidth=1
    ),
)

fig.write_image("results/3 fig5 lr.webp", scale=1.3, height=600, width=900)
fig.write_image("results/3 fig5 hr.png", scale=13, height=600, width=900)

#####
###
# # Illustration gradient descent

from environment import define_sound_speed
from scipy.optimize import minimize

O = cercle_circonscrit(*[m.get_coord[0:2] for m in [m1, m2, m3]])

lags = signal.correlation_lags(len(lr[0]), len(lr[1])) / RATE
ts = []
t_range = [(0, 1), (0, 2), (1, 2)]

for i, s in enumerate([signal.correlate(lr[0], lr[1]), signal.correlate(lr[0], lr[2]), signal.correlate(lr[1],
lr[2])]):
    ts.append(lags[np.argmax(s)])

ts = np.array(ts) * define_sound_speed()

x = np.linspace(2, 11, 1000, endpoint=True)
y = np.linspace(2, 11, 1000, endpoint=True)

X, Y = np.meshgrid(x, y)

def euclidian_distance(x0, y0, z0, x1, y1, z1) -> float | np.ndarray:
    return ((x0 - x1) ** 2 + (y0 - y1) ** 2 + (z0 - z1) ** 2) ** 0.5

microphones = [m1, m2, m3]

```

```

def error_function(M):
    cx, cy = M
    error = np.zeros(cx.shape)
    k = 0
    for i, j in t_range:
        di = ((cx - microphones[i].get_coord[0]) ** 2 + (cy - microphones[i].get_coord[1]) ** 2) ** 0.5
        dj = ((cx - microphones[j].get_coord[0]) ** 2 + (cy - microphones[j].get_coord[1]) ** 2) ** 0.5
        delta = (di - dj)

        error += np.abs(delta - ts[k])
        k += 1

    return error / k

Z = error_function((X, Y))

steps = [0]

minimize(error_function, x0=np.array([0[0], 0[1]]), callback=lambda x: steps.append(x), method="Nelder-Mead")

steps = np.array(steps)

fig = go.Figure(data=[
    go.Contour(x=x, y=y, z=Z,
               line_smoothing=0.85,
               ncontours=35 * int(1 / 0.65) * 2,
               colorscale="Reds",
               zmax=1 / 0.65 * np.max(Z),
               zmin=0),
    go.Scatter(
        x=steps[:, 0],
        y=steps[:, 1],
        line=dict(color='royalblue', width=2),
        mode="lines",
        showlegend=True,
        name="$\\large{\\text{Recherche minimum}}$"
    ),

```

```

go.Scatter(x=[0[0]],
           y=[0[1]],
           mode='markers',
           marker_line_color=["black"],
           marker=dict(color=["red"],
                       size=[10]),
           marker_symbol=["x"],
           showlegend=True,
           name="$\\large{\\text{Barycentre}}$",
           textposition="bottom center",
           opacity=1,
           ),
go.Scatter(x=mx,
           y=my,
           mode='markers+text',
           marker_line_color=["black" for _ in range(3)],
           marker=dict(color=["lime" for _ in range(3)],
                       size=[10 for _ in range(3)]),
           marker_symbol=["circle" for _ in range(3)],
           showlegend=True,
           name="$\\large{\\text{Microphones}}$",
           text=["$\\Large{M_1}$", "$\\Large{M_2}$", "$\\Large{M_3}$"],
           textposition="bottom center",
           opacity=1,
           ),
go.Scatter(x=sx,
           y=sy,
           mode='markers+text',
           marker_line_color=["black" for _ in range(3)],
           marker=dict(color=["blue" for _ in range(3)],
                       size=[10 for _ in range(3)]),
           marker_symbol=['pentagon' for _ in range(3)],
           showlegend=True,
           name="$\\large{\\text{Sources}}$",
           text=["$\\Large{S_1}$", "$\\Large{S_2}$", "$\\Large{S_3}$"],
           textposition="bottom center",
           opacity=1,
           ),

```

```

],
)

# fig.show(config={'scrollZoom': True})

# Save image compressed
fig.update_xaxes(scaleanchor='x',
                 linewidth=1,
                 linecolor='black',
                 mirror=True,
                 ticks='outside',
                 showline=True,
                 showgrid=False,
                 scaleratio=1)
fig.update_yaxes(scaleanchor='y',
                 linewidth=1,
                 linecolor='black',
                 mirror=True,
                 ticks='outside',
                 showline=True,
                 showgrid=False,
                 scaleratio=1)

fig.update_layout(
    template='plotly_white',
    legend=dict(
        yanchor="top",
        y=0.99,
        xanchor="left",
        x=0.025,
        bordercolor="black",
        borderwidth=1),
    title={"text": r"$\LARGE{\text{Erreurs à minimiser en } m}$"},
    xaxis={"title": {"text": r"$\LARGE{\text{Position } x \text{ en } m}$"}},
    yaxis={"title": {"text": r"$\LARGE{\text{Position } y \text{ en } m}$"}},
)

fig.write_image("results/3 fig6 lr.webp", scale=1.3 * 2, height=600, width=600)
fig.write_image("results/3 fig6 hr.png", scale=13, height=600, width=600)

```



```
#####
###
# # Illustration trilateration

def equation_liant_deux_points(p1, p2, distance_shift):
    x, y = symbols('x y')
    a, b = p1
    c, d = p2

    equation = Eq(
        (sqrt((x - a) ** 2 + (y - b) ** 2) - sqrt((x - c) ** 2 + (y - d) ** 2)) - distance_shift, 0
    )

    solutions = solve(equation, y)

    return solutions

s1 = Source(sounds_list_eval[0], "s1", 7, 6, 0)
s2 = Source(sounds_list_eval[1], "s2", 9, 8, 0)
s3 = Source(sounds_list_eval[2], "s3", 8.5, 6.5, 0)

ds12 = euclidian_distance(*m1.get_coord, *s1.get_coord) - euclidian_distance(*m2.get_coord, *s1.get_coord)
ds23 = euclidian_distance(*m2.get_coord, *s1.get_coord) - euclidian_distance(*m3.get_coord, *s1.get_coord)
ds31 = euclidian_distance(*m3.get_coord, *s1.get_coord) - euclidian_distance(*m1.get_coord, *s1.get_coord)

sol1_12 = equation_liant_deux_points(m1.get_coord[:2], m2.get_coord[:2], ds12)
sol1_23 = equation_liant_deux_points(m2.get_coord[:2], m3.get_coord[:2], ds23)
sol1_31 = equation_liant_deux_points(m3.get_coord[:2], m1.get_coord[:2], ds31)

pio.renderers.default = "browser"

O = cercle_circonscribit(*[m.get_coord[0:2] for m in [m1, m2, m3]])

mx = [m.get_coord[0] for m in [m1, m2, m3]]
my = [m.get_coord[1] for m in [m1, m2, m3]]

sx = [s.get_coord[0] for s in [s1, s2, s3]]

```

```

sy = [s.get_coord[1] for s in [s1, s2, s3]]

line_x = np.linspace(0, 100, 1000)

fig = go.Figure(data=[
    go.Scatter(x=line_x,
               y=eval(repr(sol1_12[0]).replace("x", "line_x").replace("sqrt", "np.sqrt")),
               mode='lines',
               name="$\\large{\\text{Trilateration}}$",
               textposition="bottom center",
               line=dict(color='crimson'),
               showlegend=True,
               ),
    go.Scatter(x=line_x,
               y=eval(repr(sol1_23[1]).replace("x", "line_x").replace("sqrt", "np.sqrt")),
               mode='lines',
               textposition="bottom center",
               line=dict(color='crimson'),
               showlegend=False,
               ),
    go.Scatter(x=line_x,
               y=eval(repr(sol1_31[0]).replace("x", "line_x").replace("sqrt", "np.sqrt")),
               mode='lines',
               textposition="bottom center",
               line=dict(color='crimson'),
               showlegend=False,
               ),

    go.Scatter(x=mx,
               y=my,
               mode='markers+text',
               marker_line_color=["black" for _ in range(3)],
               marker=dict(color=["lime" for _ in range(3)],
                           size=[10 for _ in range(3)]),
               marker_symbol=["circle" for _ in range(3)],
               showlegend=True,
               name="$\\large{\\text{Microphones}}$",
               text=["$\\Large{M_1}$", "$\\Large{M_2}$", "$\\Large{M_3}$"],
               textposition="bottom center"
    ])

```

```

    ),
    go.Scatter(x=sx,
               y=sy,
               mode='markers+text',
               marker_line_color=["black" for _ in range(3)],
               marker=dict(color=["blue" for _ in range(3)],
                           size=[10 for _ in range(3)]),
               marker_symbol=['pentagon' for _ in range(3)],
               showlegend=True,
               name="$\\large{\\text{Sources}}$",
               text=["$\\Large{S_1}$", "$\\Large{S_2}$", "$\\Large{S_3}$"],
               textposition="bottom center"
    ),
])

# fig.show(config={'scrollZoom': True})

# Save image compressed
fig.update_xaxes(scaleanchor='x',
                 linewidth=1,
                 linecolor='black',
                 mirror=True,
                 ticks='outside',
                 showline=True,
                 showgrid=False,
                 scaleratio=1,
                 range=[min(mx) * 0.9, max(mx) * 1.1])
fig.update_yaxes(scaleanchor='y',
                 linewidth=1,
                 linecolor='black',
                 mirror=True,
                 ticks='outside',
                 showline=True,
                 showgrid=False,
                 scaleratio=1,
                 range=[min(my) * 0.85, max(my) * 1.1])

fig.update_layout(
    template='plotly_white',

```

```

legend=dict(
    yanchor="top",
    y=0.99,
    xanchor="left",
    x=0.025,
    bordercolor="black",
    borderwidth=1),
xaxis={"title": {"text": r"$\Large{\text{Position } x \text{ en } m}$"}},
yaxis={"title": {"text": r"$\Large{\text{Position } y \text{ en } m}$"}}
)

```

```

fig.write_image("results/3 fig7 lr.webp", scale=1.3, height=600, width=600)
fig.write_image("results/3 fig7 hr.png", scale=13, height=600, width=600)

```

```

#####
###

```

Trilateration formula

```

import sympy
from sympy import symbols, Eq, solve, sqrt
from IPython.display import display, Math
from itertools import product
import time

```

```

x, y = symbols('x y')

```

```

a, b = (0, 0) # symbols('a b')

```

```

x1, y1 = symbols('x1 y1')

```

```

x2, y2 = symbols('x2 y2')

```

```

t1 = symbols('t1')

```

```

t2 = symbols('t2')

```

```

t = symbols('t')

```

```

equation1 = Eq((sqrt(x ** 2 + y ** 2) - sqrt((x - x1) ** 2 + (y - y1) ** 2)) - t1, 0)

```

```

equation2 = Eq((sqrt(x ** 2 + y ** 2) - sqrt((x - x2) ** 2 + (y - y2) ** 2)) - t2, 0)

```

```

display(Math(r'\text{Equation 1: } %s' % sympy.latex(equation1)))

```

```

display(Math(r'\text{Equation 2: } %s' % sympy.latex(equation2)))

```

```

solutions1 = solve(equation1, y, check=True, simplify=True)
solutions2 = solve(equation2, y, check=True, simplify=True)

for i, sol in enumerate(solutions1):
    display(Math(r'\text{Solution equation 1, %d: } %s' % (i, sympy.latex(sol))))
    print(sol)

for i, sol in enumerate(solutions2):
    display(Math(r'\text{Solution equation 2, %d: } %s' % (i, sympy.latex(sol))))
    print(sol)

equation1 = Eq((sqrt(x ** 2 + y ** 2) - sqrt((x - x1) ** 2 + (y - y1) ** 2)) - t1, 0)
equation2 = Eq((sqrt(x ** 2 + y ** 2) - sqrt((x - x2) ** 2 + (y - y2) ** 2)) - t2, 0)

solutions1 = solve(equation1, y, check=True, simplify=True)
solutions2 = solve(equation2, y, check=True, simplify=True)

for (i, e1), (j, e2) in product(enumerate(solutions1), enumerate(solutions2)):
    for s in solve(e1 - e2, x, check=False, simplify=True):
        # display(Math(r'\text{Final solutions %d %d: } %s' % (i, j, sympy.latex(s))))
        print(i, j)
        print(s)

#####
###
# # Illustration cluster

from math import floor
from sklearn.cluster import MeanShift
from itertools import product

def graph1(gc1):
    gc1 = np.array(gc1)
    print(len(gc1[:, 0][np.logical_and(np.logical_and(gc1[:, 0] < 12, gc1[:, 0] > 5), gc1[:, 1] < 12)]))
    fig = go.Figure(data=[
        go.Scatter(x=gc1[:, 0][np.logical_and(np.logical_and(gc1[:, 0] < 12, gc1[:, 0] > 5), gc1[:, 1] < 12)],
            y=gc1[:, 1][np.logical_and(np.logical_and(gc1[:, 0] < 12, gc1[:, 0] > 5), gc1[:, 1] < 12)],

```

```

        mode='markers',
        showlegend=True,
        name="$\\large{\\text{Coordonnees candidates}}$",
        marker_symbol="hexagram",
        marker=dict(color="orange",
                    size=7.5),
    ),

go.Scatter(x=mx,
           y=my,
           mode='markers+text',
           marker_line_color=["black" for _ in range(3)],
           marker=dict(color=["lime" for _ in range(3)],
                       size=[10 for _ in range(3)]),
           marker_symbol=["circle" for _ in range(3)],
           showlegend=True,
           name="$\\large{\\text{Microphones}}$",
           text=["$\\Large{M_1}$", "$\\Large{M_2}$", "$\\Large{M_3}$"],
           textposition="bottom center",
           opacity=1,
           ),
go.Scatter(x=sx,
           y=sy,
           mode='markers+text',
           marker_line_color=["black" for _ in range(3)],
           marker=dict(color=["blue" for _ in range(3)],
                       size=[10 for _ in range(3)]),
           marker_symbol=['pentagon' for _ in range(3)],
           showlegend=True,
           name="$\\large{\\text{Sources}}$",
           text=["$\\Large{S_1}$", "$\\Large{S_2}$", "$\\Large{S_3}$"],
           textposition="bottom center",
           opacity=1,
           ),
],
)

fig.update_xaxes(scaleanchor='x',
                 linewidth=1,

```

```

        linecolor='black',
        mirror=True,
        ticks='outside',
        showline=True,
        showgrid=False,
        scaleratio=1)
fig.update_yaxes(scaleanchor='y',
                 linewidth=1,
                 linecolor='black',
                 mirror=True,
                 ticks='outside',
                 showline=True,
                 showgrid=False,
                 scaleratio=1)

fig.update_layout(
    template='plotly_white',
    legend=dict(
        yanchor="top",
        y=0.99,
        xanchor="left",
        x=0.025,
        bordercolor="black",
        borderwidth=1),
    title={"text": "$\\Large{\\text{Candidats pour localiser des sources}} \\quad "
          "2000 \\text{ candidats - Spectrogramme 128 + Pics 5 (Minimisation)}$"},
    xaxis={"title": {"text": r"$\\Large{\\text{Position } x \\text{ en } m}$"}},
    yaxis={"title": {"text": r"$\\Large{\\text{Position } y \\text{ en } m}$"}}
)

fig.write_image("results/3 fig8 lr.webp", scale=1.3 * 2, height=600, width=600)
fig.write_image("results/3 fig8 hr.png", scale=13, height=600, width=600)

```

```

def graph2(gc, center, repartition, dl):
    gc = np.array(gc)

```

```

    fig = go.Figure(data=[
        go.Scatter(x=mx,

```



```

fig.add_trace(
    go.Scatter(x=sx,
               y=sy,
               mode='markers+text',
               marker_line_color=["black" for _ in range(3)],
               marker=dict(color=["blue" for _ in range(3)],
                           size=[10 for _ in range(3)]),
               marker_symbol=['pentagon' for _ in range(3)],
               showlegend=True,
               name="$\\large{\\text{Source}}$",
               text=["$\\Large{S_1}$", "$\\Large{S_2}$", "$\\Large{S_3}$"],
               textposition="bottom center",
               opacity=1,
            ),
)

fig.update_xaxes(scaleanchor='x',
                 linewidth=1,
                 linecolor='black',
                 mirror=True,
                 ticks='outside',
                 showline=True,
                 showgrid=False,
                 scaleratio=1)
fig.update_yaxes(scaleanchor='y',
                 linewidth=1,
                 linecolor='black',
                 mirror=True,
                 ticks='outside',
                 showline=True,
                 showgrid=False,
                 scaleratio=1)

fig.update_layout(
    template='plotly_white',
    legend=dict(
        yanchor="top",
        y=0.99,

```

```

        xanchor="left",
        x=0.025,
        bordercolor="black",
        borderwidth=1),
    title={"text": "$\\Large{\\text{Regroupements candidats pour localiser des sources}} \\ \\ \\ "
          "2000 \\text{ candidats - Spectrogramme 128 + Pics 5 (Minimisation)}$"},
    xaxis={"title": {"text": r"$\\Large{\\text{Position } x \\text{ en } m}$"}},
    yaxis={"title": {"text": r"$\\Large{\\text{Position } y \\text{ en } m}$"}}
)

```

```

fig.write_image("results/3 fig9 lr.webp", scale=1.3 * 2, height=600, width=600)
fig.write_image("results/3 fig9 hr.png", scale=13, height=600, width=600)

```

C'est le même code que dans isolate

```

def signal_time_shift(signal1: np.ndarray, signal2: np.ndarray, unit: str = "s", rate: int = None) -> int | float:
    corr: np.ndarray = signal.correlate(signal1, signal2, method="fft", mode="full")
    n = len(corr)
    if unit == "index":
        return floor(np.argmax(corr) - n / 2)

def euclidian_distance(x0, y0, z0, x1, y1, z1) -> float | np.ndarray:
    return ((x0 - x1) ** 2 + (y0 - y1) ** 2 + (z0 - z1) ** 2) ** 0.5

def labels_barycenter(coordinates: np.ndarray, labels: np.ndarray) -> dict:
    label_groups = {}

    for coord, label in zip(coordinates, labels):
        if label not in label_groups:
            label_groups[label] = []
        label_groups[label].append(coord)

    bc = {label: np.mean(coords, axis=0) for label, coords in label_groups.items()}

    return bc

```

```
def possible_coordinate(microphones: list[Microphone],
                        distance_shift: list[tuple | list],
                        start_point: np.ndarray,
                        max_error: float,
                        clustering_algo,
                        ) -> list[tuple[float, float, float] | tuple[float, float]]:
```

```
# Create the error function
```

```
n = len(distance_shift[0])
```

```
dim = len(start_point)
```

```
t_range = [(i, j) for j in range(n) for i in range(n) if j < i]
```

```
def error_function(m: np.array, selected_ts: tuple) -> float:
```

```
    if dim == 3:
```

```
        x, y, z = m
```

```
    elif dim == 2:
```

```
        x, y = m
```

```
        z = 0
```

```
    else:
```

```
        raise Exception("Dimension unsupported.")
```

```
    error = 0
```

```
    k = 0
```

```
    for i, j in t_range:
```

```
        di = euclidian_distance(x, y, z, *microphones[i].get_coord)
```

```
        dj = euclidian_distance(x, y, z, *microphones[j].get_coord)
```

```
        delta = (di - dj)
```

```
        error += abs(delta - selected_ts[k])
```

```
        k += 1
```

```
    return error / k
```

```
# For each timeshift minimize the error for x, y, z coordinates
```

```
G1_C = []
```

```
G2_C = []
```

```

coords = []

min_ts = {(i, j): euclidian_distance(*microphones[i].get_coord, *microphones[j].get_coord)
          for i in range(n) for j
          in range(n) if j < i
          }

for elt in distance_shift:
    # remove impossible distance shift
    if any(map(lambda c: abs(c[0]) > min_ts[c[1]], zip(elt, t_range))):
        continue

    res = minimize(error_function, start_point, args=(elt,), method="Nelder-Mead")

    if error_function(res.x, elt) < max_error:
        if dim == 3:
            x1, y1, z1 = res.x
        elif dim == 2:
            x1, y1 = res.x
            z1 = 0.0
        else:
            raise Exception("Dimension unsupported.")

        coords.append([x1, y1, z1])
        G2_C.append([x1, y1])

    x1, y1 = res.x
    G1_C.append([x1, y1])

graph1(G2_C)

coords = np.array(coords)

# Clustering to find barycentre of each cluster corresponding to potential source coordinate
result: list[tuple[float, float, float] | tuple[float, float]] = []

clustering = clustering_algo()
clustering.fit(coords)

```

```

labels = clustering.labels_
cluster_centers = labels_barycenter(coords, labels)

unique, counts = np.unique(labels, return_counts=True)
dict_labels = dict(zip(unique, counts))

for i, center in sorted([*cluster_centers.items()], key=lambda c: dict_labels[c[0]], reverse=True):
    if dict_labels[i] > 1:
        if dim == 3:
            xc, yc, zc = center
            result.append((xc, yc, zc))
        elif dim == 2:
            xc, yc, _ = center
            result.append((xc, yc))

graph2(coords, result, labels, dict_labels)

return result

```

```

def locate_many_sources_spectrogram(microphones: list[Microphone],
                                     records: list[np.ndarray],
                                     sample_rate: int,
                                     start_point: np.ndarray,
                                     length: int = 16,
                                     max_error: float = 1.0) -> list[tuple[float, float, float]]:
    n_tries = 5
    # Prepare variable
    t, freq, _ = spectrogram(records[0], sample_rate, length)
    n = len(records)
    mask = np.array([freq for _ in range(len(t))]).T

    spectrogram_list = [spectrogram(r, sample_rate, length, workers=12)[2] for r in records]

    signal_list = [
        [
            inverse_spectrogram(np.where(mask != f, 0, spectrogram_list[i]), length, workers=12) for f in freq
        ] for i in range(n)
    ]

```

```

t_range = [(i, j) for j in range(n) for i in range(n) if j < i]

list_distance_shift = []

for sig1, sig2, sig3 in zip(*signal_list):

    k = 0
    sig = [sig1, sig2, sig3]

    init_ds = []

    for (i, j) in t_range:
        corr: np.ndarray = signal.correlate(sig[i], sig[j], method="fft", mode="full")
        peaks, _ = signal.find_peaks(corr)
        peaks: np.ndarray = peaks
        c: np.ndarray = np.argpartition(corr[peaks], -n_tries)[-n_tries:]

        ds = np.round(peaks[c] - len(corr) / 2) * 1 / sample_rate * define_sound_speed()

        init_ds.append(ds)

    k += 1

    list_distance_shift += [*product(*init_ds)]

return possible_coordinate(microphones, list_distance_shift, start_point, max_error, MeanShift)

locate_many_sources_spectrogram([m1, m2, m3], lr, RATE, np.array([0[0], 0[1]]), max_error=0.34, length=128, )

```

chart/4_test_locate_stats.py

Génération de statistique pour la localisation

```

from copy import deepcopy
from itertools import product
from multiprocessing import Pool
import os
import pickle
import random
from random import uniform
import sqlite3

import numpy as np

from audio_locate_isolate.locate import locate_source
from audio_locate_isolate.locate import locate_many_sources_spectrogram
from audio_locate_isolate.locate import locate_many_sources_spikes
from audio_locate_isolate.locate import locate_many_sources_both
from environnement import DEFAULT_TEMPERATURE
from environnement import Environment
from environnement import euclidian_distance
from environnement import Microphone
from environnement import Sound
from environnement import Source

from chart_utils import cercle_circonscrit

RATE = 44100

PATH_1 = "C:/Users/Utilisateur/Documents/Python/ADVANCED/GeoSoundTIPE/sound_data/"
PATH_2 = "C:/Users/Utilisateur/Documents/Python/ADVANCED/GeoSoundTIPE/sound_data/test/"

PRECISE_RES = True

sounds_list_wav = ([PATH_1 + f for f in os.listdir(PATH_1) if "test" not in f] +
                   [PATH_2 + f for f in os.listdir(PATH_2) if "test" not in f])

if not PRECISE_RES:
    # Resultat database
    NAME: str = "results_stats_OPTIMIZE_ALL"
    conn = sqlite3.connect(f"F:/DATASET/ALI TIPE/3 result/{NAME}.db", check_same_thread=False)
    cursor = conn.cursor()

```



```

cursor.execute("""
    SELECT name
    FROM sqlite_master
    WHERE type = 'table'
    AND name = 'traite'
    """)
if cursor.fetchone() is None:
    cursor.execute("""
        CREATE TABLE "simple"
        (
            "file_num"    INTEGER NOT NULL,
            "k_sources"   INTEGER NOT NULL,
            "record_num"  INTEGER NOT NULL,
            "conf"        INTEGER NOT NULL,
            "res_x"       REAL    NOT NULL,
            "res_y"       REAL    NOT NULL
        );
        """)
    conn.commit()
    cursor.execute("""
        CREATE TABLE "spikes"
        (
            "file_num"    INTEGER NOT NULL,
            "k_sources"   INTEGER NOT NULL,
            "record_num"  INTEGER NOT NULL,
            "conf"        INTEGER NOT NULL,
            "res_x"       REAL    NOT NULL,
            "res_y"       REAL    NOT NULL
        );
        """)
    conn.commit()
    cursor.execute("""
        CREATE TABLE "spectro"
        (
            "file_num"    INTEGER NOT NULL,
            "k_sources"   INTEGER NOT NULL,
            "record_num"  INTEGER NOT NULL,
            "conf"        INTEGER NOT NULL,
            "res_x"       REAL    NOT NULL,

```

```

        "res_y"      REAL    NOT NULL
    );
    """
cursor.execute("""
    CREATE TABLE "ss"
    (
        "file_num"    INTEGER NOT NULL,
        "k_sources"   INTEGER NOT NULL,
        "record num"  INTEGER NOT NULL,
        "conf"        INTEGER NOT NULL,
        "res_x"       REAL    NOT NULL,
        "res_y"       REAL    NOT NULL
    );
    """)
conn.commit()
cursor.execute("""
    CREATE TABLE "traite"
    (
        "model"      TEXT     NOT NULL,
        "k_sources"   INTEGER NOT NULL,
        "file_num"    INTEGER NOT NULL
    );
    """)
conn.commit()

```

else:

```

#
NAME: str = "result_prceise"
conn = sqlite3.connect(f"F:/DATASET/ALI TIPE/3 result/{NAME}.db", check_same_thread=False)
cursor = conn.cursor()

```

try:

```

cursor.execute("""
    CREATE TABLE "traite"
    (
        "k_sources"   INTEGER NOT NULL,
        "file_num"    INTEGER NOT NULL
    );
    """)

```

```
conn.commit()
```

```
cursor.execute("""
```

```
    CREATE TABLE "results"
```

```
    (
```

```
        "file_num"    INTEGER NOT NULL,
```

```
        "k_sources"   INTEGER NOT NULL,
```

```
        "record_num"  INTEGER NOT NULL,
```

```
        "s1"          REAL,
```

```
        "s2"          REAL    NOT NULL,
```

```
        "s3"          REAL    NOT NULL,
```

```
        "s4"          REAL    NOT NULL,
```

```
        "s5"          REAL    NOT NULL,
```

```
        "s6"          REAL    NOT NULL,
```

```
        "s7"          REAL    NOT NULL,
```

```
        "s8"          REAL    NOT NULL,
```

```
        "s9"          REAL    NOT NULL,
```

```
        "s10"         REAL    NOT NULL
```

```
    );
```

```
""")
```

```
conn.commit()
```

```
except Exception as e:
```

```
    print(e)
```

```
def random_records_3(sounds_list: list[str],
```

```
    mic_coordinate_interval: tuple[float, float] = (-20, 20),
```

```
    min_mics_distance: float = 2.0,
```

```
    source_amount: int = 3,
```

```
    source_coordinate_interval: tuple[float, float] = (-20, 20),
```

```
    min_sources_distance: float = 0.1,
```

```
    record_duration: float = 5.0,
```

```
    record_start: tuple[int, int] = (5, 85),
```

```
    temperature: float = DEFAULT_TEMPERATURE
```

```
):
```

```

mics_amount = 3

min_mic_coordinate, max_mic_coordinate = mic_coordinate_interval
min_source_coordinate, max_source_coordinate = source_coordinate_interval

# Create microphones
rayon = max_mic_coordinate + 1
mics = []

while rayon > max_mic_coordinate:
    mics = []

    for k in range(mics_amount):
        x1, y1 = uniform(min_mic_coordinate, max_mic_coordinate), uniform(min_mic_coordinate, max_mic_coordinate)

        # Check the minimum distance between microphones
        while any(map(lambda m: euclidian_distance(x1, y1, 0, *m.get_coord) < min_mics_distance, mics)):
            x1, y1 = uniform(min_mic_coordinate, max_mic_coordinate), uniform(min_mic_coordinate,
                                                                                max_mic_coordinate)

        mics.append(Microphone(f"m{k}", x1, y1, 0))

    barycentre = cercle_circonscrit(*[*map(lambda m: m.get_coord[0:2], mics)])
    rayon = euclidian_distance(*barycentre, 0, *mics[0].get_coord)

# Create sources
sources = []
coordinates = []

for k in range(source_amount):
    x, y = uniform(min_source_coordinate, max_source_coordinate), uniform(min_source_coordinate,
                                                                              max_source_coordinate)

    # Check the minimum distance between sources and microphones

    """condition1 = any(map(lambda c: euclidian_distance(x, y, 0, *c) < min_sources_distance, coordinates))
    condition2 = any(map(lambda m: euclidian_distance(x, y, 0, *m.get_coord) < min_sources_distance, mics))
    condition3 = (x - barycentre[0]) ** 2 + (y - barycentre[1]) ** 2 > rayon ** 2"""

```

```

while (x - barycentre[0]) ** 2 + (y - barycentre[1]) ** 2 > rayon ** 2:
    x, y = uniform(min_mic_coordinate, max_mic_coordinate), uniform(min_mic_coordinate, max_mic_coordinate)

    """condition1 = any(map(lambda c: euclidian_distance(x, y, 0, *c) < min_sources_distance, coordinates))
    condition2 = any(map(lambda m: euclidian_distance(x, y, 0, *m.get_coord) < min_mics_distance, mics))
    condition3 = (x - barycentre[0]) ** 2 + (y - barycentre[1]) ** 2 > rayon ** 2"""

sources.append(Source(
    sound=Sound(f"{random.choice(sounds_list)}", rate=RATE),
    name=f"s{k}",
    x=x, y=y, z=0
))
coordinates.append((x, y, 0))

# Create the environment
env = Environment(
    sources=sources,
    microphones=mics,
    rate=None,
    microphone_tensors=[],
    temperature=temperature
)

start_time = random.randint(record_start[0], record_start[1])
_, list_records = env.listen_from_microphone([f"m{k}" for k in range(mics_amount)],
                                              start_time,
                                              start_time + record_duration,
                                              start_time - 1)

return env, list_records

def create_dataset(args):
    k_source, num = args

    print("File : ", k_source, num)
    res = []

    for i in range(100):

```

```

print(f"\r{k_source} - {i}", end="")

env, lr = random_records_3(sounds_list_wav, source_amount=k_source)

res.append([(env.microphones, [s.get_coord for s in env.sources.values()]), lr])

with open(f'F:/DATASET/ALI TIPE/3 result/test_data/data_{k_source}_{num}.pickle', 'wb') as handle:
    pickle.dump(res, handle)

function_to_test = [
    (0, "simple", lambda mics, recs, center, conf: locate_source(mics, recs, 44100, center, method="optimize"))
]

function_to_test += [
    (k, "spectro",
     lambda mics, recs, center, conf: locate_many_sources_spectrogram(mics, recs, 44100, center, length=conf,
                                                                         method="optimize"))
    for k in [8, 16, 32, 64, 128, 256, 512]
]

function_to_test += [
    (k, "spikes",
     lambda mics, recs, center, conf: locate_many_sources_spikes(mics, recs, 44100, n_peaks=conf, start_point=center,
                                                                    method="optimize")) for k in range(2, 11, 2)
]

function_to_test += [
    (c, "ss",
     lambda mics, recs, center, conf: locate_many_sources_both(mics, recs, 44100, n_tries=conf[0], length=conf[1],
                                                                 start_point=center,
                                                                 method="optimize")) for c in product([2, 4], [64,
128])
]

def iter_dataset(k_source, num, path: str = "F:/DATASET/ALI TIPE/3 result/test_data/"):
    list_files = [f for f in os.listdir(path) if
                   f.split("_")[1] == f"{k_source}" and f.split("_")[2].split(".")[0] == f"{num}"]

```

```

for f in list_files:
    data = pickle.load(open(f"{path}/{f}", "rb"))

    for coords, lr in data:
        yield coords, lr

return StopIteration

def test_method(args):
    fi, k_sources, file_num = args
    conf, table, function = function_to_test[fi]

    if table == "ss":
        conf_PARAM = deepcopy(conf)
        conf = 10000 * conf[0] + conf[1]
    else:
        conf_PARAM = conf

    # Verifier que la table n'existe pas
    print(f"\n{conf} - {table} - {k_sources} - {file_num} Started")

    cursor.execute("SELECT * FROM traite WHERE model = ? AND k_sources = ? AND file_num = ?;",
                    (table + f"{conf}", k_sources, file_num)
                    )
    already = cursor.fetchall()

    if len(already) == 0:
        cursor.execute(
            f"DELETE FROM {table} WHERE conf = ? AND k_sources = ? AND file_num = ?",
            (conf, k_sources, file_num)
        )
        conn.commit()

    for i, ((microphones, sources_coordinates), list_records) in enumerate(iter_dataset(k_sources, file_num)):
        print(f"\r{i} {k_sources} {file_num}", end="")

        list_records = [r[:len(r) // 5] for r in list_records]

```

```

try:
    barycentre = cercle_circonscriit(*[*map(lambda m: m.get_coord[0:2], microphones.values())])

    res = function([*microphones.values()], list_records, np.array(barycentre), conf)

    if isinstance(res, tuple):
        res = res
    else:
        if len(res) == 0:
            continue
        res = min(res, key=lambda coord: min(
            [euclidian_distance(*coord[:2], 0.0, *sc) for sc in sources_coordinates], default=[]
        ), default=None)

    if res is None:
        continue

    if euclidian_distance(*barycentre, 0.0, *res[:2], 0.0) < euclidian_distance(*barycentre, 0.0,
                                                                                   *microphones[
                                                                                       "m1"].get_coord):

        cursor.execute(
            f"INSERT INTO {table} (conf, file_num, k_sources, record_num, res_x, res_y) VALUES "
            "(?, ?, ?, ?, ?, ?)",
            (conf, file_num, k_sources, i, res[0], res[1])
        )
        conn.commit()

except:
    pass

cursor.execute(
    f"INSERT INTO traite (model, k_sources, file_num) VALUES "
    "(?, ?, ?)",
    (table + f"{conf}", k_sources, file_num)
)
conn.commit()
print(f"\n{conf} - {table} - {k_sources} - {file_num} Done")
else:

```



```

    print(f"\n{conf} - {table} - {k_sources} - {file_num} Already done")

def test_precise(args):
    k_sources, file_num = args

    # Verifier que la table n'existe pas
    print(f"\n{k_sources} - {file_num} Started")

    cursor.execute("SELECT * FROM traite WHERE k_sources = ? AND file_num = ?;",
                    (k_sources, file_num)
                    )
    already = cursor.fetchall()

    if len(already) == 0:
        cursor.execute(
            f"DELETE FROM results WHERE k_sources = ? AND file_num = ?",
            (k_sources, file_num)
        )
        conn.commit()

    for i, ((microphones, sources_coordinates), list_records) in enumerate(iter_dataset(k_sources, file_num)):
        try:
            print(f"\r{i} {k_sources} {file_num}", end="")

            list_records = [r[:len(r) // 5] for r in list_records]

            barycentre = cercle_circonscriit(*[*map(lambda m: m.get_coord[0:2], microphones.values())])

            res = locate_many_sources_spectrogram([*microphones.values()],
                                                    list_records,
                                                    44100,
                                                    barycentre,
                                                    length=128,
                                                    method="optimize"
                                                    )

            dict_res = {f"s{k}": -1 for k in range(1, 11)}

```

```

    for k, rk in enumerate(res):
        err = min(
            [euclidian_distance(*rk[:2], 0.0, *sc) for sc in sources_coordinates], default=-1
        )

        d_max = euclidian_distance(*barycentre, 0.0, *microphones["m1"].get_coord)
        if euclidian_distance(*barycentre, 0.0, *rk[:2], 0.0) < d_max:
            dict_res[f"s{k}"] = err

    cursor.execute(
        f"INSERT INTO results (file_num, k_sources, record_num, s1, s2, s3, s4, s5, s6, s7, s8, s9, s10)
        VALUES "
        "(?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)",
        (file_num, k_sources, i) + tuple([dict_res[f"s{k}"] for k in range(1, 11)])
    )
    conn.commit()
except Exception as e:
    print(e)

cursor.execute(f"INSERT INTO traite (k_sources, file_num) VALUES (?, ?)", (k_sources, file_num))
conn.commit()
print(f"\n{k_sources} - {file_num} Done")
else:
    print(f"\n{k_sources} - {file_num} Already done")

"""if __name__ == '__main__':
    # Create the dataset
    quit()
    # Save data
    r = set([*product(range(1, 11), range(10))])

    for f in os.listdir("F:/DATASET/ALI TIPE/3 result/test_data/"):
        r.remove((int(f.split("_")[1]), int(f.split("_")[2].split(".")[0])))

    with Pool(processes=10) as pool:
        map_result = pool.map(create_dataset, [*r])
    print("end")"""

```

```

"""if __name__ == '__main__':
    # Create stats
    K_SOURCES = range(1, 11)
    FILE_NUM = range(0, 10)
    FI_RANGE = range(0, len(function_to_test))

    ALL_COUPLE = product(FI_RANGE, K_SOURCES, FILE_NUM)

    with Pool(processes=10) as pool:
        map_result = pool.imap_unordered(test_method, [*ALL_COUPLE])
        for r in map_result:
            continue"""

# Spectrogram 128 details
if __name__ == '__main__':

    K_SOURCES = range(1, 11)
    FILE_NUM = range(0, 10)

    ALL_COUPLE = product(K_SOURCES, FILE_NUM)

    with Pool(processes=10) as pool:
        map_result = pool.imap_unordered(test_precise, [*ALL_COUPLE])
        for r in map_result:
            continue

```

chart/4_test_locate.ipynb

Etude statistique de la localisation

```

import os
import pickle
import sqlite3

import numpy as np
import plotly.graph_objects as go
from plotly.subplots import make_subplots
import plotly.io as pio
from scipy.stats import t as student

pio.renderers.default = "browser"

sound_speed = 331.5 + 0.607 * 20

#####
###
# # Illustrate compare methode

LIST_SPIKES = [*range(2, 11, 2)]
LIST_SPECTRO = [8, 16, 32, 64, 128, 256]
# F:/DATASET/ALI TIPE/3 result/results_stats_TRILATERATION_ALL.db
# F:/DATASET/ALI TIPE/3 result/results_stats_OPTIMIZE_ALL.db
FILE = "F:/DATASET/ALI TIPE/3 result/results_stats_OPTIMIZE_ALL.db"

conn = sqlite3.connect(FILE, check_same_thread=False)
cursor = conn.cursor()

#####
###
# ## Load records data

# open data
from chart_utils import cercle_circonscrit
from environnement import Microphone, euclidian_distance

path: str = "F:/DATASET/ALI TIPE/3 result/test_data/"

```

```

ALL_COORDS = {}
NORM = {}

for k_source in range(1, 11):
    for num in range(10):
        list_files = [f for f in os.listdir(path) if
                        f.split("_")[1] == f"{k_source}" and f.split("_")[2].split(".")[0] == f"{num}"]

        ALL_COORDS[(k_source, num)] = []
        NORM[(k_source, num)] = []

        for f in list_files:
            data = pickle.load(open(f"{path}/{f}", "rb"))
            a = 0
            for coords, lr in data:
                o = cercle_circonscri(*[c.get_coord[:2] for c in coords[0].values()])
                norm = euclidian_distance(*o, 0, *coords[0]["m1"].get_coord)
                ALL_COORDS[(k_source, num)].append(coords[1])
                NORM[(k_source, num)].append(norm)
                """# 6 1 5
                if k_source == 1 and a == 2 and num == 3:
                    print(coords[0]["m1"].get_coord, coords[0]["m2"].get_coord, coords[0]["m0"].get_coord, coords[1])
                    a += 1"""

#####
###
# ## Simple results

# Charts Simple
from plotly.subplots import make_subplots

cursor.execute("SELECT * FROM simple;")
data = cursor.fetchall()

simple_result = [[] for _ in range(10)]

error_count = {i: 0 for i in range(1, 11)}

for fn, k, rn, _, x, y in data:

```

```

C = ALL_COORDS[(k, fn)][rn]

index = min(range(len(C)), key=lambda i: ((C[i][0] - x) ** 2 + (C[i][1] - y) ** 2) ** 0.5)
rx, ry, _ = ALL_COORDS[(k, fn)][rn][index]
norm = NORM[(k, fn)][index]
err = ((rx - x) ** 2 + (ry - y) ** 2) ** 0.5

# if euclidian_distance(x, y, 0, rx, ry, 0) < norm:
simple_result[k - 1].append(err)

error_count[k] += 1

error_count = {k: 100 - v / 1000 * 100 for k, v in error_count.items()}

simple_average = np.array([np.average(d) for d in simple_result])
simple_incertitude = np.array([np.std(d) for d in simple_result])
simple_len = np.array([len(d) for d in simple_result])

fig = make_subplots(specs=[[{"secondary_y": True}]])

fig.add_trace(go.Bar(
    x=[i for i in range(1, 11)], y=simple_average,
    error_y=dict(type='data', array=simple_incertitude / simple_len ** 0.5 * student.ppf(0.95, simple_len)),
    marker_color="steelblue",
    showlegend=False,
),
    secondary_y=False
)

fig.add_trace(go.Scatter(
    name="Taux d'échec",
    x=[i for i in range(1, 11)], y=[error_count[i] for i in range(1, 11)],
    marker_line_color="red",
    marker_symbol="line-ew",
    marker_line_width=4.75,
    marker_size=17.5,
    mode='markers',
),
    secondary_y=True
)

```

```

)

# fig.update_layout(barmode='group', secondary_y=False)

fig.update_xaxes(
    zeroline=False,
    linewidth=1,
    linecolor='black',
    mirror=True,
    ticks='outside',
    showline=True,
    showgrid=False,
    scaleratio=1,
    type="category"
)
fig.update_yaxes(
    zeroline=False,
    linewidth=1,
    linecolor='black',
    mirror=True,
    ticks='outside',
    showline=True,
    showgrid=False,
    scaleratio=1,
    range=(0, 3.5),
    secondary_y=False
)

fig.update_layout(
    template='plotly_white',
    legend=dict(
        yanchor="top",
        y=0.99,
        xanchor="left",
        x=0.025,
        bordercolor="white",
        borderwidth=1),
    xaxis={"title": {"text": r"$\Large{\text{Nombres de sources}}$"}},

```



```

)

fig.update_yaxes(
    secondary_y=True,
    zeroline=False,
    linewidth=1,
    linecolor='black',
    mirror=True,
    ticks='outside',
    showline=True,
    showgrid=False,
    scaleratio=1,
    range=(0, 100),
)

if FILE == "F:/DATASET/ALI TIPE/3 result/results_stats_TRILATERATION_ALL.db":
    fig.update_layout(title={
        "text": r"$\Large{\text{Erreur moyenne de la methode simple en } \\\ "
                r"\text{fonction du nombre de sources (trilatération)}} \\\ "
                r"1000 \text{ mesures aléatoires par barre }$",
        yaxis={"title": {"text": r"$\Large{\text{Erreur moyenne en } m}$"}}
    })

    fig.write_image("results/4 fig3 tri lr.webp", scale=1.3, height=600, width=1037 // 2)
    fig.write_image("results/4 fig3 tri hr.png", scale=13, height=600, width=1037 // 2)
else:
    fig.update_layout(title={
        "text": r"$\Large{\text{Erreur moyenne de la methode simple en } \\\ "
                r"\text{fonction du nombre de sources (minimisation)}} \\\ "
                r"1000 \text{ mesures aléatoires par barre }$"
    })

    fig.update_yaxes(
        secondary_y=True,
        title_text=r"$\Large{\text{Taux d'échec en } \%}$",
    )

    fig.write_image("results/4 fig4 opt lr.webp", scale=1.3, height=600, width=1037 // 2)
    fig.write_image("results/4 fig4 opt hr.png", scale=13, height=600, width=1037 // 2)

```

```
#####
###
# ## Spectro results

# Chart spikes
fig = go.Figure()

fig.add_trace(go.Bar(
    name='Simple',
    x=[i for i in range(1, 11)], y=simple_average,
    error_y=dict(type='data', array=simple_incertitude / simple_len ** 0.5 * student.ppf(0.95, simple_len)),
    marker_color="steelblue",
))

for length in LIST_SPECTRO: # 512
    cursor.execute(f"SELECT * FROM spectro WHERE conf = {length};")
    data = cursor.fetchall()

    res = [[] for _ in range(10)]

    for fn, k, rn, _, x, y in data:
        C = ALL_COORDS[(k, fn)][rn]
        index = min(range(len(C)), key=lambda i: ((C[i][0] - x) ** 2 + (C[i][1] - y) ** 2) ** 0.5)
        rx, ry, _ = ALL_COORDS[(k, fn)][rn][index]

        # norm = NORM[(k, fn)][index]
        res[k - 1].append(((rx - x) ** 2 + (ry - y) ** 2) ** 0.5)

    res_average = np.array([np.average(d) for d in res])
    res_incertitude = np.array([np.std(d) for d in res])
    res_len = np.array([len(d) for d in res])

    fig.add_trace(go.Bar(
        name=str(length),
        x=[i for i in range(1, 11)], y=res_average,
        error_y=dict(type='data', array=res_incertitude / res_len ** 0.5 * student.ppf(0.95, res_len)),
    ))

fig.update_layout(barmode='group')
```

```

fig.update_xaxes(
    zeroline=False,
    linewidth=1,
    linecolor='black',
    mirror=True,
    ticks='outside',
    showline=True,
    showgrid=False,
    scaleratio=1,
    type="category"
)
fig.update_yaxes(
    zeroline=False,
    linewidth=1,
    linecolor='black',
    mirror=True,
    ticks='inside',
    showline=True,
    showgrid=False,
    scaleratio=1,
    range=(0, 3)
)

fig.update_layout(
    template='plotly_white',
    legend=dict(
        orientation="h",
        yanchor="top",
        y=0.99,
        xanchor="left",
        x=0.025,
        bordercolor="black",
        borderwidth=1),
    xaxis={"title": {"text": r"$\Large{\text{Nombres de sources}}\ $" }},
    yaxis={"title": {"text": r"$\Large{\text{Erreur en } m}\ $" } }
)

if FILE == "F:/DATASET/ALI TIPE/3 result/results_stats_TRILATERATION_ALL.db":

```

```

fig.update_layout(title={
    "text": r"$\Large{\text{Erreur moyenne des méthodes spectrogramme en } } "
            r"\text{fonction du nombre de sources}} \small{\text{ (trilateration)}} \\"
            r"1000 \text{ mesures aléatoires par barre }$")

fig.write_image("results/4 fig5 tri lr.webp", scale=1.3, height=600, width=1038)
fig.write_image("results/4 fig5 tri hr.png", scale=13, height=600, width=1038)

else:
    fig.update_layout(title={
        "text": r"$\Large{\text{Erreur moyenne des méthodes spectrogramme en } } "
                r"\text{fonction du nombre de sources}} \small{\text{ (optimisation)}} \\"
                r"1000 \text{ mesures aléatoires par barre }$")

    fig.write_image("results/4 fig5 opt lr.webp", scale=1.3, height=600, width=1038)
    fig.write_image("results/4 fig5 opt hr.png", scale=13, height=600, width=1038)

#####
###
# ## Spikes results

# Chart spikes
fig = go.Figure()

fig.add_trace(go.Bar(
    name='Simple',
    x=[i for i in range(1, 11)], y=simple_average,
    error_y=dict(type='data', array=simple_incertitude / simple_len ** 0.5 * student.ppf(0.95, simple_len)),
    marker_color="steelblue",
))

for spikes in LIST_SPIKES:
    cursor.execute(f"SELECT * FROM spikes WHERE conf = {spikes};")
    data = cursor.fetchall()

    res = [[] for _ in range(10)]

    for fn, k, rn, _, x, y in data:
        C = ALL_COORDS[(k, fn)][rn]

```

```

index = min(range(len(C)), key=lambda i: ((C[i][0] - x) ** 2 + (C[i][1] - y) ** 2) ** 0.5)
rx, ry, _ = ALL_COORDS[(k, fn)][rn][index]

# norm = NORM[(k, fn)][index]
res[k - 1].append(((rx - x) ** 2 + (ry - y) ** 2) ** 0.5)

res_average = np.array([np.average(d) for d in res])
res_incertitude = np.array([np.std(d) for d in res])
res_len = np.array([len(d) for d in res])

fig.add_trace(go.Bar(
    name=str(spikes),
    x=[i for i in range(1, 11)], y=res_average,
    error_y=dict(type='data', array=res_incertitude / res_len ** 0.5 * student.ppf(0.95, res_len)),
))

fig.update_layout(barmode='group')

fig.update_xaxes(
    zeroline=False,
    linewidth=1,
    linecolor='black',
    mirror=True,
    ticks='outside',
    showline=True,
    showgrid=False,
    scaleratio=1,
    type="category"
)
fig.update_yaxes(
    zeroline=False,
    linewidth=1,
    linecolor='black',
    mirror=True,
    ticks='inside',
    showline=True,
    showgrid=False,
    scaleratio=1,
    range=(0, 5)
)

```

```

)

fig.update_layout(
    template='plotly_white',
    legend=dict(
        orientation="h",
        yanchor="top",
        y=0.99,
        xanchor="left",
        x=0.025,
        bordercolor="black",
        borderwidth=1),
    xaxis={"title": {"text": r"$\Large{\text{Nombres de sources}}$"}},
    yaxis={"title": {"text": r"$\Large{\text{Erreur en } m}$"}},
)

if FILE == "F:/DATASET/ALI TIPE/3 result/results_stats_TRILATERATION_ALL.db":
    fig.update_layout(title={
        "text": r"$\Large{\text{Erreur moyenne des méthodes \textit{pics} en } "
            r"\text{fonction du nombre de sources (trilateration)}} \\"
            r"1000 \text{ mesures aléatoires par barre }$"}

    fig.write_image("results/4 fig6 tri lr.webp", scale=1.3, height=600, width=1038)
    fig.write_image("results/4 fig6 tri hr.png", scale=13, height=600, width=1038)

else:
    fig.update_layout(title={
        "text": r"$\Large{\text{Erreur moyenne des méthodes pics en } "
            r"\text{fonction du nombre de sources (optimisation)}} \\"
            r"1000 \text{ mesures aléatoires par barre }$"}

    fig.write_image("results/4 fig6 opt lr.webp", scale=1.3, height=600, width=1038)
    fig.write_image("results/4 fig6 opt hr.png", scale=13, height=600, width=1038)

```

```

#####
###
# ## Final

```

```

cursor.execute(f"SELECT * FROM spectro WHERE conf = {128};")
data = cursor.fetchall()

X = []
Y = []

R = [[] for _ in range(10)]

for fn, k, rn, _, x, y in data:

    C = ALL_COORDS[(k, fn)][rn]
    index = min(range(len(C)), key=lambda i: ((C[i][0] - x) ** 2 + (C[i][1] - y) ** 2) ** 0.5)
    rx, ry, _ = ALL_COORDS[(k, fn)][rn][index]

    norm = NORM[(k, fn)][index]
    if norm > 0.01:
        R[k - 1].append(((rx - x) ** 2 + (ry - y) ** 2) ** 0.5)

for k, r in enumerate(R):
    print(k + 1, np.average(r), np.std(r) / np.sqrt(len(r)))

#####
###
# # Illustrate correlation matrix

N = len(LIST_SPIKES) + len(LIST_SPECTRO) + 1
NS = len(LIST_SPECTRO)
ALL_SERIES = [[np.nan for _ in range(1000 * 10)] for _ in range(N)]

#
# Charts Simple

cursor.execute("SELECT * FROM simple;")
data = cursor.fetchall()

for fn, k, rn, _, x, y in data:
    C = ALL_COORDS[(k, fn)][rn]

    index = min(range(len(C)), key=lambda i: ((C[i][0] - x) ** 2 + (C[i][1] - y) ** 2) ** 0.5)

```

```

rx, ry, _ = ALL_COORDS[(k, fn)][rn][index]
norm = NORM[(k, fn)][index]
ALL_SERIES[0][(k - 1) * 1000 + rn + 100 * fn] = ((rx - x) ** 2 + (ry - y) ** 2) ** 0.5

```

```

#
# Chart spectro

```

```

for K, length in enumerate(LIST_SPECTRO):
    cursor.execute(f"SELECT * FROM spectro WHERE conf = {length};")
    data = cursor.fetchall()

    res = [[] for _ in range(10)]

    for fn, k, rn, _, x, y in data:
        C = ALL_COORDS[(k, fn)][rn]
        index = min(range(len(C)), key=lambda i: ((C[i][0] - x) ** 2 + (C[i][1] - y) ** 2) ** 0.5)
        rx, ry, _ = ALL_COORDS[(k, fn)][rn][index]
        ALL_SERIES[K + 1][(k - 1) * 1000 + rn + 100 * fn] = ((rx - x) ** 2 + (ry - y) ** 2) ** 0.5

```

```

#
# Chart spikes

```

```

for K, spikes in enumerate(LIST_SPIKES):
    cursor.execute(f"SELECT * FROM spikes WHERE conf = {spikes};")
    data = cursor.fetchall()

    res = [[] for _ in range(10)]

    for fn, k, rn, _, x, y in data:
        C = ALL_COORDS[(k, fn)][rn]
        index = min(range(len(C)), key=lambda i: ((C[i][0] - x) ** 2 + (C[i][1] - y) ** 2) ** 0.5)
        rx, ry, _ = ALL_COORDS[(k, fn)][rn][index]
        ALL_SERIES[K + NS + 1][(k - 1) * 1000 + rn + 100 * fn] = ((rx - x) ** 2 + (ry - y) ** 2) ** 0.5

```

```

#####
###
# Correlation matrix

```

```

correlation_matrix = np.ones((N, N))

```



```

for i in range(N):
    for j in range(N):
        if i < j:
            mask = np.logical_and(np.logical_not(np.isnan(ALL_SERIES[i])), np.logical_not(np.isnan(ALL_SERIES[j])))

            si = np.array(ALL_SERIES[i])[mask]
            sj = np.array(ALL_SERIES[j])[mask]

            corr = ((np.average((si - np.average(si)) * (sj - np.average(sj)))) / (np.std(si) * np.std(sj)))
            correlation_matrix[i][j] = corr
            correlation_matrix[j][i] = corr

axis_ticks = ["Simple"] + [f"<i>Spectro</i> {v}" for v in LIST_SPECTRO] + [f"Pics {v}" for v in LIST_SPIKES]

fig = go.Figure(data=[
    go.Heatmap(
        x=np.linspace(0, len(axis_ticks), len(axis_ticks), endpoint=False),
        y=np.linspace(0, len(axis_ticks), len(axis_ticks), endpoint=False),
        z=correlation_matrix, zmax=1, zmin=-1, colorscale="RdYlGn",
        text=[[str(round(val, 1)) for val in line] for line in correlation_matrix],
        texttemplate="%{text}",
        textfont={"size": 9}),
])
fig.update_yaxes(autorange="reversed")

fig.update_xaxes(
    linewidth=1,
    linecolor='black',
    mirror=True,
    ticks='outside',
    scaleratio=1,
    tickmode='array',
    tickvals=np.linspace(0, len(axis_ticks), len(axis_ticks), endpoint=False),
    ticktext=axis_ticks
)
fig.update_yaxes(

```

```

linewidth=1,
linecolor='black',
mirror=True,
ticks='outside',
scaleratio=1,
tickmode='array',
tickvals=np.linspace(0, len(axis_ticks), len(axis_ticks), endpoint=False),
ticktext=axis_ticks
)

fig.update_layout(
    template='plotly_white',
)

if FILE == "F:/DATASET/ALI TIPE/3 result/results_stats_TRILATERATION_ALL.db":
    fig.update_layout(title={
        "text": r"$\Large{\text{Matrice de corrélation des erreurs des différentes}} \\\ \Large{\text{méthodes (par trilatération)}}$"
    })
    fig.write_image("results/4 fig7 tri lr.webp", scale=1.3 * 2, height=600, width=600)
    fig.write_image("results/4 fig7 tri hr.png", scale=13, height=600, width=600)

else:
    fig.update_layout(title={
        "text": r"$\Large{\text{Matrice de corrélation des erreurs des différentes}} \\\ \Large{\text{méthodes (par optimisation)}}$"
    })

    fig.write_image("results/4 fig7 opt lr.webp", scale=1.3, height=600, width=600)
    fig.write_image("results/4 fig7 opt hr.png", scale=13, height=600, width=600)

fig.show()

#####
###
# ## Final result

from plotly.subplots import make_subplots

```

```
conn = sqlite3.connect("F:/DATASET/ALI TIPE/3 result/result_prceise.db", check_same_thread=False)
cursor = conn.cursor()
```

```
fig = make_subplots(specs=[[{"secondary_y": True}]])
```

```
for k in range(1, 11):
```

```
    x = []
```

```
    y = []
```

```
    s = [f"s{i}" for i in range(1, 11)]
```

```
    cursor.execute("SELECT " + ", ".join(s) + " FROM results WHERE k_sources = {k};")
```

```
    rows = cursor.fetchall()
```

```
    counter = [0 for i in range(1, 11)]
```

```
    for row in rows:
```

```
        for i, v in enumerate(row):
```

```
            if v != -1 and i + 1 < 5:
```

```
                x.append(str(i + 1))
```

```
                y.append(v)
```

```
                counter[i] += 1
```

```
    fig.add_trace(
```

```
        go.Box(x=x,
```

```
              y=y,
```

```
              name=f"{k} sources" if k > 1 else f"{k} source",
```

```
              boxpoints=False,
```

```
              notched=False,
```

```
              boxmean="sd",
```

```
              line=dict(width=1.75),
```

```
              ),
```

```
        secondary_y=False
```

```
    )
```

```
black_dote = True
```

```
for k in range(1, 11):
```

```
    x = []
```

```

y = []

s = [f"s{i}" for i in range(1, 11)]

cursor.execute("SELECT " + ", ".join(s) + f" FROM results WHERE k_sources = {k};")

rows = cursor.fetchall()
counter = [0 for i in range(1, 11)]
for row in rows:
    for i, v in enumerate(row):
        if v != -1 and i + 1 < 5:
            counter[i] += 1

fig.add_trace(go.Scatter(
    x=[str(i) for i in [1, 2, 3, 4]], y=counter,
    marker_line_color="black",
    marker_line_width=2,
    marker_size=4,
    line=dict(width=1.5, color="black"),
    mode='markers',
    name="Nb. cand. proposé",
    showlegend=black_dote,
), secondary_y=True
)

black_dote = False

fig.update_layout(
    scattergap=0.30,
    boxmode='group',
    scattermode="group",
    template='plotly_white',
    legend=dict(
        yanchor="top",
        y=1.0,
        xanchor="left",
        x=0.0,
        orientation="h",
    ),

```

```

title={"text": r"$\Large{\text{Erreur des candidats de la méthode spectrogramme 128 }} \\\ \"
      r"\Large{\text{par rapport au nombre de sources présentes }} \\\ \"
      r"1000 \text{ mesures aléatoires par boîte }$"},
xaxis={"title": {"text": '$\Large{k^e}\text{ coordonnées candidat}}$'}},
yaxis={"title": {"text": "$\Large{\text{Erreur en } m }$"}, "range": [0, 25]}
)

fig.update_yaxes(
    title={"text": "$\Large{\text{Nombre de candidat proposé }}$"}, range=[0, 1000],
    secondary_y=True,
)

fig.write_image("results/4 fig8 lr.webp", scale=1.3, height=600, width=1038)
fig.write_image("results/4 fig8 hr.png", scale=13, height=600, width=1038)

```

chart/**5_test_isolate.ipynb**

Etude statistique de l'isolation

```
#####
###
# # Resultats isolation
import sqlite3
import plotly.graph_objects as go
import plotly.io as pio

import numpy as np

pio.renderers.default = "browser"

DISK_F = "F:"
conn = sqlite3.connect(f"{DISK_F}/DATASET/ALI TIPE/5 result/res{9}.db", check_same_thread=False)
cursor = conn.cursor()

#####
###
# ## Methodes induitives
# - ICA
# - PCA

SK_MODEL_NAME = ["NMF", "FastICA", "PCA", "TruncatedSVD"]
SK_MAP = {"NMF": "NMF", "FastICA": "ICA", "PCA": "PCA", "TruncatedSVD": "SVD", "R_GRU_256": "GRU 256 (Brut)"}

res = {m: [[], [], []] for m in SK_MODEL_NAME}

for m in SK_MODEL_NAME:
    cursor.execute(f"SELECT avg, k, ABS(fft) FROM {m};")

    for avg, k, corr in cursor.fetchall():
        res[m][0].append(str(k))
        res[m][1].append(avg)
        res[m][2].append(corr)

# Correlation
fig = go.Figure()

for key, value in res.items():
    fig.add_trace(
```

```

        go.Box(x=value[0],
               y=value[2],
               name=SK_MAP[key],
               boxpoints=False,
               notched=False,
               boxmean=True,
               line=dict(width=1.75),
               )
    )

fig.update_layout(
    boxmode='group',
    template='plotly_white',
    legend=dict(
        yanchor="top",
        y=1.0,
        xanchor="left",
        x=0.00,
        orientation="h",
    ),
    title={"text": r"$\Large{\text{Corrélation entre le signal original et le signal isolé }} \\\ "
           r"1000 \text{ mesures aléatoires par boîte }$"},
    xaxis={"title": {"text": '$\Large{\text{Nombre de sources}}$'}},
    yaxis={"title": {"text": "$\Large{\text{Corrélation}}$"}, "range": [0, 1.1]}
)

fig.write_image("results/5 fig1 lr.webp", scale=1.3, height=600, width=1038)
fig.write_image("results/5 fig1 hr.png", scale=13, height=600, width=1038)

# Erreur
fig = go.Figure()

for key, value in res.items():
    fig.add_trace(
        go.Box(x=value[0],
               y=value[1],
               name=SK_MAP[key],
               notched=True,
               boxpoints=False,

```



```

        boxmean=True,
        line=dict(width=1.75),
    )
)

cursor.execute(f"SELECT AVG(avg), k FROM random GROUP BY k ORDER BY k ASC;")

err_y = [*map(lambda c: c[0], cursor.fetchall())]

fig.add_trace(go.Scatter(
    name="Aléatoire",
    x=[str(i) for i in [1, 2, 3, 5, 7]], y=err_y,
    marker_line_color="black",
    marker_symbol="line-ew",
    marker_line_width=2,
    marker_size=100,
    mode='markers',
),
)

fig.update_layout(
    boxmode='group',
    template='plotly_white',
    legend=dict(
        yanchor="top",
        y=1.0,
        xanchor="left",
        x=0.00,
        orientation="h", ),
    title={"text": r"$\Large{\text{Erreur entre le signal original et le signal isolé }} \\\ "
        r"1000 \text{ mesures aléatoires par boîte }$"},
    xaxis={"title": {"text": '$\Large{\text{Nombre de sources}}$'}},
    yaxis={"title": {"text": "$\Large{\text{Erreur}}$"}, "range": [0, 1.1]}
)

fig.write_image("results/5 fig2 lr.webp", scale=1.3, height=600, width=1038)
fig.write_image("results/5 fig2 hr.png", scale=13, height=600, width=1038)

```

```
#####
###
# ## Comparaison Spectro et Brute

MODELS = ['R LSTM 64', 'R LSTM 128', 'R LSTM 256', 'R LSTM 64-64', 'R LSTM 128-128', 'R GRU 64', 'R GRU 128',
          'R GRU 256', 'R GRU 64-64', 'R GRU 128-128', 'R Bi-GRU 64', 'R Bi-GRU 128', 'R Bi-GRU 256', 'R Bi-GRU 64-64',
          'R Bi-GRU 128-128', 'R Bi-LSTM 64', 'R Bi-LSTM 128', 'R Bi-LSTM 256', 'R Bi-LSTM 64-64', 'R Bi-LSTM 128-128',
          'S LSTM 64', 'S LSTM 128', 'S LSTM 256', 'S LSTM 64-64', 'S LSTM 128-128', 'S GRU 64', 'S GRU 128',
          'S GRU 256', 'S GRU 64-64', 'S GRU 128-128', 'S Bi-GRU 64', 'S Bi-GRU 128', 'S Bi-GRU 256', 'S Bi-GRU 64-64',
          'S Bi-GRU 128-128', 'S Bi-LSTM 64', 'S Bi-LSTM 128', 'S Bi-LSTM 256', 'S Bi-LSTM 64-64', 'S Bi-LSTM 128-128']

MODELS = [*map(lambda x: x.replace(' ', '_').replace('-', '_'), MODELS)]

#####
###
# Correlation

fig = go.Figure()

x_spectro = []
y_spectro = []

for av in ["R", "S"]:
    for m in MODELS:
        if m[0] == av:
            continue
        cursor.execute(f"SELECT fft, k FROM {m};")

        for c, k in cursor.fetchall():
            x_spectro.append(str(k))
            y_spectro.append(abs(c))

fig.add_trace(
    go.Violin(x=x_spectro,
              y=y_spectro,
```

```

        box_visible=False,
        points=False,
        meanline_visible=True,
        line_color='#636EFA' if av == "S" else '#00CC96',
        side='negative' if av == "S" else 'positive',
        name="Brut" if av == "S" else "Spectrogramme",
        width=1.6,
        spanmode='hard'),
    )

fig.update_layout(
    violinmode='overlay',
    template='plotly_white',
    legend=dict(
        yanchor="top",
        y=1.0,
        xanchor="left",
        x=0.025,
        orientation="h", ),
    title={"text": r"$\Large{\text{Corr lation entre le signal original et le signal isol }} \\\ "
            r"1000 \text{ mesures al atoires par violon }$"},
    xaxis={"title": {"text": '$\Large{\text{Nombre de sources}}$'}},
    yaxis={"title": {"text": "$\Large{\text{Corr lation}}$"}, "range": [0, 1.1]}
)

fig.write_image("results/5 fig3 lr.webp", scale=1.3, height=600, width=1038)
fig.write_image("results/5 fig3 hr.png", scale=13, height=600, width=1038)

#####
###
# Erreur

fig = go.Figure()

x_spectro = []
y_spectro = []

for av in ["R", "S"]:
    for m in MODELS:

```

```

    if m[0] == av:
        continue
    cursor.execute(f"SELECT avg, k FROM {m};")

    for c, k in cursor.fetchall():
        x_spectro.append(str(k))
        y_spectro.append(c)

fig.add_trace(
    go.Violin(x=x_spectro,
              y=y_spectro,
              box_visible=False,
              points=False,
              meanline_visible=True,
              line_color='#636EFA' if av == "S" else '#00CC96',
              side='negative' if av == "S" else 'positive',
              name="Brut" if av == "S" else "Spectrogramme",
              width=1.3,
              spanmode='hard'),
)

cursor.execute(f"SELECT AVG(avg), k FROM random GROUP BY k ORDER BY k ASC;")

err_y = [*map(lambda c: c[0], cursor.fetchall())]

fig.add_trace(go.Scatter(
    name="Aléatoire",
    x=[str(i) for i in [1, 2, 3, 5, 7]], y=err_y,
    marker_line_color="red",
    marker_symbol="line-ew",
    marker_line_width=2,
    marker_size=50,
    mode='markers',
),
)

fig.update_layout(
    violinmode='overlay',
    template='plotly_white',

```

```

legend=dict(
    yanchor="top",
    y=1.0,
    xanchor="left",
    x=0.00,
    orientation="h",
),
title={"text": r"$\Large{\text{Erreur entre le signal original et le signal isolé}} \\\ "
        r"1000 \text{ mesures aléatoires par violon }$"},
xaxis={"title": {"text": '$\Large{\text{Nombre de sources}}$'}},
yaxis={"title": {"text": "$\Large{\text{Erreur}}$"}, "range": [0, 1.1]}
)

```

```

fig.write_image("results/5 fig4 lr.webp", scale=1.3, height=600, width=1038)
fig.write_image("results/5 fig4 hr.png", scale=13, height=600, width=1038)

```

```

#####
###
# ## Comparaison model

```

```

def model_details(name: str) -> dict:
    details = {}

    architecture = {"128_128": "128-128", "64_64": "64-64", "64": "64", "128": "128", "256": "256"}

    for key, value in architecture.items():
        if key in name:
            details["architecture"] = value
            break

    details["input"] = name[0]

    model_type = {"Bi_GRU": "Bi-GRU", "Bi_LSTM": "Bi-LSTM", "LSTM": "LSTM", "GRU": "GRU"}

    for key, value in model_type.items():
        if key in name:
            details["type"] = value
            break

```

```

    return details

#####
###
# ### Comparaison architecture

res = {n: [[], [], []] for n in ['128-128', '64-64', '64', '128', '256']}

for m in MODELS:

    det = model_details(m)

    if det["input"] == "R":
        cursor.execute(f"SELECT avg, k, ABS(fft) FROM {m};")

        for avg, k, corr in cursor.fetchall():
            res[det["architecture"]][0].append(str(k))
            res[det["architecture"]][1].append(avg)
            res[det["architecture"]][2].append(corr)

# Correlation
fig = go.Figure()

for key, value in res.items():
    fig.add_trace(
        go.Box(x=value[0],
               y=value[2],
               name=key,
               boxpoints=False,
               notched=True,
               boxmean=True,
               line=dict(width=1.75),
               )
    )

fig.update_layout(
    boxmode='group',
    template='plotly_white',
    legend=dict(

```

```

        yanchor="top",
        y=1.0,
        xanchor="left",
        x=0.00,
        orientation="h",
    ),
    title={"text": r"$\Large{\text{Corrélation entre le signal original et le signal isolé }} \\\ "
          r"1000 \text{ mesures aléatoires par boîte }$"},
    xaxis={"title": {"text": '$\\Large{\\text{Nombre de sources}}$'}},
    yaxis={"title": {"text": "$\\Large{\\text{Corrélation}}$"}, "range": [0, 1.1]}
)

```

```

fig.write_image("results/5 fig5 lr.webp", scale=1.3, height=600, width=1038)
fig.write_image("results/5 fig5 hr.png", scale=13, height=600, width=1038)

```

Erreur

```
fig = go.Figure()
```

```

for key, value in res.items():
    fig.add_trace(
        go.Box(x=value[0],
               y=value[1],
               name=key,
               notched=True,
               boxpoints=False,
               boxmean=True,
               line=dict(width=1.75),

               )
    )

```

```
cursor.execute(f"SELECT AVG(avg), k FROM random GROUP BY k ORDER BY k ASC;")
```

```
err_y = [*map(lambda c: c[0], cursor.fetchall())]
```

```

fig.add_trace(go.Scatter(
    name="Aléatoire",
    x=[str(i) for i in [1, 2, 3, 5, 7]], y=err_y,
    marker_line_color="black",

```

```

        marker_symbol="line-ew",
        marker_line_width=2,
        marker_size=100,
        mode='markers',
    ),
)

fig.update_layout(
    boxmode='group',
    template='plotly_white',
    legend=dict(
        yanchor="top",
        y=1.0,
        xanchor="left",
        x=0.00,
        orientation="h", ),
    title={"text": r"$\Large{\text{Erreur entre le signal original et le signal isolé}} \\\ "
           r"1000 \text{ mesures aléatoires par boîte }$"},
    xaxis={"title": {"text": '$\Large{\text{Nombre de sources}}$'}},
    yaxis={"title": {"text": "$\Large{\text{Erreur}}$"}, "range": [0, 1.1]}
)

```

```

fig.write_image("results/5 fig6 lr.webp", scale=1.3, height=600, width=1038)
fig.write_image("results/5 fig6 hr.png", scale=13, height=600, width=1038)

```

```
#####
```

```
###
```

```
# ### Compare Layer type
```

```
res = {n: [[], [], []] for n in ['Bi-GRU', 'Bi-LSTM', 'LSTM', 'GRU']}
```

```
for m in MODELS:
```

```
    det = model_details(m)
```

```
    if det["input"] == "R":
        cursor.execute(f"SELECT avg, k, ABS(fft) FROM {m};")
```

```
        for avg, k, corr in cursor.fetchall():
```



```

        res[det["type"]][0].append(str(k))
        res[det["type"]][1].append(avg)
        res[det["type"]][2].append(corr)

# Correlation
fig = go.Figure()

for key, value in res.items():
    fig.add_trace(
        go.Box(x=value[0],
               y=value[2],
               name=key,
               boxpoints=False,
               notched=True,
               boxmean=True,
               line=dict(width=1.75),
               )
    )

fig.update_layout(
    boxmode='group',
    template='plotly_white',
    legend=dict(
        yanchor="top",
        y=1.0,
        xanchor="left",
        x=0.00,
        orientation="h",
    ),
    title={"text": r"$\Large{\text{Corr lation entre le signal original et le signal isol }} \\\ "
            r"1000 \text{ mesures al atoires par bo te }}$"},
    xaxis={"title": {"text": '$\Large{\text{Nombre de sources}}$'}},
    yaxis={"title": {"text": "$\Large{\text{Corr lation}}$", "range": [0, 1.1]}}
)

fig.write_image("results/5 fig7 lr.webp", scale=1.3, height=600, width=1038)
fig.write_image("results/5 fig7 hr.png", scale=13, height=600, width=1038)

# Erreur

```

```

fig = go.Figure()

for key, value in res.items():
    fig.add_trace(
        go.Box(x=value[0],
               y=value[1],
               name=key,
               notched=True,
               boxpoints=False,
               boxmean=True,
               line=dict(width=1.75),

               )
    )

cursor.execute(f"SELECT AVG(avg), k FROM random GROUP BY k ORDER BY k ASC;")

err_y = [*map(lambda c: c[0], cursor.fetchall())]

fig.add_trace(go.Scatter(
    name="Aléatoire",
    x=[str(i) for i in [1, 2, 3, 5, 7]], y=err_y,
    marker_line_color="black",
    marker_symbol="line-ew",
    marker_line_width=2,
    marker_size=100,
    mode='markers',
),
)

fig.update_layout(
    boxmode='group',
    template='plotly_white',
    legend=dict(
        yanchor="top",
        y=1.0,
        xanchor="left",
        x=0.00,
        orientation="h", ),

```

```

        title={"text": r"$\Large{\text{Erreur entre le signal original et le signal isolé }} \\\ "
                r"1000 \text{ mesures aléatoires par boîte }$"},
        xaxis={"title": {"text": '$\\Large{\\text{Nombre de sources}}$'}},
        yaxis={"title": {"text": "$\\Large{\\text{Erreur}}$"}, "range": [0, 1.1]}
    )

fig.write_image("results/5 fig8 lr.webp", scale=1.3, height=600, width=1038)
fig.write_image("results/5 fig8 hr.png", scale=13, height=600, width=1038)

#####
###
# ## Compare with Sk

M2 = SK_MODEL_NAME + ["R_GRU_256"]

res = {m: [[], [], []] for m in M2}

for m in M2:
    cursor.execute(f"SELECT avg, k, ABS(fft) FROM {m};")

    for avg, k, corr in cursor.fetchall():
        res[m][0].append(str(k))
        res[m][1].append(avg)
        res[m][2].append(corr)

# Correlation
fig = go.Figure()

for key, value in res.items():
    fig.add_trace(
        go.Box(x=value[0],
                y=value[2],
                name=SK_MAP[key],
                boxpoints=False,
                notched=True,
                boxmean=True,
                line=dict(width=1.75),
                )
    )

```

```

fig.update_layout(
    boxmode='group',
    template='plotly_white',
    legend=dict(
        yanchor="top",
        y=1.0,
        xanchor="left",
        x=0.0,
        orientation="h",
    ),
    title={"text": r"$\Large{\text{Corrélation entre le signal original et le signal isolé }} \\\ "
        r"1000 \text{ mesures aléatoires par boîte }}$"},
    xaxis={"title": {"text": '$\Large{\text{Nombre de sources}}$'}},
    yaxis={"title": {"text": "$\Large{\text{Corrélation}}$"}, "range": [0, 1.1]}
)

```

```

fig.write_image("results/5 fig9 lr.webp", scale=1.3, height=600, width=1038)
fig.write_image("results/5 fig9 hr.png", scale=13, height=600, width=1038)

```

Erreur

```
fig = go.Figure()
```

```

for key, value in res.items():
    fig.add_trace(
        go.Box(x=value[0],
            y=value[1],
            name=SK_MAP[key],
            notched=True,
            boxpoints=False,
            boxmean=True,
            line=dict(width=1.75),
        )
    )

```

```
cursor.execute(f"SELECT AVG(avg), k FROM random GROUP BY k ORDER BY k ASC;")
```

```
err_y = [*map(lambda c: c[0], cursor.fetchall())]
```

```

fig.add_trace(go.Scatter(
    name="Aléatoire",
    x=[str(i) for i in [1, 2, 3, 5, 7]], y=err_y,
    marker_line_color="black",
    marker_symbol="line-ew",
    marker_line_width=2,
    marker_size=100,
    mode='markers',
),
)

fig.update_layout(
    boxmode='group',
    template='plotly_white',
    legend=dict(
        yanchor="top",
        y=1.0,
        xanchor="left",
        x=0.00,
        orientation="h", ),
    title={"text": r"$\Large{\text{Erreur entre le signal original et le signal isolé}} \\\ \"
        r"1000 \text{ mesures aléatoires par boîte }$"},
    xaxis={"title": {"text": '$\Large{\text{Nombre de sources}}$'}},
    yaxis={"title": {"text": "$\Large{\text{Erreur}}$"}, "range": [0, 1.1]}
)

```

```

fig.write_image("results/5 fig10 lr.webp", scale=1.3, height=600, width=1038)
fig.write_image("results/5 fig10 hr.png", scale=13, height=600, width=1038)

```

```

#####
###
# ## Matrice de corrélation

```

```

M = SK_MODEL_NAME + MODELS

```

```

N = len(M)

```

```

correlation_matrix = np.ones((N, N))

```

```

for i in range(N):
    for j in range(N):
        if i < j:
            print(f"\r{i} {j}", end="")
            m1 = M[i]
            m2 = M[j]
            cursor.execute(f"SELECT {m1}.avg, {m2}.avg FROM {m1} JOIN {m2} ON ("
                           f"{m1}.k = {m2}.k AND {m1}.num = {m2}.num AND {m1}.i = {m2}.i)"
                           f";")

            arr1 = []
            arr2 = []

            for a1, a2 in cursor.fetchall():
                arr1.append(a1)
                arr2.append(a2)

            corr = np.corrcoef(arr1, arr2)[0, 1]
            correlation_matrix[i][j] = corr
            correlation_matrix[j][i] = corr

fig = go.Figure(data=[
    go.Heatmap(
        x=np.flip(np.linspace(0, len(M), len(M), endpoint=False)),
        y=np.linspace(0, len(M), len(M), endpoint=False),
        z=correlation_matrix, zmax=1, zmin=-1, colorscale="RdYlGn",
        text=[[str(round(val, 1)) for val in line] for line in correlation_matrix],
        texttemplate="%{text}",
        textfont={"size": 5},
        transpose=True),
])

fig.update_xaxes(
    linewidth=1,
    linecolor='black',
    mirror=True,
    ticks='outside',
    scaleratio=1,
    tickmode='array',

```

```

    tickvals=np.flip(np.linspace(0, len(M), len(M), endpoint=False)),
    ticktext=M
)
fig.update_yaxes(
    linewidth=1,
    linecolor='black',
    mirror=True,
    ticks='outside',
    scaleratio=1,
    tickmode='array',
    tickvals=np.linspace(0, len(M), len(M), endpoint=False),
    ticktext=M
)

fig.update_layout(
    template='plotly_white',
    title={
        "text": r"$\LARGE{\text{Matrice de corrélation des erreurs des différents modèles}}$"
    }
)

fig.write_image("results/5 fig11 lr.webp", scale=1.3 * 2, height=600 * 1.5, width=600 * 1.5)
fig.write_image("results/5 fig11 hr.png", scale=13, height=600 * 1.5, width=600 * 1.5)

```

chart/**5_test_isolate.py**

Génération de statistique pour l'isolation


```

# __import__("os").environ['CUDA_VISIBLE_DEVICES'] = '-1'
__import__("os").environ["KERAS_BACKEND"] = "jax"

from itertools import product
from multiprocessing import Pool
import os
import pickle
import random
import sqlite3
import numpy as np
from sklearn.linear_model import LinearRegression

from environnement import DEFAULT_TEMPERATURE
from environnement import euclidian_distance
from environnement import Environment
from environnement import Microphone
from environnement import Source
from environnement import Sound

from audio_locate_isolate.isolate import isolate_source_rnn_triple
from audio_locate_isolate.isolate import isolate_source_sklearn
from audio_locate_isolate.isolate import SK_MODEL_NAME
from audio_locate_isolate.utils import normalize_array

from audio_locate_isolate.neural_networks import MODELS

RATE = 44100
DISK_C = "/mnt/c" # /mnt/c C:
DISK_F = "/mnt/f" # /mnt/f F:
DISK_C = "C:" # /mnt/c C:
DISK_F = "F:" # /mnt/f F:
PATH_1 = f"{DISK_C}/Users/Utilisateur/Documents/Python/ADVANCED/GeoSoundTIPE/sound_data/"
PATH_2 = f"{DISK_C}/Users/Utilisateur/Documents/Python/ADVANCED/GeoSoundTIPE/sound_data/test/"

sounds_list_wav_name = ([PATH_1 + f for f in os.listdir(PATH_1) if "test" not in f] +
                        [PATH_2 + f for f in os.listdir(PATH_2) if "test" not in f])

conn = sqlite3.connect(f"{DISK_F}/DATASET/ALI TIPE/5 result/res{9}.db", check_same_thread=False)
cursor = conn.cursor()

```

```

def random_records(sounds_list_name: list[str],

    mic_coordinate_interval: tuple[float, float] = (-25, 25),
    min_mics_distance: float = 2.0,

    source_amount: int = 1,
    source_coordinate_interval: tuple[float, float] = (-25, 25),
    min_sources_distance: float = 1.0,

    record_start: tuple[int, int] = (5, 85),
    data_length: int = 44100,

    temperature: float = DEFAULT_TEMPERATURE
):
    min_mic_coordinate, max_mic_coordinate = mic_coordinate_interval
    min_source_coordinate, max_source_coordinate = source_coordinate_interval

    # Create microphones
    mics = []

    for k in range(3):
        x1, y1 = np.random.uniform(min_mic_coordinate, max_mic_coordinate, (2,))
        z1 = 0

        # Check the minimum distance between microphones
        while any(map(lambda m: euclidian_distance(x1, y1, z1, *m.get_coord) < min_mics_distance, mics)):
            x1, y1 = np.random.uniform(min_mic_coordinate, max_mic_coordinate, (2,))
            z1 = 0

        mics.append(Microphone(f"m{k}", x1, y1, z1))

    # Create sources
    sources = []
    coordinates = []

    for k in range(source_amount):
        x, y = np.random.uniform(min_source_coordinate, max_source_coordinate, (2,))

```

```

z = 0

# Check the minimum distance between sources and microphones
while (any(map(lambda c: euclidian_distance(x, y, z, *c) < min_sources_distance, coordinates)) and
      any(map(lambda m: euclidian_distance(x, y, z, *m.get_coord) < min_mics_distance, mics))):
    x, y = np.random.uniform(min_source_coordinate, max_source_coordinate, (2,))
    z = 0

sources.append(Source(
    sound=Sound(random.choice(sounds_list_name), rate=RATE),
    name=f"s{k}",
    x=x, y=y, z=z
))
coordinates.append((x, y, z))

# Create the environment
env = Environment(
    sources=sources,
    microphones=mics,
    rate=None,
    microphone_tensors=[],
    temperature=temperature
)
min_sound_start, max_sound_start = record_start
start = random.uniform(min_sound_start, max_sound_start)
end = (start + data_length / env.get_rate * 1.0)

_, records = env.listen_from_microphone(["m0", "m1", "m2"], start, end, start - 1,
                                       attenuate=True,
                                       compress=True,
                                       noise=0.0
                                       )

records = [r for r in records]

# Select a source to isolate
selected = random.choice(list(env.sources.values()))
x, y, z = selected.get_coord

_, result = env.listen_from_microphone((x, y, z),

```

```
start, end, start - 1,  
attenuate=False,  
specific_sound=[selected.name],  
compress=True,  
noise=0.0)
```

```
return records, result[0], [*env.microphones.values()], (x, y, z)
```

```
def create_dataset(args):
```

```
    k_source, num = args
```

```
    print("\nFile : ", k_source, num)
```

```
    res = []
```

```
    for i in range(100):
```

```
        print(f"\r{k_source} - {num} - {i}", end="")
```

```
        rr = random_records(sounds_list_wav_name, source_amount=k_source)
```

```
        res.append(rr)
```

```
    with open(f'{DISK_F}/DATASET/ALI TIPE/5 result/test_data/data_{k_source}_{num}.pickle', 'wb') as handle:
```

```
        pickle.dump(res, handle)
```

```
def init_db():
```

```
    try:
```

```
        cursor.execute("""
```

```
            CREATE TABLE "traite"
```

```
            (
```

```
                "model" TEXT NOT NULL,
```

```
                "k" INTEGER NOT NULL,
```

```
                "num" INTEGER
```

```
            );
```

```
        """)
```

```
    except Exception as e:
```

```
        print(e)
```

```

for model_name in MODELS:
    try:
        cursor.execute(f""" CREATE TABLE "{model_name.replace(' ', '_').replace('-', '_)}" (
            "k" INTEGER NOT NULL,
            "num"    INTEGER NOT NULL,
            "i" INTEGER NOT NULL,
            "avg"    REAL NOT NULL,
            "fft"    REAL NOT NULL,
            "std"    REAL NOT NULL
        );
        """)
    except Exception as e:
        print(e)

for name in SK_MODEL_NAME:
    try:
        cursor.execute(f""" CREATE TABLE "{name.replace(' ', '_').replace('-', '_)}" (
            "k" INTEGER NOT NULL,
            "num"    INTEGER NOT NULL,
            "i" INTEGER NOT NULL,
            "avg"    REAL NOT NULL,
            "fft"    REAL NOT NULL,
            "std"    REAL NOT NULL
        );
        """)
    except Exception as e:
        print(e)

try:
    cursor.execute(f"""CREATE TABLE "random" (
        "k" INTEGER NOT NULL,
        "num"    INTEGER NOT NULL,
        "i" INTEGER NOT NULL,
        "avg"    REAL NOT NULL,
        "fft"    REAL NOT NULL,
        "std"    REAL NOT NULL
    );
    """)
except Exception as e:

```

```

print(e)

def max_recoverable_information(x, y_list):
    y_combined = np.column_stack(y_list)

    model = LinearRegression()
    model.fit(y_combined, x)

    x_pred = model.predict(y_combined)

    return np.average(np.abs(x - x_pred)), np.std(np.abs(x - x_pred)) / 0.31

def test_model(args):
    model_name, k_sources, num = args

    cursor.execute("SELECT * FROM traite WHERE model = ? AND k = ? AND num = ?;", (model_name, k_sources, num))
    already = cursor.fetchall()

    if len(already) == 0:
        sql_model_name = model_name.replace(' ', '_').replace('-', '_')

        data = pickle.load(
            open(f"{DISK_F}/DATASET/ALI TIPE/5 result/test_data_OLD/data_{k_sources}_{num}.pickle", "rb"))

        # Check the type of model
        if model_name not in ["random", "entropy"] + SK_MODEL_NAME:
            # Neural network models
            model = MODELS[model_name]
            _model_res = np.load(

f"{DISK_C}/Users/Utilisateur/Documents/Python/ADVANCED/GeoSoundTIPE/audio_locate_isolate/res/{model_name}.npy",
            allow_pickle=True).item()
            # _model_res = np.load(f"{DISK_F}/DATASET/ALI TIPE/5 result/res/{model_name}.npy",
allow_pickle=True).item()

            weights = _model_res["weights"][-1]
            model.set_weights(weights)

```

```

del _model_res
del weights

print(f"\n{model_name} - {k_sources} - {num} Started")

cursor.execute(f"DELETE FROM {sql_model_name} WHERE k = ? AND num = ?", (k_sources, num))
conn.commit()

# For each test records
for i, (records, result, mics, (x, y, z)) in enumerate(data):
    # If record is random
    if model_name == "random":
        pred = np.random.normal(-1, 1, size=(len(records[0]),))

    # If model is Sklearn model
    elif model_name in SK_MODEL_NAME:
        """pred_tot = isolate_source_sklearn(x, y, z, microphones=mics, records=records, sample_rate=44100,
                                             methode_name=model_name)

        l = int(len(pred_tot[0]) / 1.5)

        def choice(x1, x2):
            return np.average(np.abs(x1 - x2))

        pred = max(pred_tot, key=lambda p: np.corrcoef(result[:l], p[:l])[0][1])"""
        pred = isolate_source_sklearn(x, y, z, microphones=mics, records=records, sample_rate=44100,
                                       methode_name=model_name)

    # If model is a neural network
    else:
        pred = isolate_source_rnn_triple(x, y, z, microphones=mics, records=records, sample_rate=44100,
                                         model=model)

l = int(len(pred) / 1.5)
s1 = normalize_array(result[:l], -1, 1)
s2 = normalize_array(pred[:l], -1, 1)

```

```

err = np.abs(s1 - s2)
avg = np.average(err)
std = np.std(err)
fft = np.corrcoef(s1, s2)[0][1]

print(f"\r{model_name} {k_sources} {num} {i} {avg / 0.31 * 100:.3f}% {fft:.3f}", end="")

cursor.execute(f"INSERT INTO {sql_model_name} (k, num, i, avg, std, fft) VALUES (?, ?, ?, ?, ?, ?)",
               (k_sources, num, i, avg, std, fft))
conn.commit()

cursor.execute(f"INSERT INTO traite (model, k, num) VALUES (?, ?, ?)", (model_name, k_sources, num))
conn.commit()

else:
    print(f"\n{model_name} - {k_sources} - {num} Already done")

"""if __name__ == '__main__':
    # Create the dataset
    # n sources, file id
    # r = set(product([1, 2, 3, 5, 7], range(10)))
    r = set(product([4, 10], range(10)))

    with Pool(processes=7) as pool:
        map_result = pool.map(create_dataset, [*r])
        for k in map_result:
            print(k)
    print("end")"""

if __name__ == '__main__':
    # Use model on dataset
    init_db()
    config = [*product(SK_MODEL_NAME, [1, 2, 3, 5, 7], range(10))]

    for args in config:
        test_model(args)
    print("end")

```


`chart/`**charts_utils.py**

Fonctions utilitaires pour les graphiques

```

import plotly.colors
import numpy as np

def cercle_circonscrit(a, b, c):
    x1, y1 = a

    x2, y2 = b
    x3, y3 = c

    d = 2 * (x1 * (y2 - y3) + x2 * (y3 - y1) + x3 * (y1 - y2))

    x_centre = ((x1 ** 2 + y1 ** 2) * (y2 - y3) + (x2 ** 2 + y2 ** 2) * (y3 - y1) + (x3 ** 2 + y3 ** 2) * (y1 - y2))
    / d
    y_centre = ((x1 ** 2 + y1 ** 2) * (x3 - x2) + (x2 ** 2 + y2 ** 2) * (x1 - x3) + (x3 ** 2 + y3 ** 2) * (x2 - x1))
    / d

    return float(x_centre), float(y_centre)

def custom_colorscale(red_value=0, n_colors=100, v_min=0, v_max=1, colorscale='Plasma', replace="rgb(255,0,0)"):
    colors = plotly.colors.sample_colorscale(colorscale, n_colors)
    data_min = v_min
    data_max = v_max

    normalized_red_value = (red_value - data_min) / (data_max - data_min)

    positions = np.linspace(0, 1, n_colors)

    red_idx = np.argmin(np.abs(positions - normalized_red_value))

    custom_colorscale = []
    for i, (pos, color) in enumerate(zip(positions, colors)):
        if i == red_idx:
            custom_colorscale.append((pos, replace))
        else:
            custom_colorscale.append((pos, color))

    return custom_colorscale

```

/environnement.py

Environnement de simulation du son

```
"""
```

```
Environment de simulation du son dans un espace.
```

```
"""
```

```
from copy import deepcopy
from math import floor
from typing import Callable
```

```
import numpy as np
from scipy.io import wavfile
from scipy.io.wavfile import write
```

```
DEFAULT_TEMPERATURE: float = 20.0
SOURCE_RADIUS: float = 0.1
COMPRESS_ALPHA: float = 0.9
NOISE_ALPHA: float = 0.00
```

```
def play_sound(sound: np.ndarray, rate: int = 44100) -> None:
```

```
    """
```

```
    This function plays sound in headphone.
```

```
    :param sound: Sound to play.
```

```
    :param rate: The rate of the music in Hz.
```

```
    :return: None
```

```
    """
```

```
    import pyaudio
```

```
    sound = np.clip(sound, -1, 1)
```

```
    p = pyaudio.PyAudio()
```

```
    stream = p.open(format=pyaudio.paFloat32,
                     channels=1,
                     rate=rate,
                     output=True)
```

```
    stream.write(sound.astype(np.float32).tobytes())
```

```
    stream.stop_stream()
```

```
    stream.close()
```

```
    p.terminate()
```

```
def save_sound(name: str, sound: np.ndarray, rate: int) -> None:
```

```
    """
```

```
    This function saves a sound as an .wav file.
```

```
    :param name: Name of the file without ".wav"
```

```
    :param sound: The sound as an array of float
```

```
    :param rate: The rate of the sound in Hz.
```

```
    :return: None
```

```
    """
```

```
    scaled = np.int16(sound / np.max(np.abs(sound)) * 32767)
```

```
    write(f'{name}.wav', rate, scaled)
```

```
def define_sound_speed(temperature: float = DEFAULT_TEMPERATURE) -> float:
```

```
    """
```

```
    This function calculates the speed of the sound in the air at a given temperature.
```

```
    :param temperature: The temperature in °C.
```

```
    :return: The speed of sound in the air in m.s-1
```

```
    """
```

```
    return 331.5 + 0.607 * temperature
```

```
def define_air_density(temperature: float = DEFAULT_TEMPERATURE) -> float:
```

```
    return 1.292 * (273 / (temperature + 273.15))
```

```
def distance_gain(distance: float | np.ndarray,
```

```
                 source_radius: float = SOURCE_RADIUS, array=False) -> float:
```

```
    """
```

```
    This function calculates the change of the audio gain at a specific distance of the source.
```

```
    :param source_radius: Radius of the source.
```

```
    :param distance: Distance with the center of the source.
```

```
    :param array: Array mode
```

```
    :return: Gain of the new audio signal.
```

```
    """
```

```
    if array:
```

```
        res = source_radius / distance
```

```
        res[np.abs(distance) < source_radius] = 1
```

```
    return res
```

```

else:
    if np.abs(distance) < source_radius:
        return 1
    return source_radius / distance

```

```

def euclidian_distance(x0, y0, z0, x1, y1, z1) -> float | np.ndarray:
    """
    This function calculates the euclidian distance between two points.
    """
    return ((x0 - x1) ** 2 + (y0 - y1) ** 2 + (z0 - z1) ** 2) ** 0.5

```

```

class Sound:
    """
    This class represents a sound with many properties.
    """

    def __init__(self,
                  data: np.ndarray | Callable[[float], float] | str,
                  rate: int,
                  start: float = 0.0,
                  end: float = None,
                  default_value: float = 0.0,
                  max_pressure: float = 20.0
                  ) -> None:
        """
        Init the sound class.
        :param data: Sound Data, many ways to create a sound:
            - np.ndarray: Sound as a sequence of numerical values between -1 and 1.
            - str: Path where a .wav file is stored
            - Callable: Function for the sound time -> numerical values between -1 and 1.
        :param rate: The rate of the sound.
        :param start: Time in second when the sound starts.
        :param end: Time in seconds when the sound stops.
        :param default_value: Default values where the sound is undefined (time outside [start; end]).
        """

```

```

# Get the type of data given
if isinstance(data, np.ndarray) or isinstance(data, list):
    self.rate: int = rate

    self.data = np.array(data)
    self.data_type = "array"
    self.end = len(data) + floor(rate * start)

elif isinstance(data, Callable):
    self.rate: int = rate

    self.func_data = data
    self.data_type = "function"
    self.end: float = floor(end * rate) if end is not None else None

elif isinstance(data, str):
    r, d = wavfile.read(data)
    self.rate = r
    if len(d.shape) > 1:
        self.data = d[0:, 1] / 32768.0
        self.end = len(self.data) + floor(rate * start)
    else:
        self.data = d / 32768.0
        self.end = len(self.data) + floor(rate * start)

    self.data_type = "array"

self.start: int = floor(self.rate * start)
self.default_value: float = default_value

self.max_pressure: float = max_pressure

def __getitem__(self, item: int | slice | np.ndarray) -> float | np.ndarray:
    """
    Get the numerical value of the sound at a given index (in 1/rate seconde).
    :param item: Time (in 1/rate seconde), many ways to get value at a specific time:
        - int: Gives the value of sound at one specific index
        - slice: Gives the value of sound at many indexes between start and end, and with step.
        - np.ndarray: Give the value of sound at many specific indexes.
    """

```

```

: return: Numerical value(s) of sound at given index.
"""
# Extract properties of the slice (if item is a slice)
if isinstance(item, slice):
    start = item.start
    stop = item.stop
    step = item.step if not self.data_type == "function" else 1

# Case of an array
elif isinstance(item, np.ndarray):
    if self.data_type == "array":
        return self.data[item]
    else:
        return self.func_data(item / self.rate)

# Default case
else:
    stop = item
    start = 0
    step = 1

# When item is an int or a slice.
if self.data_type == "array":
    # Only one value
    if start is None:
        if self.start <= stop < self.end:
            return self.data[stop]
        else:
            return self.default_value

    # Multiples values
    else:
        return np.concatenate((
            np.ones((min(start, self.start),)) * self.default_value,
            self.data[start:stop:step],
            np.ones((max(0, stop - self.end, ))) * self.default_value
        ))

elif self.data_type == "function":

```



```

# Only one value
if start is None:
    if self.start <= stop < self.end:
        return self.func_data(stop / self.rate)
    else:
        return self.default_value
# Multiples values
else:
    x = np.arange(start, stop, step)
    x = x / self.rate
    x[x < self.start / self.rate] = np.nan
    x[x >= self.end / self.rate] = np.nan

    x[x != np.nan] = self.func_data(x[x != np.nan])
    x[np.isnan(x)] = self.default_value

    return x
else:
    raise Exception("Unknown data_type '%s'" % self.data_type)

```

```

class Source:

```

```

    """

```

```

    This class represents a sound source with many properties.

```

```

    """

```

```

def __init__(self, sound: Sound, name: str, x: float, y: float, z: float = 0, radius: float = 0.1) -> None:

```

```

    """

```

```

    Init the sound class.

```

```

    :param sound: The sound that the source has to play.

```

```

    :param name: The name of the source.

```

```

    :param x: The x coordinate of the source.

```

```

    :param y: The y coordinate of the source.

```

```

    :param z: The z coordinate of the source.

```

```

    :param radius: Radius of the source in m.

```

```

    """

```

```

    self.name: str = name

```

```

    # En mètres

```

```

    self.x: float = x

```

```
self.y: float = y
self.z: float = z

self.sound: Sound = sound

self.radius: float = radius
```

```
@property
```

```
def get_rate(self) -> float:
    return self.sound.rate
```

```
@property
```

```
def get_coord(self) -> tuple[float, float, float]:
    return self.x, self.y, self.z
```

```
@property
```

```
def get_radius(self) -> float:
    return self.radius
```

```
@property
```

```
def get_recording_pressure(self) -> float:
    return self.sound.max_pressure
```

```
class Microphone:
```

```
    """
```

```
    This class represents a microphone with many properties.
```

```
    """
```

```
def __init__(self, name: str, x: float, y: float, z: float = 0, max_pressure: float = 20.0) -> None:
    """
```

```
    Init the microphone class.
```

```
    :param name: Name of the microphone.
```

```
    :param x: The x coordinate of the microphone.
```

```
    :param y: The y coordinate of the microphone.
```

```
    :param z: The z coordinate of the microphone.
```

```
    """
```

```
    self.name: str = name
```

```

# En mètres
self.x: float = x
self.y: float = y
self.z: float = z

self.max_pressure: float = max_pressure

```

```

@property
def get_coord(self) -> tuple[float, float, float] | tuple[float, float]:
    return self.x, self.y, self.z

```

```

@property
def get_max_pressure(self):
    return self.max_pressure

```

```

class MicrophoneTensor:

```

```

    """

```

```

    This class represents a tensor/array of microphones.

```

```

    It is used to observe sound across a lots of microphones (for example, to see the sound propagation in a region)

```

```

    """

```

```

def __init__(self, name: str, ax_x: np.ndarray, ax_y: np.ndarray, ax_z: np.ndarray,
             max_pressure: float = 20.0) -> None:

```

```

    """

```

```

    Init the microphone tensor class.

```

```

    :param name: Name of the Tensor.

```

```

    :param ax_x: Many x coordinates for microphones on the tensor.

```

```

    :param ax_y: Many y coordinates for microphones on the tensor.

```

```

    :param ax_z: Many z coordinates for microphones on the tensor.

```

```

    """

```

```

    self.name: str = name

```

```

    self.ax_x: np.ndarray = ax_x

```

```

    self.ax_y: np.ndarray = ax_y

```

```

    self.ax_z: np.ndarray = ax_z

```

```

    self.mesh_x, self.mesh_y, self.mesh_z = np.meshgrid(self.ax_x, self.ax_y, self.ax_z)

```

```
self.shape: tuple = (len(ax_x), len(ax_y), len(ax_z))

self.max_pressure: float = max_pressure
```

```
@property
```

```
def get_max_pressure(self):
    return self.max_pressure
```

```
class Environment:
```

```
    """
```

```
    This class creates an audio environment to simulate sound propagation.
    It can handle many sources, microphones and microphones tensors to create a realistic simulation of sound.
    """
```

```
def __init__(self,
              sources: list[Source],
              microphones: list[Microphone],
              rate: int | None,
              microphone_tensors: list[MicrophoneTensor] = None,
              temperature: float = 20.0,
              ) -> None:
    """
```

```
    Init the environment.
```

```
    :param sources: List of sources to add in the environment.
```

```
    :param microphones: List of microphones to add in the environment.
```

```
    :param rate: The rate of all sounds in the environment. (All sources must have the same rate)
```

```
    :param microphone_tensors: List of microphone tensors.
```

```
    :param temperature: The temperature of the iar in the environment.
```

```
    """
```

```
    assert len({s.get_rate for s in sources}) == 1, "Sources must have the same rate"
```

```
    assert "all" not in [s.name for s in sources], "Source name can't be all"
```

```
    self.sources: dict[str, Source] = {s.name: s for s in sources}
```

```
    self.microphones: dict[str, Microphone] = {m.name: m for m in microphones}
```

```
    if microphone_tensors is not None:
```

```
        self.microphone_tensors: dict[str, MicrophoneTensor] = {m.name: m for m in microphone_tensors}
```

```
    else:
```

```
        self.microphone_tensors = dict()
```

```
self.rate: int = sources[0].get_rate if rate is None else rate
self.step: float = 1 / self.rate
```

```
# In Celsius
```

```
self.temperature: float = temperature
self.sound_speed: float = define_sound_speed(temperature)
```

```
@property
```

```
def get_rate(self) -> int:
    return self.rate
```

```
def applies_time_shift(self, sound: np.ndarray, time_shift: float) -> np.ndarray:
    """
```

```
    This functions applies a timeshift to a sound.
```

```
    Sound is completed with 0.
```

```
    :param sound: Sound as array
```

```
    :param time_shift: The timeshift to add to the sound in seconds.
```

```
    :return: The shifted sound.
```

```
    """
```

```
    shift = floor(time_shift * self.rate)
```

```
    new_sound = np.roll(sound, shift)
```

```
    new_sound[:shift] = 0
```

```
    return new_sound
```

```
def listen_specific_sound(self, name: str, from_time: float, to_time: float) -> np.ndarray:
    """
```

```
    This function is used to listen to only one specific source.
```

```
    :param name: Name of the source to listen to.
```

```
    :param from_time: Time in seconde to start to listen.
```

```
    :param to_time: Time in seconde to stop to listen.
```

```
    :return: The sound during the [from_time; stop_time] period of the 'name' source.
```

```
    """
```

```
    indice_from: int = floor(self.rate * from_time)
```

```
    indice_to: int = floor(self.rate * to_time)
```

```
    return self.sources.get(name).sound[indice_from:indice_to]
```

```
@staticmethod
```

```

def audio_compressor(x: float | np.ndarray, alpha: float = COMPRESS_ALPHA) -> np.ndarray | float:
    """
    This function compresses a numerical audio signal to fit between -1 and 1 with the fewer losses of
    information as
    possible.
    :param x: Numerical audio signal.
    :param alpha: Compression level of amplitude higher than 1 (1: Very compressed, 0: Not compressed).
    :return: Compressed signal.
    """
    y = deepcopy(x)
    y[x < -1] = (1 - alpha) * np.tanh(alpha / (1 - alpha) * x[x < -1] + alpha / (1 - alpha)) - alpha
    y[x > 1] = (1 - alpha) * np.tanh(alpha / (1 - alpha) * (x[x > 1] - 2) + alpha / (1 - alpha)) + alpha
    y[np.abs(x) <= 1] = alpha * x[np.abs(x) <= 1]

    return y

@staticmethod
def audio_noise(signal: np.ndarray, alpha: float = NOISE_ALPHA) -> np.ndarray:
    """
    Add noise to a given signal
    :param signal: A numpy array signal.
    :param alpha: Percentage of noise.
    :return: Noisy signal.
    """
    return signal + np.random.normal(scale=alpha, size=signal.shape)

def listen_from_microphone(self,
                           mic_names: str | list[str] | tuple | list[tuple],
                           from_time: float,
                           to_time: float,
                           record_start: float,
                           attenuate: bool = True,
                           specific_sound: list[str] = None,
                           exclude_sound: list[str] = None,
                           compress: bool = True,
                           noise: float = NOISE_ALPHA,
                           ) -> tuple[np.ndarray, list[np.ndarray]]:
    """
    Simulate the environment for a period of time

```

```

:param mic_names: microphone(s) or coordinate(s) where the environment is recorded.
:param from_time: Start time of recordings.
:param to_time: Stop time of recordings.
:param record_start: Real time to start simulation (must be lower than start_time).
:param attenuate: If the sound must be attenuated with distance.
:param specific_sound: To allow only specific sources.
:param exclude_sound: To remove specific sources.
:param compress: If the signal has to be compressed.
:param noise: The percentage of noise.
:return: (x, [y1, ... yn])
"""

# Manage different kind of 'microphones' types:
# If 'microphone' is a string
if isinstance(mic_names, str):
    # Record every microphone.
    if mic_names == "all":
        microphone: list[Microphone] = [s for s in self.microphones.values()]
    # Record to only one microphone.
    else:
        microphone: list[Microphone] = [self.microphones[mic_names]]
# If 'microphone' is a list
elif isinstance(mic_names, list):
    # If 'microphone' is a list of string.
    if isinstance(mic_names[0], str):
        microphone: list[Microphone] = [self.microphones[n] for n in mic_names]

    # If 'microphone' is a list of coordinate tuple
    elif isinstance(mic_names[0], tuple):
        microphone: list[Microphone] = [Microphone("temp", *c) for c in mic_names]
    else:
        raise Exception("Unsupported type for mic_names")

# If 'microphone' is a coordinate tuple
elif isinstance(mic_names, tuple):
    microphone: list[Microphone] = [Microphone("temp", *mic_names)]
else:
    raise Exception("Unsupported type for mic_names")

# Prepare time properties

```

```

time_interval = np.arange(record_start, to_time, self.step)
results = []

indice_from: int = floor(self.rate * from_time)
indice_to: int = floor(self.rate * to_time)
indice_record_start: int = floor(self.rate * record_start)

# Select specific sources needed.
if specific_sound is None:
    dict_sound = {s.name: s.sound[indice_record_start:indice_to] for s in self.sources.values()}
else:
    dict_sound = {s.name: s.sound[indice_record_start:indice_to] for s in self.sources.values() if
                  s.name in specific_sound}

# Exclude the necessary sources
if exclude_sound is not None:
    for name in exclude_sound:
        del dict_sound[name]

# For each microphone, listen to the environment.
for mic in microphone:
    mic_sound = np.zeros((indice_to - indice_record_start,))

    x, y, z = mic.x, mic.y, mic.z

    for name, sound in dict_sound.items():
        sx, sy, sz = self.sources[name].x, self.sources[name].y, self.sources[name].z
        d = euclidian_distance(x, y, z, sx, sy, sz)
        time_shift = d / self.sound_speed

        delayed_sound = self.applies_time_shift(sound, time_shift)
        delayed_sound = self.audio_noise(delayed_sound, noise)

        g = distance_gain(d, self.sources[name].get_radius) if attenuate else 1

        mic_sound = mic_sound + delayed_sound * g

    mic_sound = self.audio_compressor(mic_sound) if compress else mic_sound

    results.append(mic_sound)

```



```
results = [r[(indice_from - indice_record_start):] for r in results]
time_interval = time_interval[(indice_from - indice_record_start):]
```

```
return time_interval, results
```

```
def listen(self, mic_names: str, from_time: float, to_time: float, attenuate: bool = True) -> None:
    """
```

```
    This function is used to listen to the environment during a given period.
```

```
    :param mic_names: The name of the microphone to use to listen to the environment.
```

```
    :param from_time: Time in seconde to start to listen.
```

```
    :param to_time: Time in seconde to stop to listen.
```

```
    :param attenuate: If the sound has to be attenuated with the distance.
```

```
    :return: None
```

```
    """
```

```
_, sound = self.listen_from_microphone(mic_names, from_time, to_time,
                                       record_start=max((from_time - 1.0), 0),
                                       attenuate=attenuate)
```

```
sound = sound[0]
```

```
play_sound(sound, self.rate)
```

```
def save(self, name: str, mic_names: str, from_time: float, to_time: float, attenuate: bool = True):
    """
```

```
    Save the environment sound to a .wav file.
```

```
    :param name: Name of the file
```

```
    :param mic_names: Microphone to listen to
```

```
    :param from_time: Time in seconde to start to listen.
```

```
    :param to_time: Time in seconde to stop to listen.
```

```
    :param attenuate: If the sound has to be attenuated with the distance.
```

```
    :return: None
```

```
    """
```

```
_, sound = self.listen_from_microphone(mic_names, from_time, to_time, max((from_time - 1.0), 0),
                                       attenuate=attenuate)
```

```
sound = sound[0]
```

```
scaled = np.int16(sound / np.max(np.abs(sound)) * 32767)
```

```
write(f'{name}.wav', self.rate, scaled)
```

```

def render_frame_tenor(self,
                        tensor_name: str,
                        render_time: float,
                        attenuate: bool = True,
                        compress: bool = True
                        ) -> np.ndarray:
    """
    This function renders the microphone tensor at a given time to see the environment.
    :param tensor_name: The name of the tensor to render.
    :param render_time: The time when the tensor is rendered (in second)
    :param attenuate: If the sound has to be attenuated with the distance.
    :param compress: If the signal has to be compressed.
    :return: The render of the microphone tensor.
    """
    mic_tensor = self.microphone_tensors[tensor_name]

    result = np.zeros(mic_tensor.shape)
    tx, ty, tz = mic_tensor.mesh_x, mic_tensor.mesh_y, mic_tensor.mesh_z

    for source in self.sources.values():
        sx, sy, sz = source.x, source.y, source.z

        d = euclidian_distance(tx, ty, tz, sx, sy, sz)
        time_shift = d / self.sound_speed # s
        shift = time_shift * self.rate # sans unité

        index_tensor = np.round(render_time * self.rate) - shift # sans unité
        index_tensor = np.maximum(index_tensor, 0) # sans unité
        sound_tensor = source.sound[np.array(index_tensor, dtype=np.int64)]
        sound_tensor[index_tensor < 0] = 0

        dist_gain = distance_gain(d, source.get_radius, array=True)

        result = result + sound_tensor * (dist_gain if attenuate else 1)

    result = self.audio_compressor(result) if compress else result

    return result

```

Annexe 3

Tables des valeurs



Tables diapositive 18

Nombre sources	Erreur moyenne (m)	Incertitude (m)	Taux d'échec (%)
Méthode simple par minimisation			
1	0.0870679534286648	0.048696715142113836	4.299999999999997
2	1.0394928665628005	0.13691744015430618	16.299999999999997
3	1.9987197395833078	0.17981085821907086	18.300000000000001
4	2.363618554396053	0.17360749460413044	24.0
5	2.4791352922870376	0.17139544246923233	26.099999999999994
6	2.723345924722428	0.17968666917353676	28.200000000000003
7	2.837802770851886	0.16967109024985697	25.099999999999994
8	2.969812852006297	0.17607323270906755	25.299999999999997
9	2.903542943791962	0.1539752577022322	30.0
Méthode simple par trilatération			
10	2.8960242904822056	0.1574632663111369	33.899999999999999
1	0.011244500088377555	0.0009124834632142537	5.8000000000000011
2	0.7741892048447508	0.12914426838918036	20.0
3	1.4111766873431828	0.16486477779204997	27.299999999999997
4	1.6577447336604925	0.17657606816579285	33.099999999999994
5	1.8545199821154492	0.17605324619319715	35.5
6	2.2127413456857634	0.1992255750009038	40.300000000000004
7	2.326778753262516	0.1915854745160298	40.5
8	2.3611581942691724	0.18468214743247383	42.800000000000004
9	2.3388661466759726	0.17841321376742225	44.199999999999996
10	2.4147936017554055	0.1786574321387046	45.199999999999996

Tables diapositive 19

N.S.	Erreur moy. (m)	Incertitude (m)	Taux d'échec (%)	Erreur moyenne (m)	Incertitude (m)	Taux d'échec (%)	Erreur moyenne (m)	Incertitude (m)	Taux d'échec (%)
	Pics 2			Longueur 4			Longueur 6		
1	1.04665	0.07722	50.2	1.99209	0.11736	21.9	2.47545	0.14329	19.1
2	1.06741	0.09368	64.8	1.31508	0.07993	26.6	1.7969	0.10461	20.0
3	1.16595	0.11275	71.0	1.13473	0.06945	27.6	1.46131	0.08513	20.1
4	1.41533	0.14922	75.6	1.24734	0.07912	32.6	1.513	0.08968	22.8
5	1.53645	0.1706	78.0	1.43937	0.09429	36.8	1.49499	0.09106	26.9
6	1.78853	0.20987	80.3	1.52555	0.10415	41.8	1.59779	0.09982	30.5
7	1.66833	0.19284	79.7	1.61401	0.11222	43.9	1.76696	0.10999	30.0
8	1.61789	0.19032	80.4	1.50968	0.10497	43.9	1.65163	0.1031	30.4
9	1.86496	0.24997	84.9	1.83172	0.13546	50.4	1.72636	0.11319	36.9
10	1.82473	0.2479	85.3	1.73507	0.13113	52.5	1.75632	0.11609	37.9
	Longueur 8			Longueur 10					
1	2.9744	0.17569	22.3	3.42436	0.20546	24.7			
2	2.09013	0.12184	20.2	2.4717	0.14499	21.2			
3	1.82166	0.10593	19.8	2.29203	0.13628	23.3			
4	1.79057	0.10518	21.4	2.18939	0.13002	23.1			
5	1.82032	0.10918	24.6	2.1064	0.12744	25.9			
6	1.68908	0.1022	25.9	1.90529	0.11489	25.4			
7	1.86494	0.1143	27.8	2.00244	0.12273	27.8			
8	1.80998	0.1101	26.7	1.94853	0.11765	25.6			
9	1.89009	0.11991	32.6	1.96948	0.12331	30.8			
10	1.84699	0.1165	31.8	1.96314	0.12231	30.1			

Tables diapositive 20



N.S.	Erreur moy. (m)	Incertitude (m)	Taux d'échec (%)	Erreur moyenne (m)	Incertitude (m)	Taux d'échec (%)	Erreur moyenne (m)	Incertitude (m)	Taux d'échec (%)
	Longueur 8			Longueur 16			Longueur 32		
1	0.10357	0.00565	8.8	0.13392	0.00729	8.6	0.22737	0.01259	11.6
2	0.43848	0.02761	31.6	0.46002	0.02835	28.6	0.4569	0.02749	25.1
3	0.50296	0.03494	43.8	0.51458	0.03474	40.5	0.47879	0.03037	32.6
4	0.88268	0.0677	53.9	0.89711	0.06517	48.6	0.95432	0.06422	40.1
5	0.99626	0.08064	58.6	0.9731	0.07361	52.6	0.9016	0.06481	47.5
6	1.13332	0.09525	61.6	1.17643	0.09399	57.5	1.15866	0.08612	50.9
7	1.39659	0.11722	61.5	1.36435	0.11263	60.2	1.36326	0.10153	51.1
8	1.30244	0.11667	66.2	1.3798	0.11463	60.7	1.11101	0.08369	52.2
9	1.32249	0.12576	70.0	1.43555	0.12861	66.2	1.54945	0.1265	59.3
10	1.59797	0.15646	71.7	1.66848	0.15292	67.7	1.57766	0.12756	58.5
	Longueur 64			Longueur 128			Longueur 256		
1	0.36296	0.01991	9.9	0.50727	0.02786	10.1	0.94049	0.05162	10.0
2	0.46458	0.02622	14.9	0.52406	0.02902	11.6	0.88113	0.04861	10.9
3	0.46786	0.02709	19.1	0.63468	0.03521	11.9	0.92349	0.05126	12.0
4	0.65517	0.03911	23.9	0.71437	0.04045	15.4	0.93264	0.05207	13.0
5	0.75411	0.04648	28.6	0.72321	0.04127	16.7	0.95851	0.0538	13.9
6	0.88665	0.05584	31.6	0.72967	0.04287	21.4	0.98375	0.05567	15.3
7	1.08156	0.06944	34.2	0.9913	0.05838	21.8	1.14038	0.06595	18.9
8	0.98011	0.0639	36.2	0.92356	0.05536	24.5	1.2154	0.07011	18.5
9	1.13818	0.07845	42.9	1.02647	0.06327	28.6	1.30586	0.07751	23.0
10	1.22982	0.08646	45.1	1.16771	0.07387	32.2	1.23332	0.07364	23.9

Tables diapositive 21



Nombre sources	Erreur moyenne	Incertitude
En mètres		
1	0.5072688664033941	0.0361373487926279
2	0.5240606481034953	0.035733104765214484
3	0.6346802192340849	0.04531124058303979
4	0.714372903771716	0.05208544084152354
5	0.7232104627799812	0.04955119426765893
6	0.7296737661258397	0.04673807345806431
7	0.9912959200152746	0.0635693484917258
8	0.9235649958697623	0.05126998147659938
9	1.0264651748176108	0.05985456674031849
Normalisées (%)		
10	4.294700427461316	0.3163759037703368
1	4.063588514650077	0.2882700207457614
2	4.836430458441907	0.36454018914380587
3	5.845844640445725	0.425609139814762
4	5.450682657907943	0.3758129958145026
5	6.199156107210398	0.4630074780961004
6	8.213265816343402	0.5443641577411346
7	7.115416074076768	0.3865863864441079
8	8.162039384412262	0.5538996376655518
9	9.45410090809582	0.6200925904370667
10	4.294700427461316	0.3163759037703368

Tables diapositive 21

	S	S8	S16	S32	S64	S128	S256	P2	P4	P6	P8	P10
S	1.0	0.37654	0.39266	0.35962	0.30706	0.23841	0.19064	0.60463	0.27587	0.2003	0.12967	0.08874
s8	0.37654	1.0	0.71815	0.61213	0.51583	0.4541	0.37998	0.39384	0.18974	0.13749	0.06144	0.04898
S16	0.39266	0.71815	1.0	0.64727	0.48197	0.4238	0.30417	0.40144	0.16742	0.11409	0.05636	0.04259
S32	0.35962	0.61213	0.64727	1.0	0.55808	0.45206	0.3768	0.44467	0.23563	0.18845	0.12876	0.11858
S64	0.30706	0.51583	0.48197	0.55808	1.0	0.53304	0.39394	0.41625	0.27759	0.2261	0.16493	0.11802
S128	0.23841	0.4541	0.4238	0.45206	0.53304	1.0	0.54979	0.3375	0.22384	0.20325	0.15253	0.10679
S256	0.19064	0.37998	0.30417	0.3768	0.39394	0.54979	1.0	0.29649	0.22254	0.19949	0.15654	0.12943
P2	0.60463	0.39384	0.40144	0.44467	0.41625	0.3375	0.29649	1.0	0.68554	0.57736	0.43828	0.34203
P4	0.27587	0.18974	0.16742	0.23563	0.27759	0.22384	0.22254	0.68554	1.0	0.65937	0.53639	0.44152
P6	0.2003	0.13749	0.11409	0.18845	0.2261	0.20325	0.19949	0.57736	0.65937	1.0	0.73734	0.60816
P8	0.12967	0.06144	0.05636	0.12876	0.16493	0.15253	0.15654	0.43828	0.53639	0.73734	1.0	0.78039
P10	0.08874	0.04898	0.04259	0.11858	0.11802	0.10679	0.12943	0.34203	0.44152	0.60816	0.78039	1.0

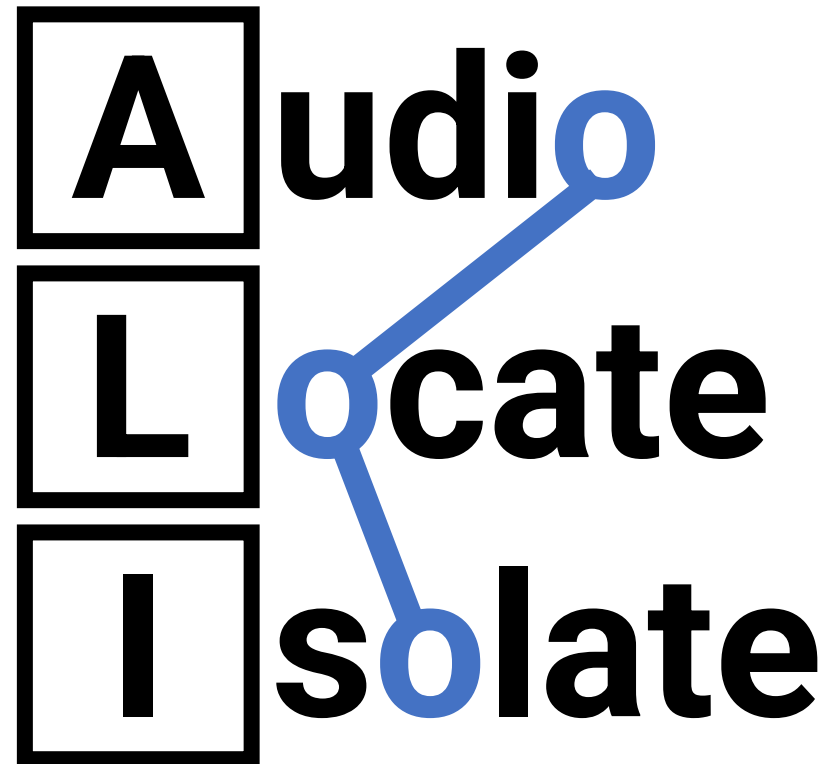
Tables diapositive 22

Cand	Min	Q1	Med.	Moy.	Q3	Max	Std	Err%	Min	Q1	Med.	Moy.	Q3	Max	Std	Err%
	1 Sources								2 Sources							
1	0.008	0.197	0.366	1.026	20.634	0.799	2.011	12.6	0.006	0.187	0.334	0.913	14.574	0.796	1.682	13.8
2	0.01	0.225	0.585	1.476	18.793	1.574	2.363	30.5	0.008	0.239	0.53	1.369	17.429	1.477	2.137	34.8
3	0.006	0.237	0.593	1.262	18.504	1.326	2.022	57.0	0.005	0.295	0.79	1.596	14.146	2.032	2.075	64.2
4	0.005	0.277	0.652	1.286	14.563	1.491	1.836	79.6	0.026	0.33	0.869	1.557	9.006	1.99	1.89	84.3
5	0.011	0.224	0.559	0.898	4.113	1.224	0.883	94.1	0.054	0.29	0.997	1.707	8.157	2.209	1.995	95.8
	3 Sources								4 Sources							
1	0.006	0.184	0.348	0.926	14.541	0.849	1.63	16.1	0.018	0.178	0.346	1.128	18.706	0.991	2.009	19.1
2	0.003	0.232	0.571	1.315	14.667	1.404	1.994	37.4	0.007	0.224	0.569	1.439	12.447	1.826	2.009	40.8
3	0.012	0.324	0.753	1.477	14.651	2.016	1.825	66.2	0.019	0.306	0.795	1.652	12.191	2.297	2.026	67.4
4	0.013	0.38	1.083	1.645	7.389	2.224	1.656	89.4	0.022	0.356	1.173	1.912	14.074	2.407	2.254	90.1
5	0.02	0.489	0.783	1.509	5.59	1.97	1.542	97.8	0.03	0.362	1.019	1.263	4.738	1.694	1.172	98.0
	5 Sources								6 Sources							
1	0.013	0.186	0.373	1.075	12.305	1.081	1.719	20.6	0.007	0.18	0.402	1.098	12.147	1.251	1.636	24.2
2	0.005	0.242	0.571	1.369	15.501	1.535	2.033	42.2	0.006	0.258	0.678	1.392	15.959	1.606	1.894	48.5
3	0.023	0.288	0.732	1.602	18.901	2.036	2.227	72.2	0.01	0.366	0.984	1.836	12.641	2.73	2.121	75.5
4	0.057	0.362	1.152	1.689	7.817	2.316	1.825	92.7	0.109	0.411	1.129	2.006	11.5	2.975	2.206	93.0
5	0.213	0.692	0.868	1.442	3.827	2.333	1.074	98.7	0.167	0.382	0.623	1.947	7.586	2.7	2.336	98.7
	7 Sources								8 Sources							
1	0.013	0.186	0.373	1.075	12.305	1.081	1.719	20.6	0.007	0.18	0.402	1.098	12.147	1.251	1.636	24.2
2	0.005	0.242	0.571	1.369	15.501	1.535	2.033	42.2	0.006	0.258	0.678	1.392	15.959	1.606	1.894	48.5
3	0.023	0.288	0.732	1.602	18.901	2.036	2.227	72.2	0.01	0.366	0.984	1.836	12.641	2.73	2.121	75.5
4	0.057	0.362	1.152	1.689	7.817	2.316	1.825	92.7	0.109	0.411	1.129	2.006	11.5	2.975	2.206	93.0
5	0.213	0.692	0.868	1.442	3.827	2.333	1.074	98.7	0.167	0.382	0.623	1.947	7.586	2.7	2.336	98.7
	9 Sources								10 Sources							
1	0.013	0.186	0.373	1.075	12.305	1.081	1.719	20.6	0.007	0.18	0.402	1.098	12.147	1.251	1.636	24.2
2	0.005	0.242	0.571	1.369	15.501	1.535	2.033	42.2	0.006	0.258	0.678	1.392	15.959	1.606	1.894	48.5
3	0.023	0.288	0.732	1.602	18.901	2.036	2.227	72.2	0.01	0.366	0.984	1.836	12.641	2.73	2.121	75.5
4	0.057	0.362	1.152	1.689	7.817	2.316	1.825	92.7	0.109	0.411	1.129	2.006	11.5	2.975	2.206	93.0
5	0.213	0.692	0.868	1.442	3.827	2.333	1.074	98.7	0.167	0.382	0.623	1.947	7.586	2.7	2.336	98.7

Tables diapositive 32 & 33

NS	Min	Q1	Med.	Moy.	Q3	Max	Std	Err%	Min	Q1	Med.	Moy.	Q3	Max	Std	Err%
	NMF Erreur								NMF Corrélation							
1	0.072	0.332	0.437	0.476	1.0	0.593	0.187	0.072	0.072	0.332	0.437	0.476	1.0	0.593	0.187	0.072
2	0.001	0.135	0.258	0.276	0.936	0.394	0.175	0.001	0.001	0.135	0.258	0.276	0.936	0.394	0.175	0.001
3	0.0	0.082	0.182	0.201	0.774	0.292	0.145	0.0	0.0	0.082	0.182	0.201	0.774	0.292	0.145	0.0
5	0.0	0.044	0.102	0.128	0.612	0.187	0.105	0.0	0.0	0.044	0.102	0.128	0.612	0.187	0.105	0.0
7	0.0	0.039	0.081	0.103	0.801	0.146	0.088	0.0	0.0	0.039	0.081	0.103	0.801	0.146	0.088	0.0
	ICA Erreur								ICA Corrélation							
1	0.0	0.008	0.119	0.501	1.0	1.0	0.491	0.0	0.0	0.008	0.119	0.501	1.0	1.0	0.491	0.0
2	0.0	0.096	0.347	0.434	0.999	0.8	0.353	0.0	0.0	0.096	0.347	0.434	0.999	0.8	0.353	0.0
3	0.0	0.128	0.354	0.417	0.997	0.689	0.311	0.0	0.0	0.128	0.354	0.417	0.997	0.689	0.311	0.0
5	0.0	0.104	0.261	0.335	0.997	0.522	0.269	0.0	0.0	0.104	0.261	0.335	0.997	0.522	0.269	0.0
7	0.001	0.098	0.232	0.288	0.987	0.417	0.233	0.001	0.001	0.098	0.232	0.288	0.987	0.417	0.233	0.001
	PCA Erreur								PCA Corrélation							
1	0.962	1.0	1.0	1.0	1.0	1.0	0.002	0.962	0.962	1.0	1.0	1.0	1.0	1.0	0.002	0.962
2	0.007	0.484	0.739	0.681	1.0	0.916	0.261	0.007	0.007	0.484	0.739	0.681	1.0	0.916	0.261	0.007
3	0.01	0.335	0.553	0.545	0.999	0.774	0.266	0.01	0.01	0.335	0.553	0.545	0.999	0.774	0.266	0.01
5	0.003	0.209	0.361	0.389	0.977	0.548	0.227	0.003	0.003	0.209	0.361	0.389	0.977	0.548	0.227	0.003
7	0.001	0.166	0.286	0.32	0.965	0.447	0.202	0.001	0.001	0.166	0.286	0.32	0.965	0.447	0.202	0.001
	SVD Erreur								SVD Corrélation							
1	0.962	1.0	1.0	1.0	1.0	1.0	0.002	0.962	0.962	1.0	1.0	1.0	1.0	1.0	0.002	0.962
2	0.007	0.485	0.738	0.682	1.0	0.915	0.26	0.007	0.007	0.485	0.738	0.682	1.0	0.915	0.26	0.007
3	0.01	0.334	0.547	0.544	0.999	0.77	0.266	0.01	0.01	0.334	0.547	0.544	0.999	0.77	0.266	0.01
5	0.003	0.207	0.358	0.387	0.977	0.547	0.227	0.003	0.003	0.207	0.358	0.387	0.977	0.547	0.227	0.003
7	0.001	0.165	0.286	0.319	0.965	0.446	0.202	0.001	0.001	0.165	0.286	0.319	0.965	0.446	0.202	0.001
	GRU Erreur								GRU Corrélation							
1	0.708	0.937	0.967	0.953	0.999	0.984	0.044	0.708	0.708	0.937	0.967	0.953	0.999	0.984	0.044	0.708
2	0.009	0.685	0.824	0.766	0.993	0.908	0.191	0.009	0.009	0.685	0.824	0.766	0.993	0.908	0.191	0.009
3	0.003	0.537	0.719	0.66	0.979	0.83	0.218	0.003	0.003	0.537	0.719	0.66	0.979	0.83	0.218	0.003
5	0.003	0.376	0.554	0.532	0.975	0.703	0.214	0.003	0.003	0.376	0.554	0.532	0.975	0.703	0.214	0.003
7	0.004	0.308	0.472	0.461	0.937	0.606	0.199	0.004	0.004	0.308	0.472	0.461	0.937	0.606	0.199	0.004

Audio
Locate
Isolate



TIPE | Louis H. | 00000